第6章 数据库物理存储

西安交通大学计算机科学与技术学院 任雪斌

xuebinren@mail.xjtu.edu.cn

第6章 数据库物理存储

6.1 物理存储与文件

6.2 数据库索引

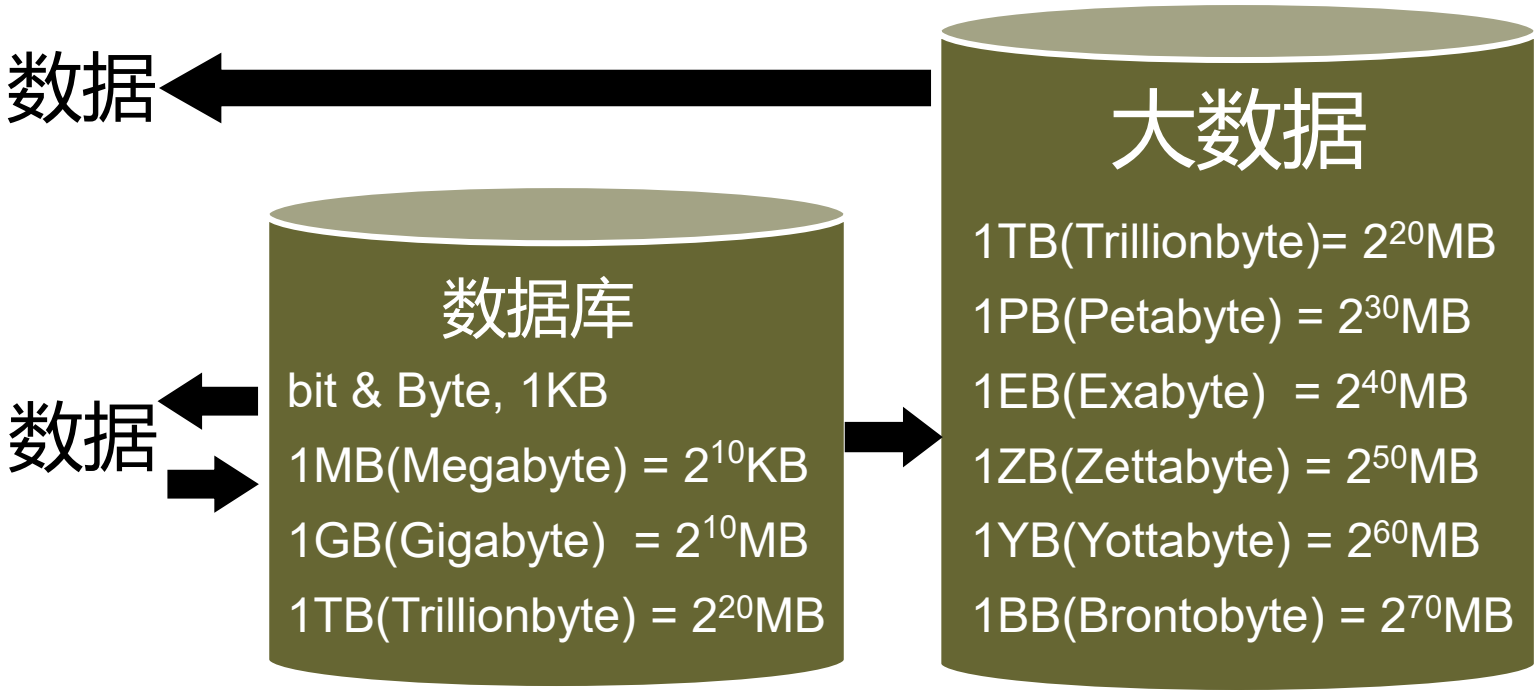
6.1 物理存储与文件

➢基本内容

1. 基础回顾-计算机系统的存储体系
2. 磁盘的结构与特性
3. DBMS数据存储与查询实现的基本思想
4. 数据库之表和记录与磁盘块的映射
5. 数据库之文件组织方法
6. openGauss物理存储简介

6.1 基础回顾-计算机系统的存储体系

➢(1) 数据库的存储与检索问题



性能评估与改进

既是DBA的职责—借助于软件管理与维护，又是计算机科学家需要研究和解决的问题

算法设计与实现

两个基本问题如何解决？

- 如何高效率的存储？--数据组织与索引
- 如何快速的检索？--查询实现与查询优化

面向大规模用户，又如何解决？

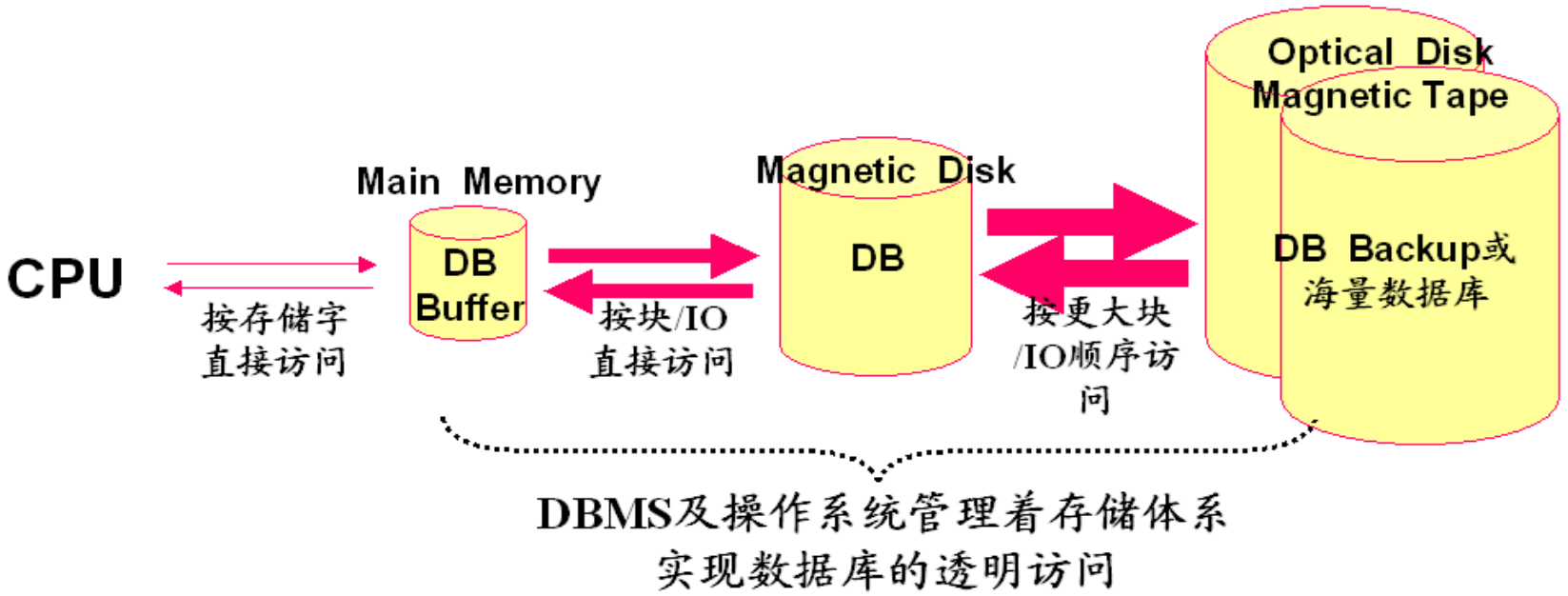
6.1 基础回顾-计算机系统的存储体系

➢(2) 什么是存储体系

数据组织的基础--存储体系：将不同性价比的存储器组织在一起，满足高速度、大容量、低价格需求。

- CPU与内存直接交换信息，按存储单元(存储字)进行访问
- 外存按存储块进行访问，其信息需先装入内存，才能被CPU处理

•磁带-数据库
•磁盘-数据库
•内存-数据库
在数据组织和实现查询方面有什么差别吗?



6.1 基础回顾-计算机系统的存储体系

➢(3) 不同层次存储的访问时间上的差异

数据组织的基础--存储体系：

➢ 高速缓冲存储器

存取速度最快的存储介质，而单位价格却比较昂贵

➢ DRAM主存(内存)

- 主要用于存放程序和其处理的数据。一般为4GB～32GB（2022年）

➢ 磁盘

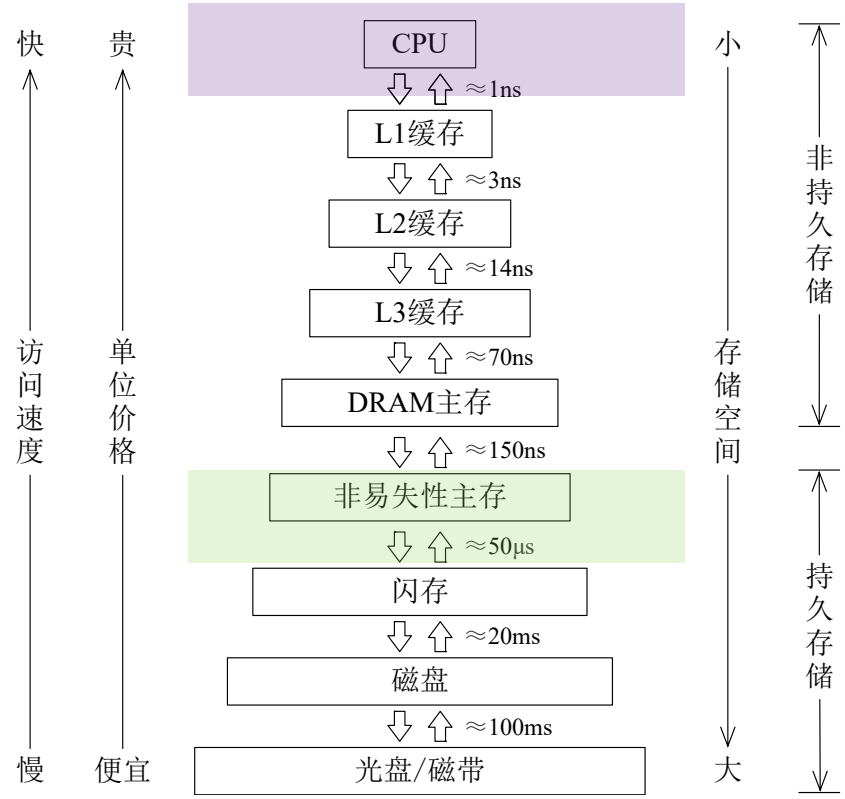
- 容量大、数据持久性好，存储长期数据。一般为256GB～10TB（2022年）

➢ 闪存

- 又叫固态硬盘，个人计算机闪存大小一般为128GB～1TB（2022年）

➢ 非易失性主存

- 速度能DRAM相媲美，字节级寻址，在断电时不会丢失数据



6.1 基础回顾-计算机系统的存储体系

➢(3) 不同层次存储的访问时间上的差异

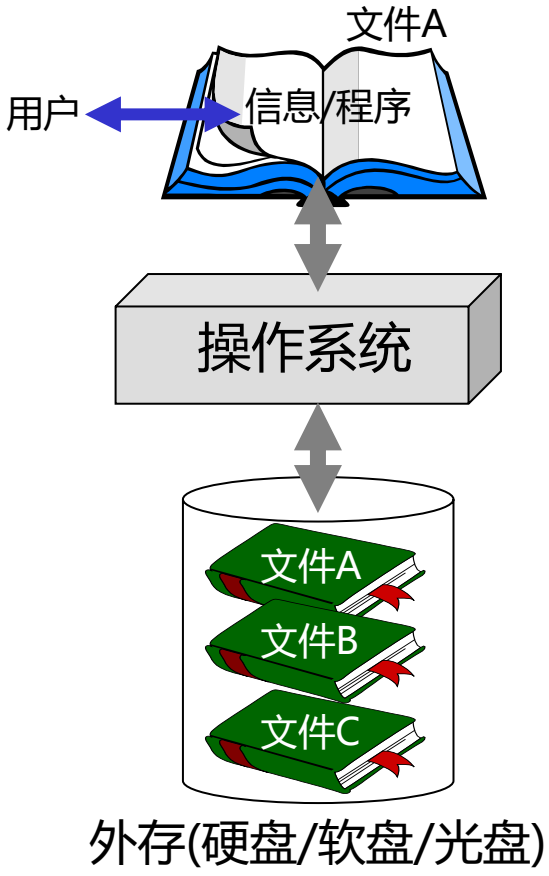
数据组织的基础--存储体系：

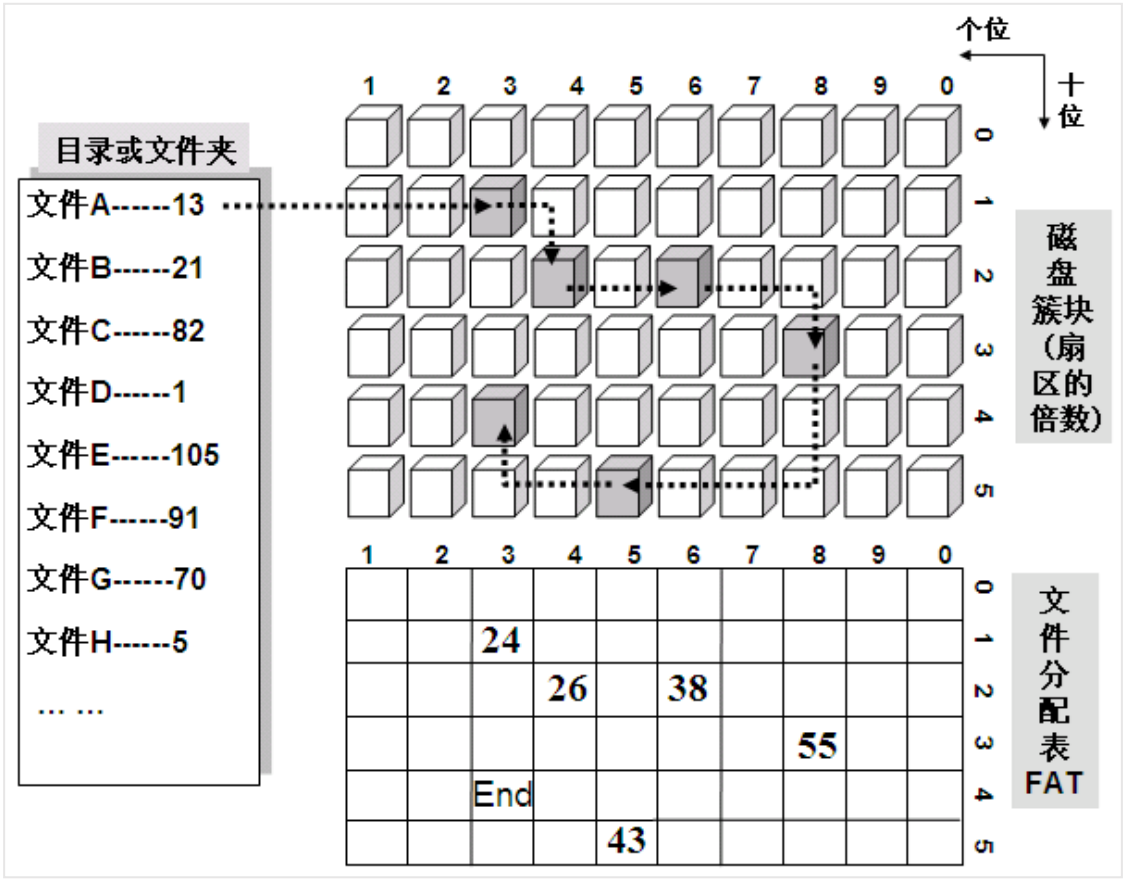
存储介质	访问速度	存储空间	单位价格	数据持久性
高速缓存	快	小	高	易失
DRAM	较快	中等	中等	易失
非易失性主存	中等	中等	较高	非易失
闪存	较慢	较大	较低	非易失
磁盘	慢	大	低	非易失

6.1 基础回顾-计算机系统的存储体系

➢(4) 操作系统如何管理磁盘和数据

操作系统对数据的组织：FAT (文件分配表-File Allocation Table) -目录(文件夹)-磁盘块/簇





6.1 基础回顾-计算机系统的存储体系

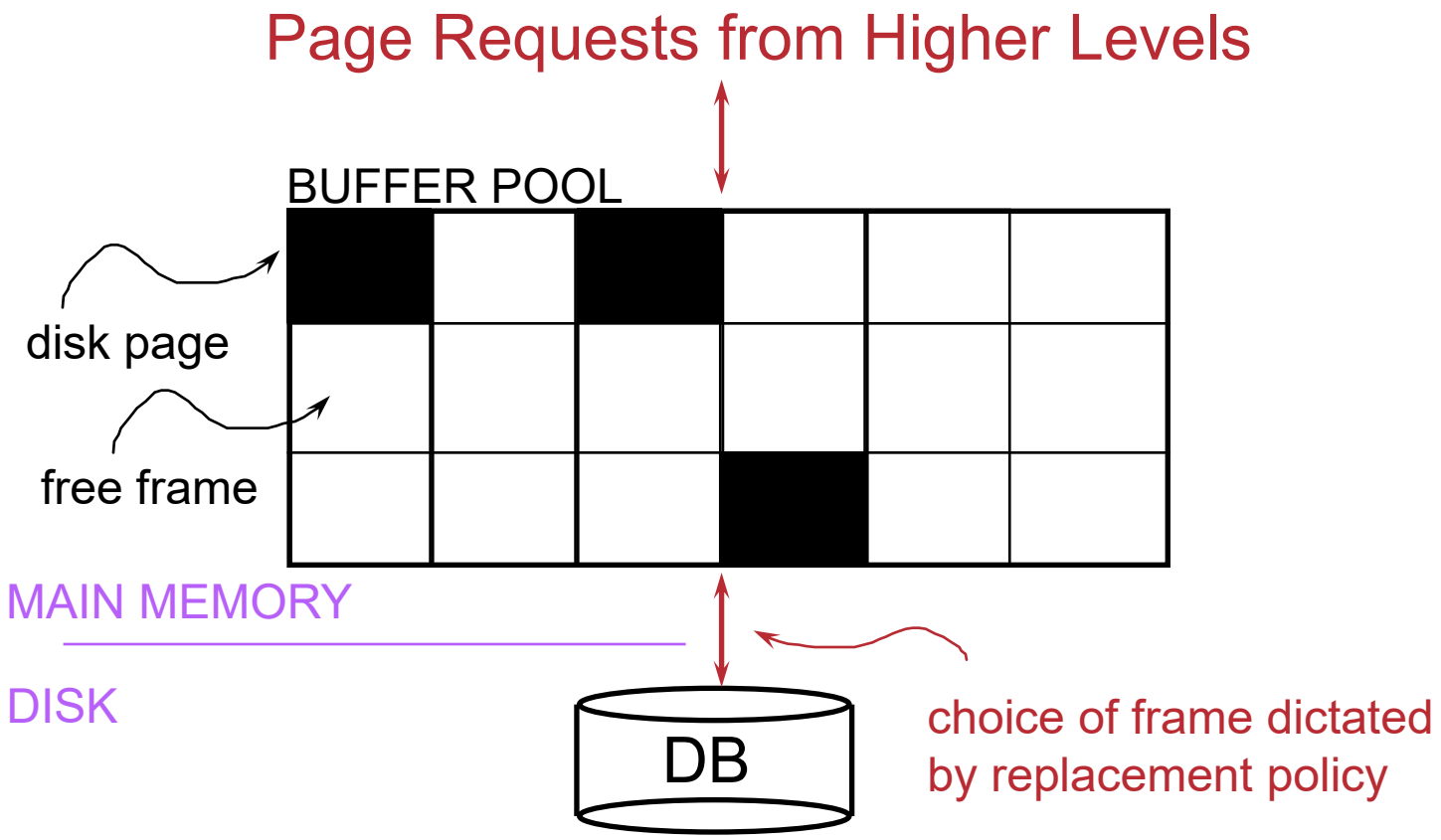
➢(5) 操作系统对内存-缓冲区的管理

内存管理

- 一条记录的地址=存储单元的地址=

内存地址=页面：页内偏移量

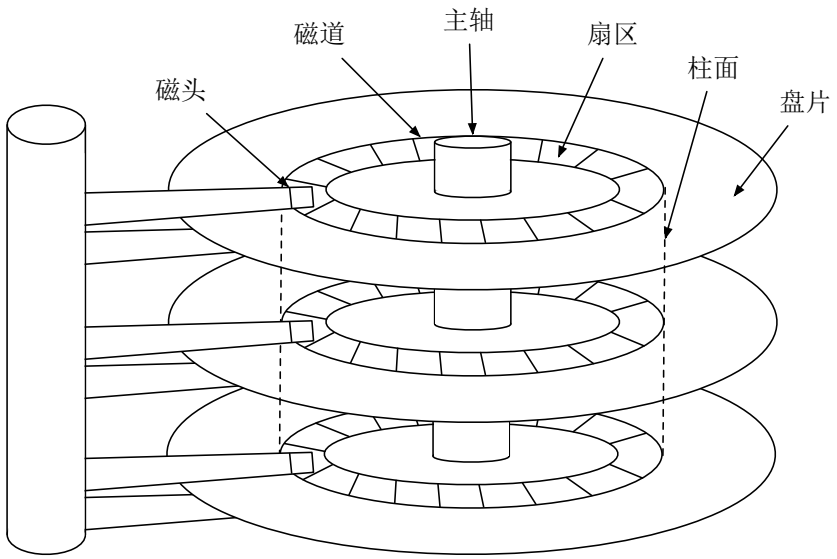
- 页面(Page) = 块(Block)
- 内存页面的分配
- 内存页面的置换



6.1 基础回顾-磁盘的结构与特性

➢(1) 磁盘及磁盘的容量

- 盘片
  - 覆盖磁性物质，通过改变磁性物质的磁场方向来存储二进制的0和1，高速旋转每分钟5400转至7200转
- 磁头
  - 改变磁性物质状态并且读写数据
- 磁道
  - 盘片上的圆环区域
- 柱面
  - 纵向对齐的多个磁道



6.1 基础回顾-磁盘的结构与特性

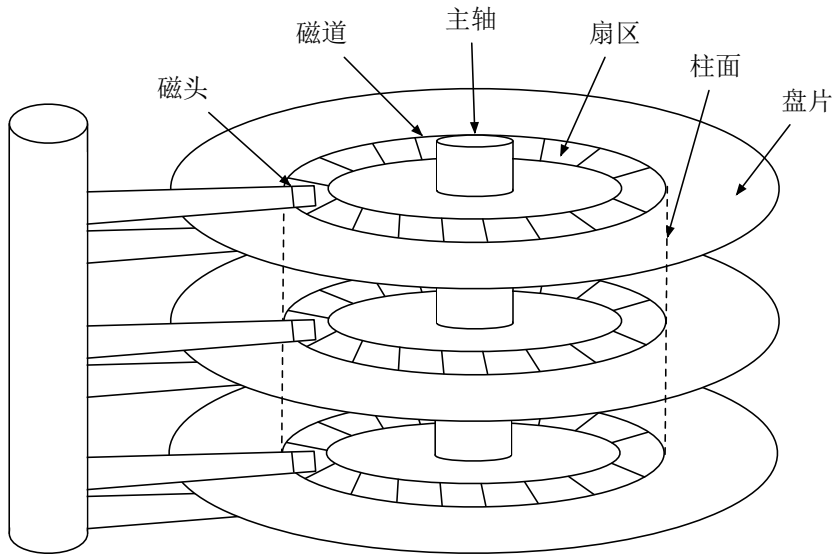
➢(1) 磁盘及磁盘的容量

盘面: 磁道: 扇区

➢磁盘读写单位：Sector ➔ 簇Cluster/块Block:若干个连续的扇区

示例：一个磁盘的基本信息
- 8个圆盘，16个盘面
- 每个盘面有2^{16}或65536个磁道
- 每个磁道(平均)有2^8=256个扇区
- 每个扇区有2^{12}=4096个字节

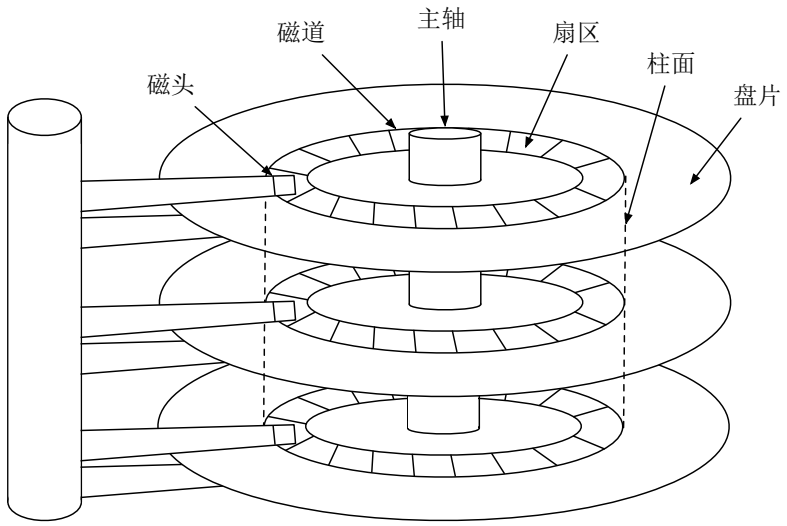
磁盘的容量=$2^4*2^{16}*2^8*2^{12}=2^{40}$字节



6.1 基础回顾-磁盘的结构与特性

➢(1) 磁盘及磁盘的容量

- **磁盘读写时延**
  - 寻道时延：磁头移动时延10ms
  - 旋转时延：盘片旋转时延
    - 5400转/分钟的磁盘平均时延是60*1000/5400*0.5=5.56毫秒
  - 数据读写时延：
    - 100MB/秒的磁盘
    - 读写4KB：4KB/100MB*1000=0.04毫秒
    - 读写1MB：1MB/100MB*1000=10毫秒
- **顺序读写：快（只有读写时延）**
- **随机读写：慢（寻道+旋转+读写）**



尽量顺序读写避免随机读写

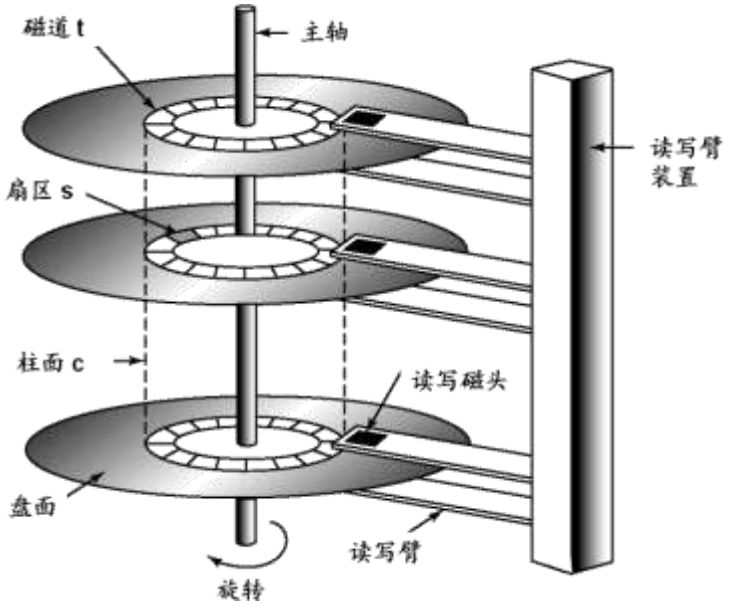
6.1 基础回顾-磁盘的结构与特性

➢(2) 磁盘数据的访问：磁盘数据读写时间

- 寻道时间(约在1-20ms)
- 旋转时间(约0-10ms)
- 传输时间(每4KB页<1ms)

示例：一个磁盘的基本信息
•磁盘以7200转/min旋转，则：8.33ms内旋转一周
•柱面之间移动磁头组合从启动到停止花费1ms,每移动4000个柱面另加1ms. 即磁头在0.00025ms内移动一个磁道，则：从最内圈移动到最外圈，移动65536个磁道大约用17.38ms.
•一个磁道中扇区间的空隙大约占10%的空间
•一个磁盘块 = 4个扇区 = 16384个字节

读一个磁盘块(=4个扇区16384个字节)：
最小时间=传输时间大约是0.13ms
最长时间 = 寻道时间+旋转时间+传输时间 = 17.38+8.33+0.13=25.84ms
平均时间 =6.36+4.17+0.13=10.76ms



物理存取算法考虑的关键:

- **降低I/O次数**
- **降低排队等待时间**
- **降低寻道/旋转延迟时间:**
  - **同一磁道连续块存储;**
  - **同一柱面不同磁道并行块存储;**
  - **多个磁盘并行块存储**

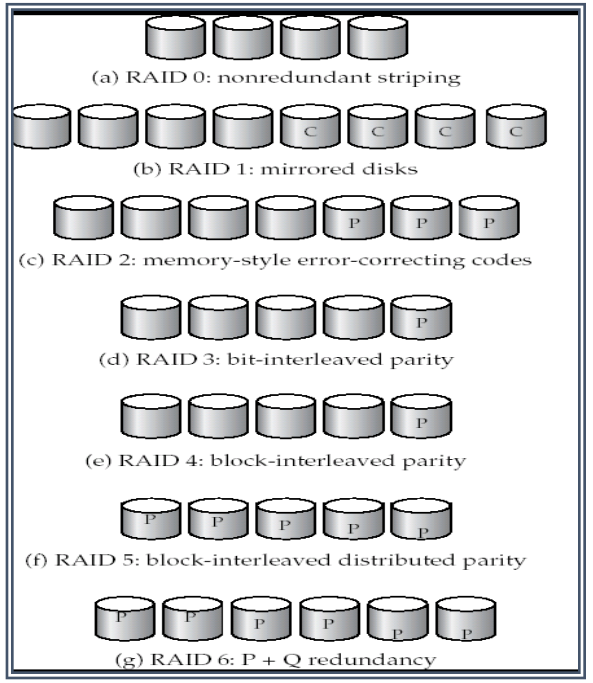
6.1 基础回顾-磁盘的结构与特性

➢(3) 提高磁盘数据读写时间与存储可靠性的方法

RAID技术: Redundant Array of Independent Disk

- 并行处理: 并行读取多个磁盘
- 可靠性: 奇偶校验与纠错

(a)块级拆分但无冗余
(b)镜像处理：每一个磁盘有一个镜像磁盘C
(c)位交叉纠错处理：4个磁盘存储4位+3个校验盘P存储3校验位
(d)位交叉校验：4个磁盘存储4位+1个校验盘存储1校验位，位拆分存储(借助于扇区读写校验判断出错磁盘，再依据校验盘进行纠错)
(e)块交叉校验：块拆分存储，其他同。
(f)块交叉分布式校验：块拆分存储，互为校验盘
(g)更为复杂的冗余处理(略)

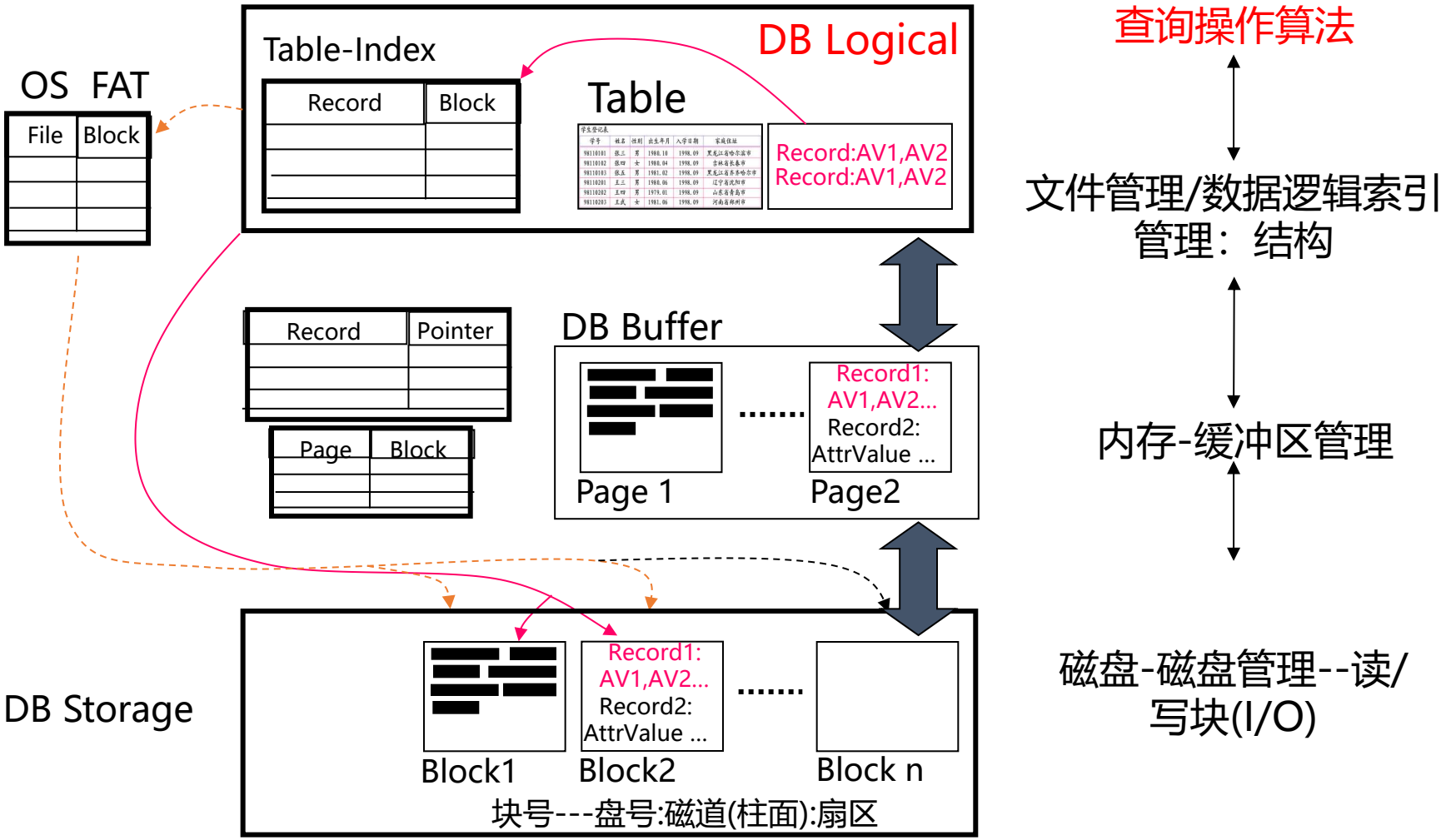


- 比特级拆分: 一个字节被拆分成8个比特位，不同比特位存储于不同磁盘。
- 块级拆分: 一个文件由多个块组成，不同块存储于不同磁盘。

- 扇区/块读写校验：对一个扇区/块读写做校验。
- 磁盘间读写校验: 多个磁盘间共同构成的信息读写做校验。

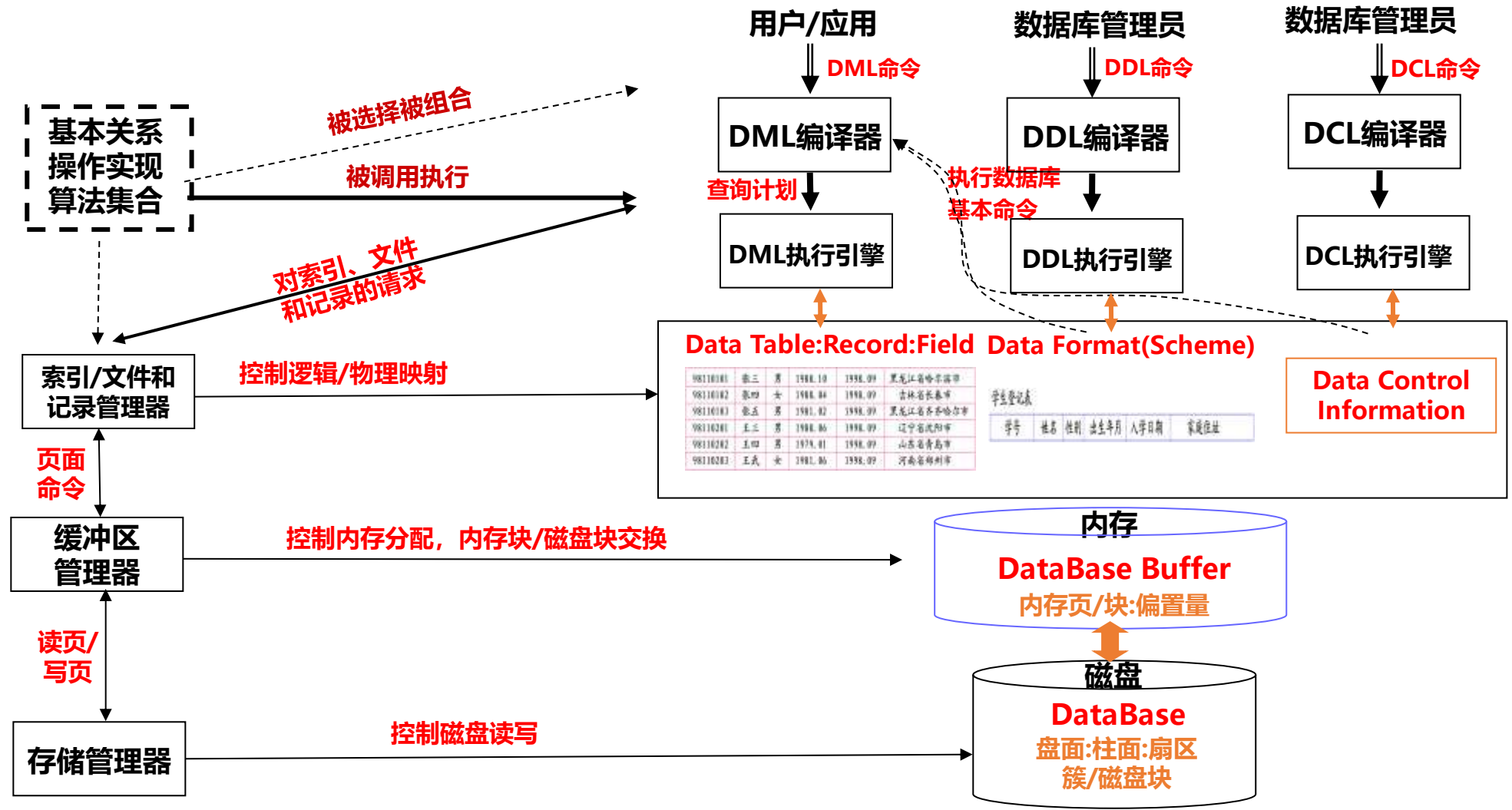
6.1 基础回顾-DBMS数据存储与查询实现基本思想

➢(1) 数据存储的映射关系示意



6.1 基础回顾-DBMS数据存储与查询实现的基本思想

➢(2) 数据存储与查询实现的基本框架示意



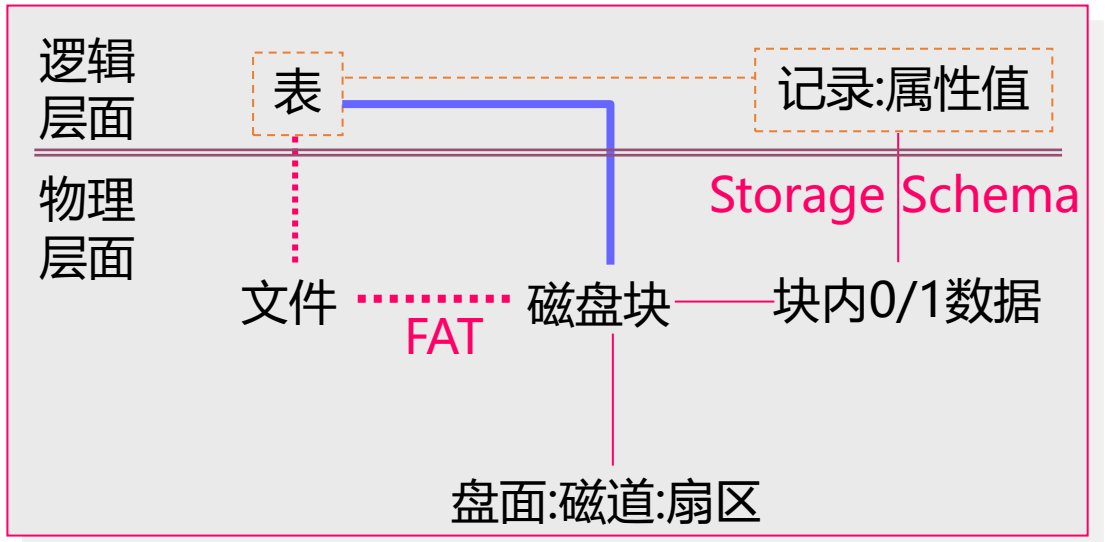
6.1 基础回顾-数据库之表-记录与磁盘块的映射

➢(1) 数据库概念与磁盘相关概念的映射示意

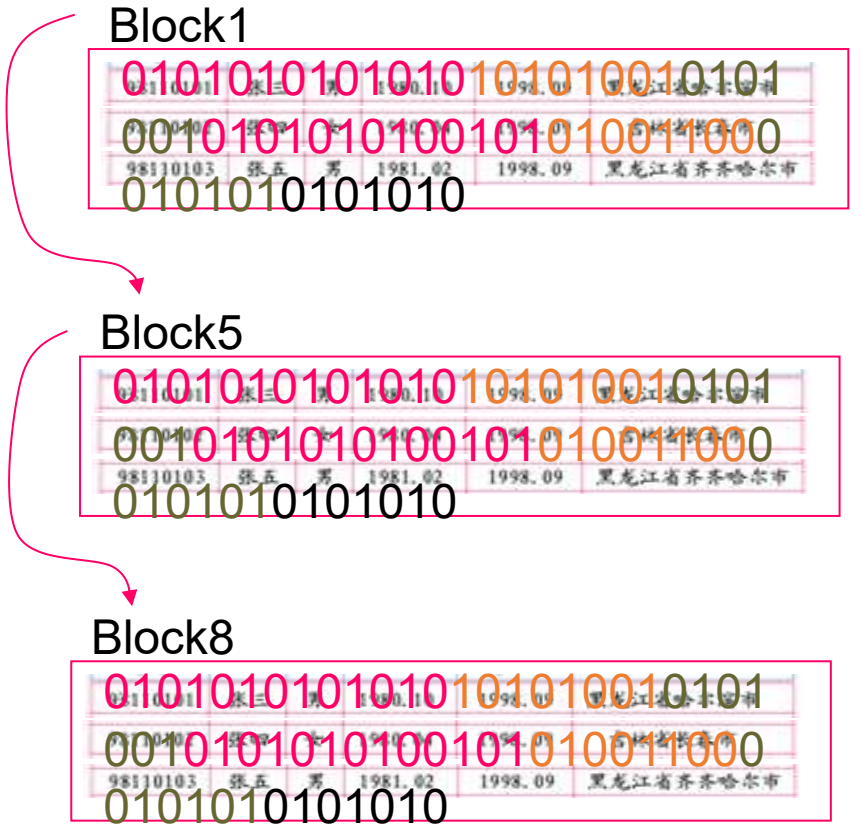
SQL—表:记录:属性值

学生登记表

学号	姓名	性别	出生年月	入学日期	家庭住址
98110101	张三	男	1980.10	1998.09	黑龙江省哈尔滨市
98110102	张四	女	1980.04	1998.09	吉林省长春市
98110103	张五	男	1981.02	1998.09	黑龙江省齐齐哈尔市
98110201	王三	男	1980.06	1998.09	辽宁省沈阳市
98110202	王四	男	1979.01	1998.09	山东省青岛市
98110203	王武	女	1981.06	1998.09	河南省郑州市



Storage—盘面:磁道:扇区

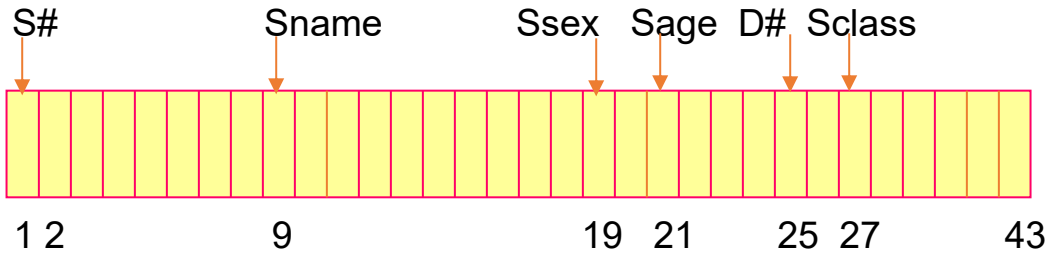


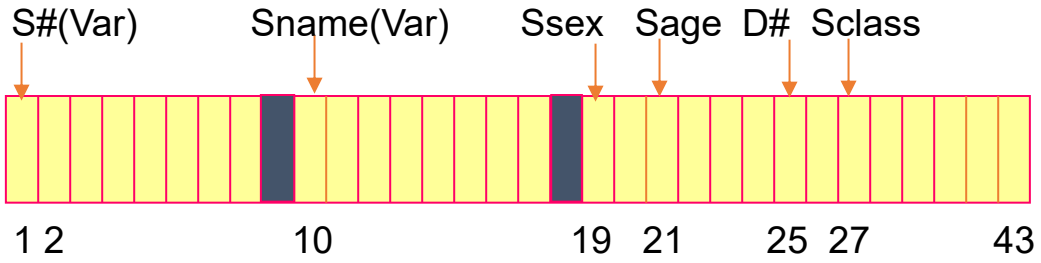
6.1 基础回顾-数据库之表-记录与磁盘块的映射

➢(2) 数据库中记录的区分及记录内属性值的区分

- 数据库记录在磁盘上的存储

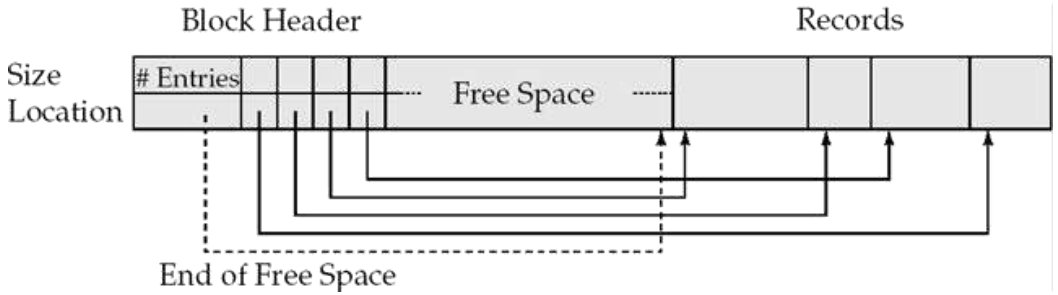
定长记录，还是变长记录(靠分隔符区分开始与结束)





•按长度区分记录
•按指针(或标志)区分记录

•块头如何设计

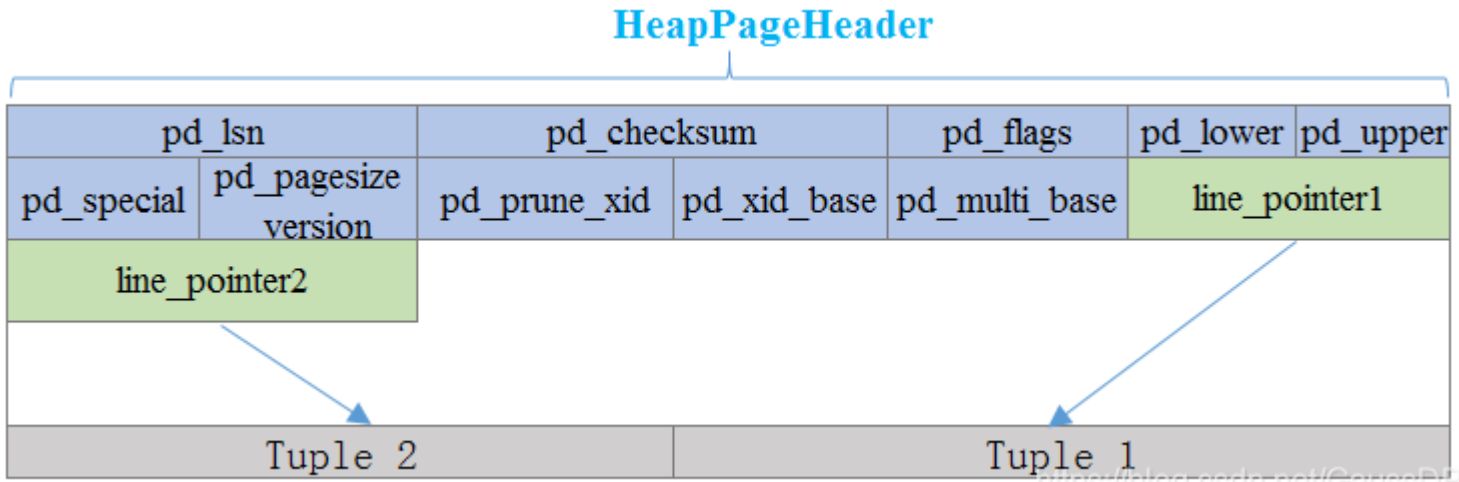


6.1 基础回顾-数据库之表-记录与磁盘块的映射

➢(2) 数据库中记录的区分及记录内属性值的区分

- 数据库记录在磁盘上的存储

定长记录，还是变长记录(靠分隔符区分开始与结束)

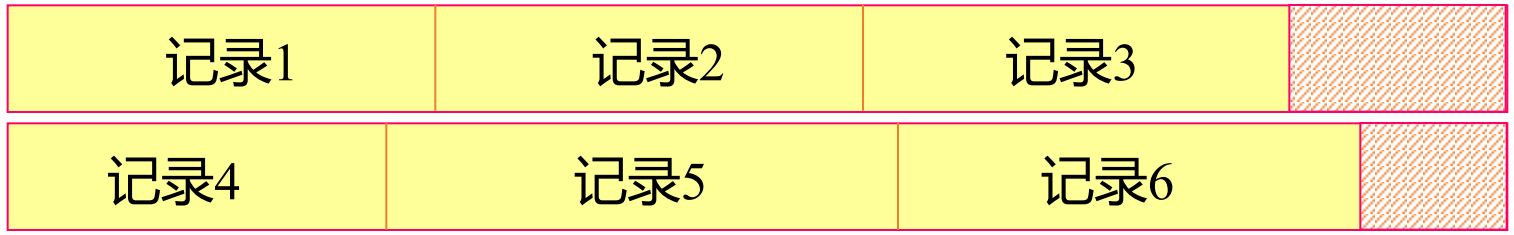


openGauss中采用的astore堆表页面结构示意图

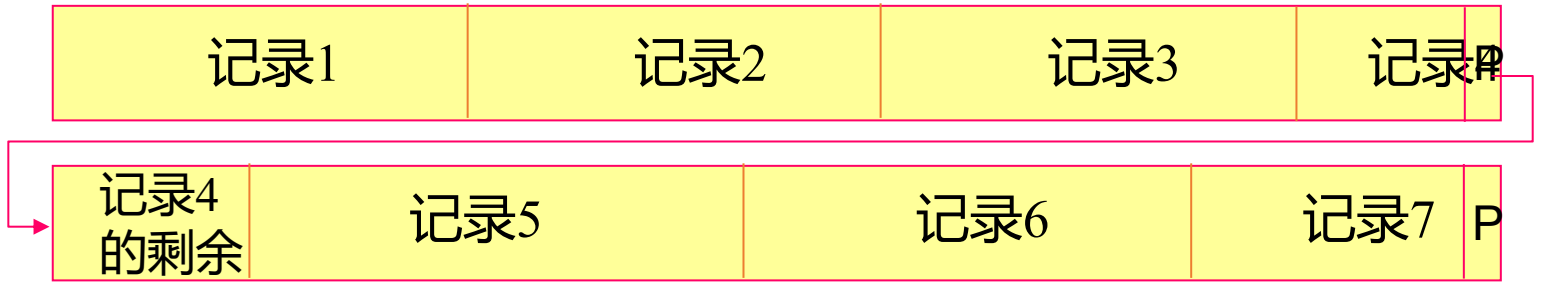
6.1 基础回顾-数据库之表-记录与磁盘块的映射

➢(3) 数据库中的记录 vs. 磁盘块

- 记录是非跨块存储，还是跨块存储(靠指针连接)



•浪费一些存储空间
•磁盘块间无关联可并行



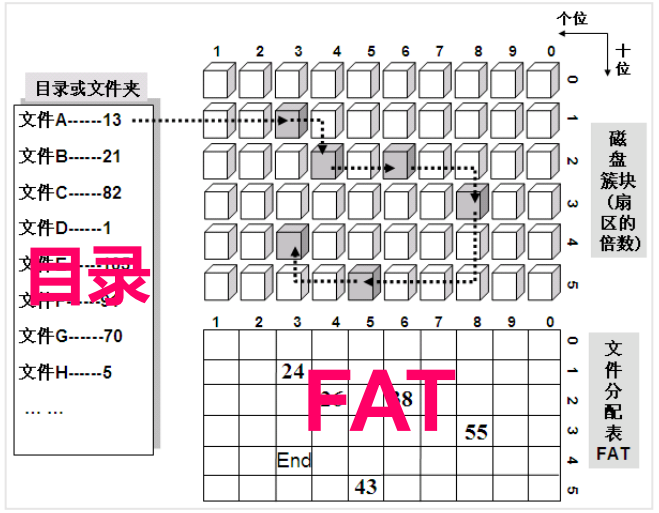
•节省一些存储空间
•磁盘块间有关联需串行

6.1 基础回顾-数据库之表-记录与磁盘块的映射

➢(4) 数据库的表 vs. 磁盘块

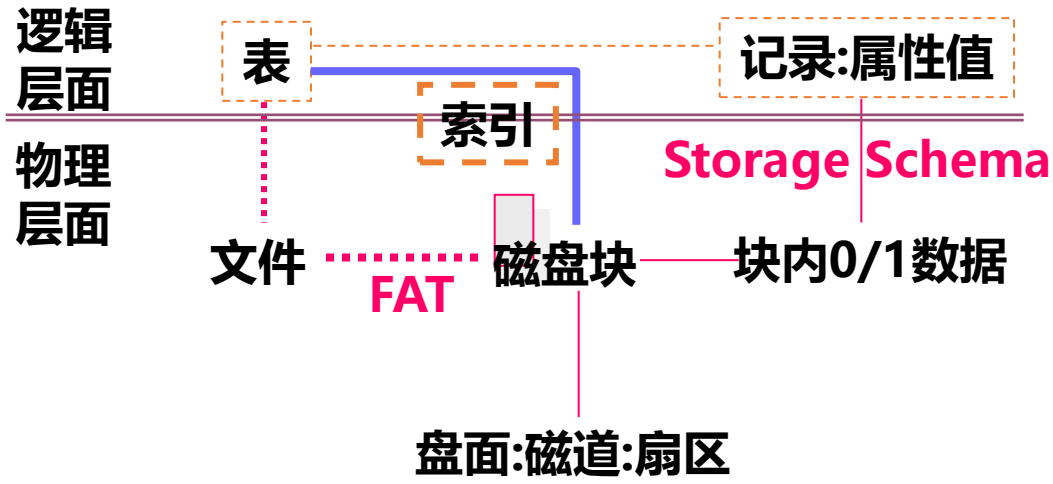
➢数据库-表所占磁盘块的分配方法

- 连续分配: 数据块被分配到连续的磁盘块上(会存在扩展困难问题)
- 链接分配: 数据块中包含指向下一数据块的指针(访问速度问题)
- 按簇分配: 按簇分配，簇是若干连续的磁盘块，簇之间靠指针连接;



➢簇有时也称片段Segment或盘区extent

- 索引分配: 索引块中存放指向实际数据块的指针



6.1 基础回顾-数据库之文件组织方法

➢ 数据组织与存取方法

➢ 数据组织要考虑更新(增、删、改)和检索需求

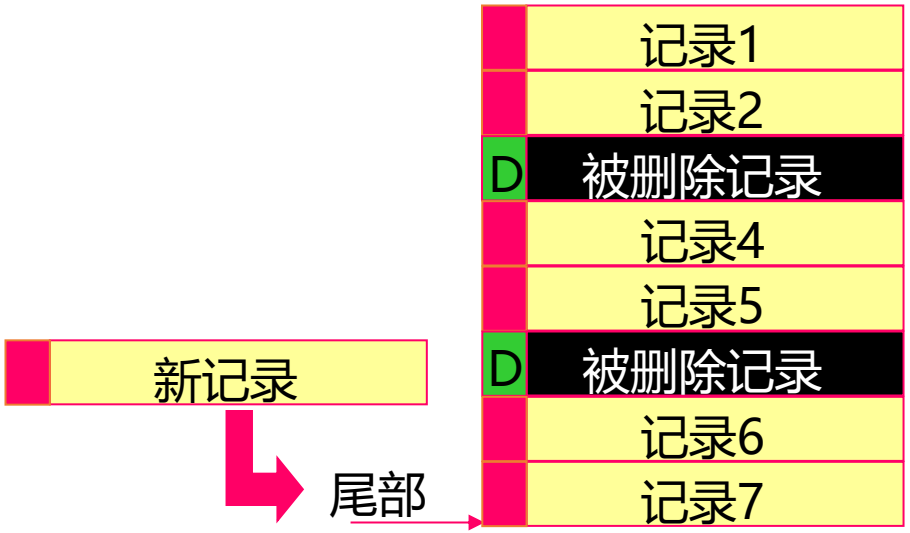
- 更新将涉及数据存储空间的扩展与回收问题
- 检索将涉及扫描整个数据库的问题、大批量处理数据问题
- 不同的需求要求不同的数据组织方法和存取方法
- 文件组织(File Organization)指的是数据组织成记录、块和访问结构的方式，包括把记录和块存储在磁盘上的方式，以及记录和块之间相互联系的方法
- 存取方法(Access Method)指的是对文件所采取的存取操作方法
- 一种文件组织可以采取多种存取方法进行访问

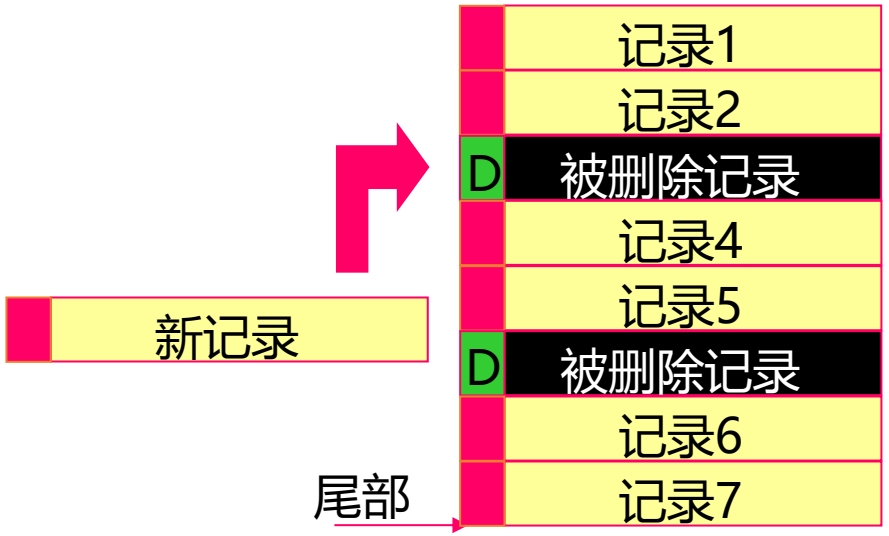
6.1 基础回顾-数据库之文件组织方法

➢(2) 无序文件组织

➢文件组织方法之一：无序记录文件(堆文件heap或pile file)

- 特点：记录可存储于任意有空间的位置，磁盘上存储的记录是无序的。更新效率高，但检索效率可能低
- 方法1：新记录总插入到文件尾部；删除记录时，可以直接删除该记录所在位置的内容，也可以在该记录前标记“删除标记”
- 方法2：在前者基础上，新增记录可以利用那些标记为“删除标记”的记录空间



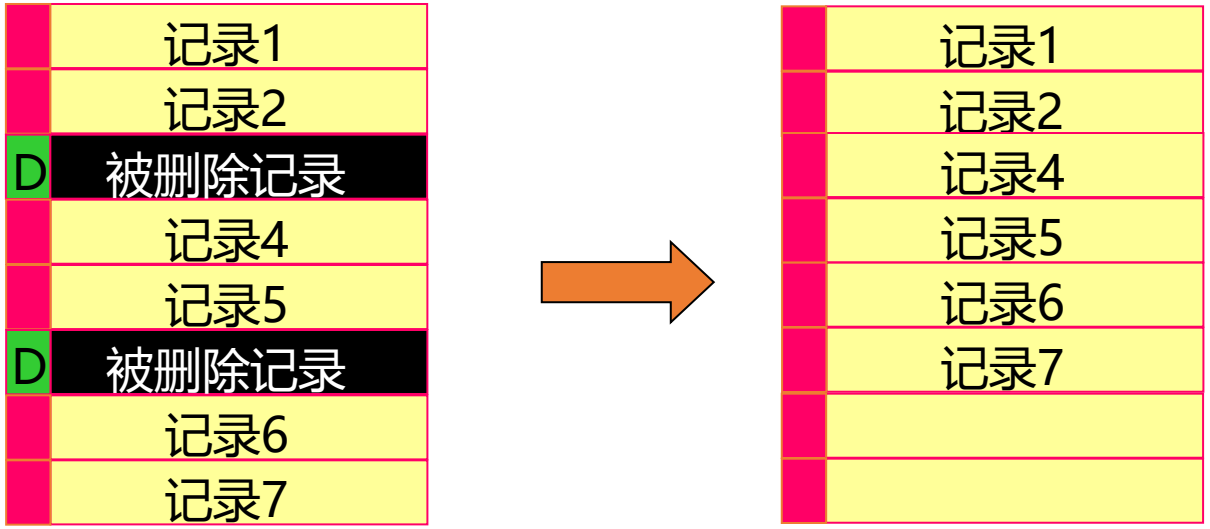


6.1 基础回顾-数据库之文件组织方法

➢(2) 无序文件组织

➢文件组织方法之一：无序记录文件(堆文件heap或pile file)

- 频繁删增记录时会造成空间浪费，所以需要周期性重新组织数据库
- 数据库重组(Reorganization)是通过移走被删除的记录使有效记录连续存放，从而回收那些由删除记录而产生的未利用空间。



6.1 基础回顾-数据库之文件组织方法

➢(3) 有序文件组织

➢文件组织方法之二：有序记录文件(排序文件Sequential)

- 特点：记录按某属性或属性组值的顺序插入，磁盘上存储的记录是有序的。检索效率可能高。
- 用于存储排序的属性通常称为排序字段(Ordering field)，通常，排序字段使用关系中的主码， 所以又称排序码(Ordering key) 。
- 当按排序字段进行检索时，速度得到很大提高；但当按非排序字段检索时，速度可能不会提高很多 。

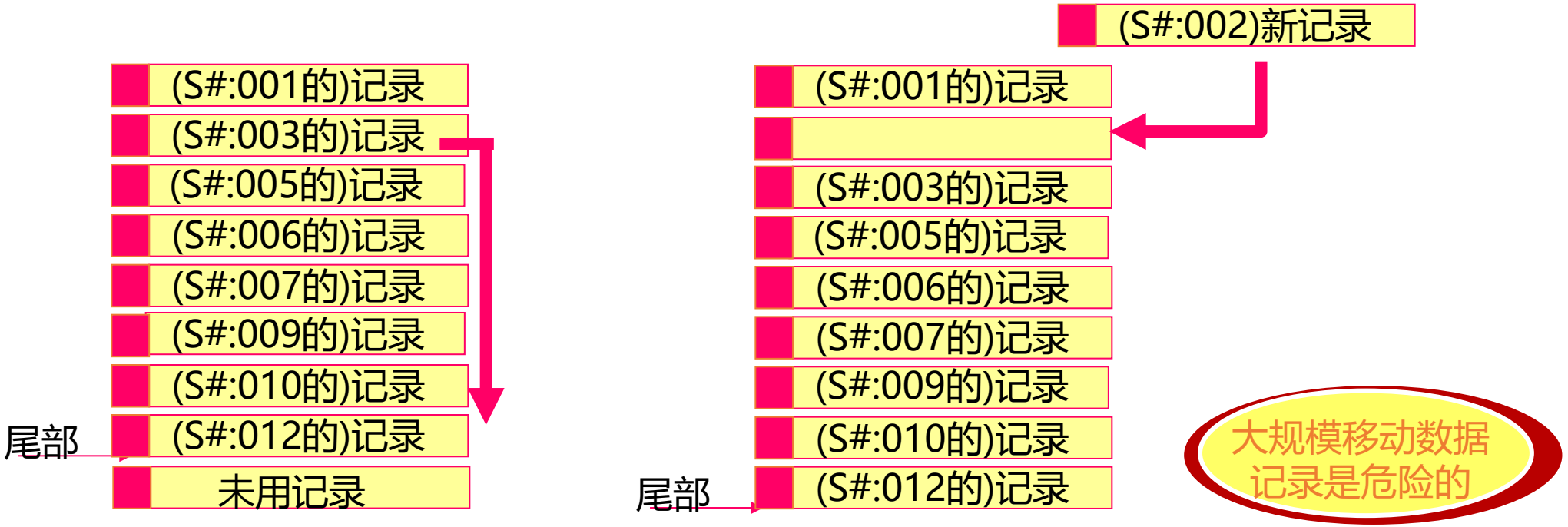


6.1 基础回顾-数据库之文件组织方法

➢(3) 有序文件组织

➢文件组织方法之二：有序记录文件(排序文件Sequential)

- 有序记录文件的更新效率可能很低。
- 因为：在更新时要移动其他记录，为插入记录留出空间。

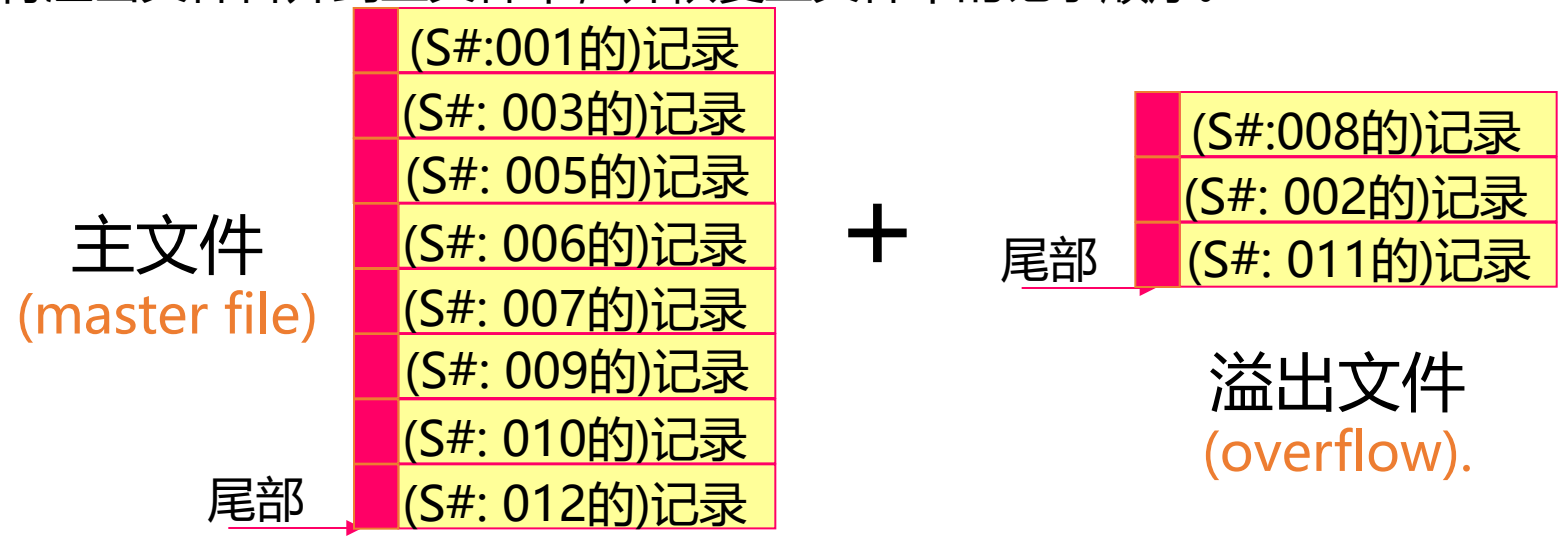


6.1 基础回顾-数据库之文件组织方法

➢(3) 有序文件组织

➢文件组织方法之二：有序记录文件(排序文件Sequential)

- 改进措施是可为将来有可能插入的元组预留空间(这可能造成空间浪费),或者再使用一个临时的无序文件(被称为溢出文件)保留新增的记录。
- 当采取溢出文件措施时，检索操作既要操作主文件，又要操作溢出文件。
- 所以需要周期性重新组织数据库
- 数据库重组是将溢出文件合并到主文件中，并恢复主文件中的记录顺序。

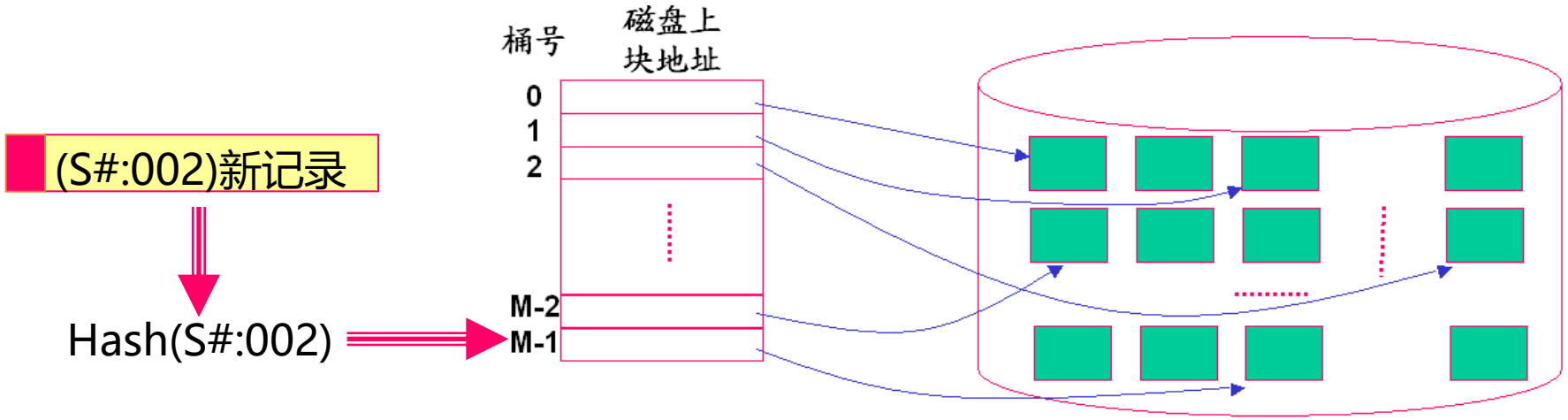


6.1 基础回顾-数据库之文件组织方法

➢(4) 散列文件组织

➢文件组织方法之三：散列文件(Hash file)

- 特点：可以把记录按某属性或属性组的值，依据一个散列函数来计算其应存放的位置：桶号(Bucket，块号或簇号等)。检索效率和更新效率都有一定程度的提高
- 用于进行散列函数计算的属性通常称为散列字段(Hash field)，散列字段通常也采用关系中的主码，所以又称散列码(hash key)

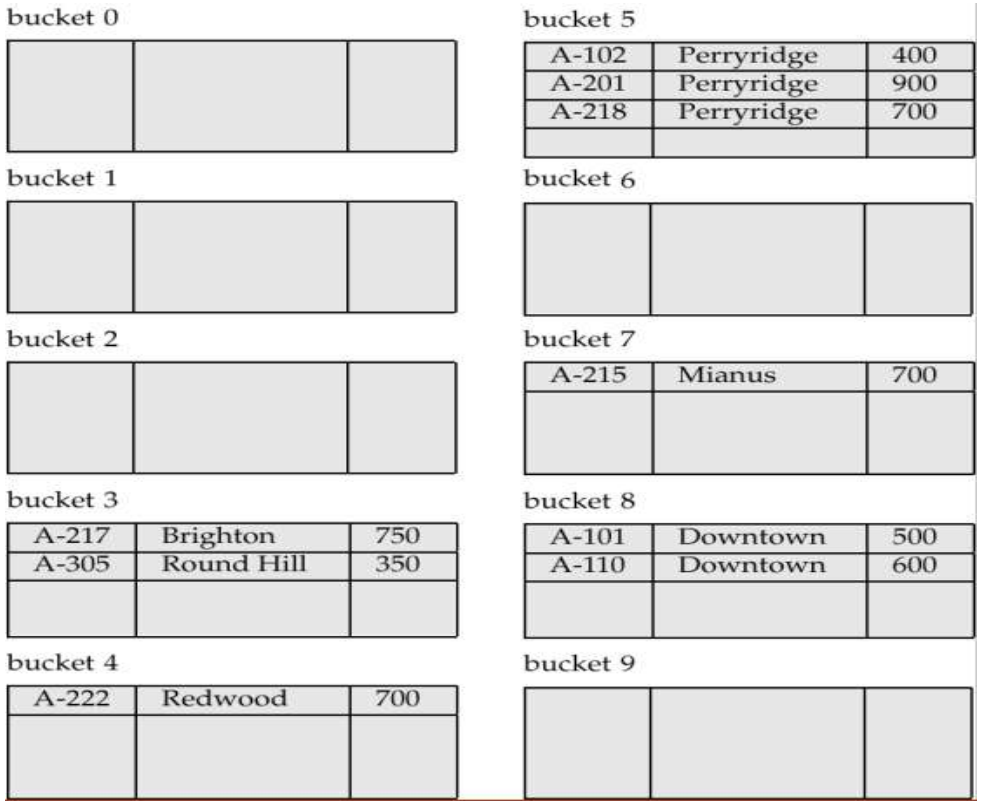


6.1 基础回顾-数据库之文件组织方法

➢(4) 散列文件组织

➢文件组织方法之三：散列文件(Hash file)

- 不同记录可能被hash成同一桶号，此时需在桶内顺序检索出某一记录

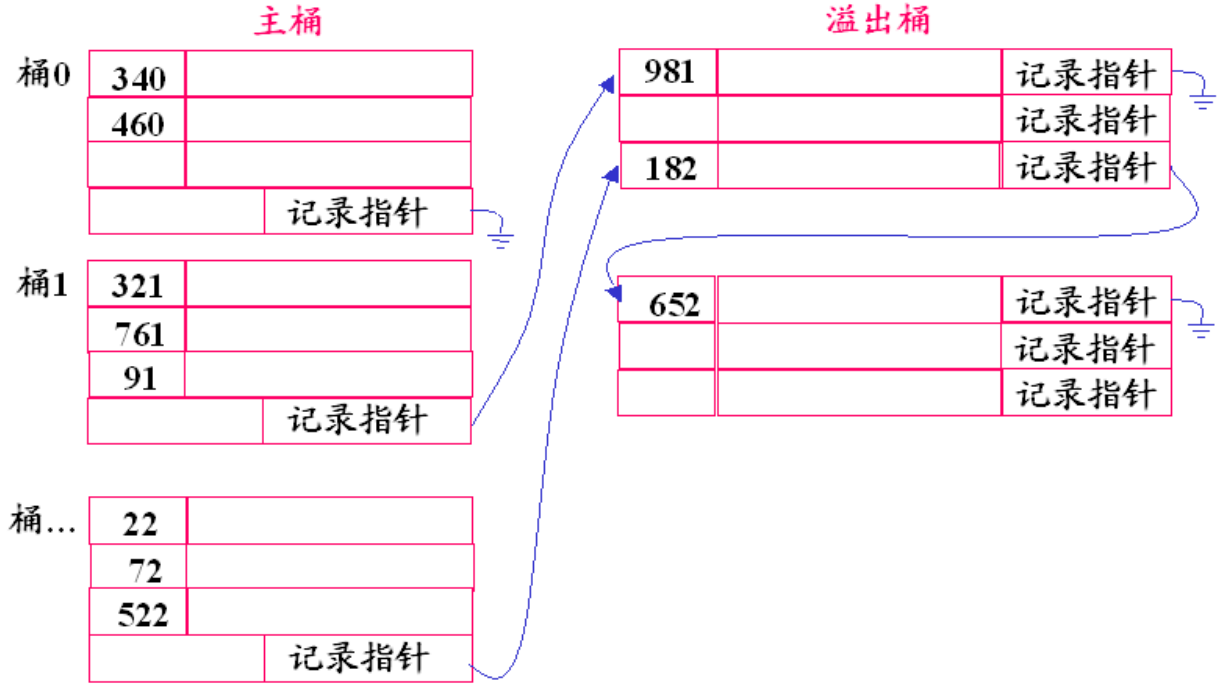


6.1 基础回顾-数据库之文件组织方法

➢(4) 散列文件组织

➢文件组织方法之三：散列文件(Hash file)

- 链接法处理溢出
- 散列还有许多问题及许多的处理技巧, 如散列桶的数目以及桶的大小，动态散列技术等等(这里不再叙述)



6.1 基础回顾-数据库之文件组织方法

➢(5) 聚簇文件组织

➢文件组织方法之四：聚簇文件(Clustering file)

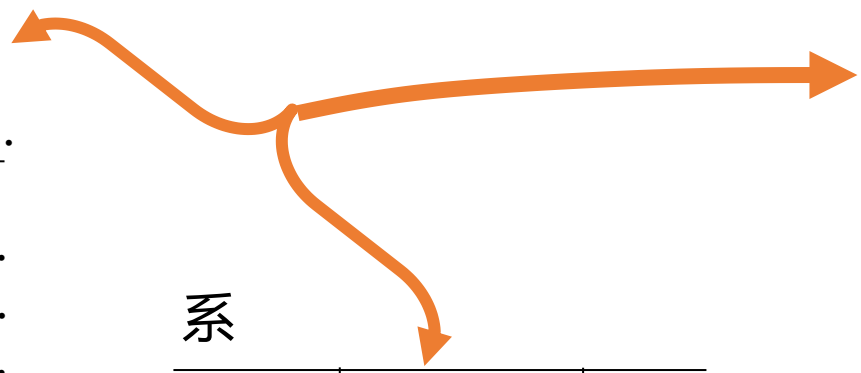
- 聚簇：将具有相同或相似属性值的记录存放于连续的磁盘簇块中
- 多表聚簇：将若干个相互关联的Table存储于一个文件中—这可提高多表情况下的查询速度(有很多问题及相关的处理技巧，不再详细叙述)

教师

教师号	教师名	系编号	....
教师1	张三	系1	....
教师5	李一	系2	....
教师8	王一	系2	....
教师2	张四	系1	....
教师9	李二	系4	....
教师3	王二	系1	....
教师6	王三	系4	....

系

系编号	系名		....
系1	计算机		....
系2	自动控制		....
系4	化学		....



(系1的)记录
(系1的教师1的)记录
(系1的教师2的)记录
(系1的教师3的)记录
(系2的)记录
(系2的教师5的)记录
(系2的教师8的)记录
(系4的)记录
(系4的教师6的)记录
(系4的教师9的)记录

6.1 基础回顾-数据库之文件组织方法

➢(6) 小结

➢增删改时如何快速 “存”

➢检索查询时如何快速 “取”

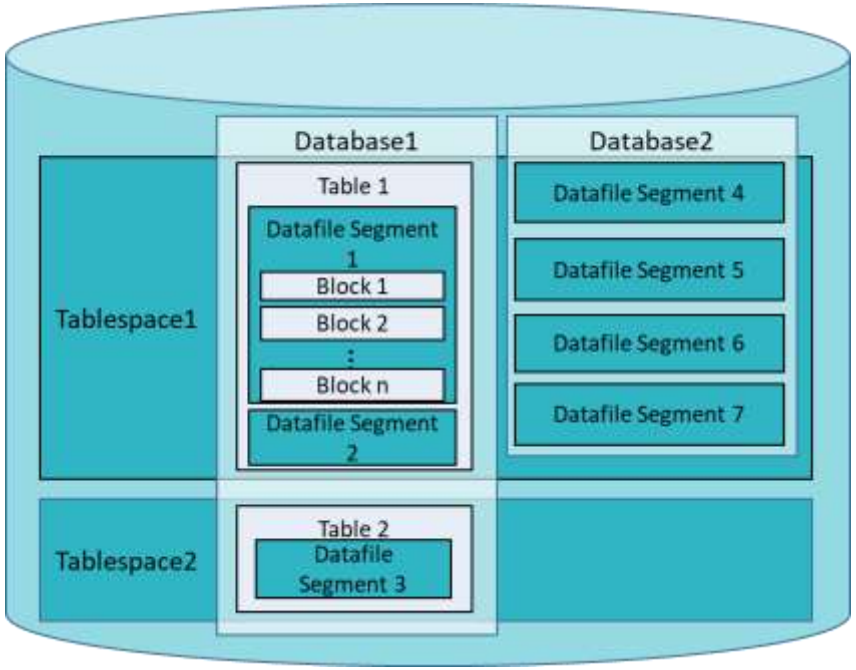


6.1 基础回顾-openGauss物理存储简介

- **openGauss的数据库节点负责存储数据，其存储介质也是磁盘，本节主要从逻辑视角介绍数据库节点都有哪些对象，以及这些对象之间的关系。**

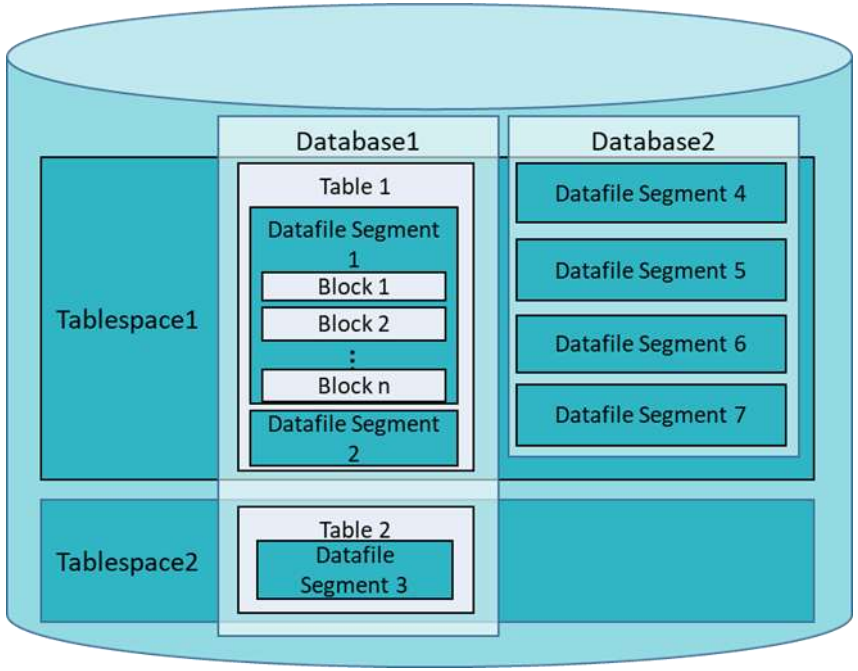
数据库逻辑结构如下图：

- Tablespace：表空间，是一个目录，可以存在多个，里面存储的是它所包含的数据库的各种物理文件。每个表空间可以对应多个Database。
- Database：数据库，用于管理各类数据对象，各数据库间相互隔离。数据库管理的对象可分布在多个Tablespace上。
- Datafile Segment：数据文件，通常每张表只对应一个数据文件。如果某张表的数据大于1GB，则会分为多个数据文件存储。
- Table：表，每张表只能属于一个数据库，也只能对应到一个Tablespace。每张表对应的数据文件必须在同一个Tablespace中。
- Block：数据块，是数据库管理的基本单位，默认大小为8KB。



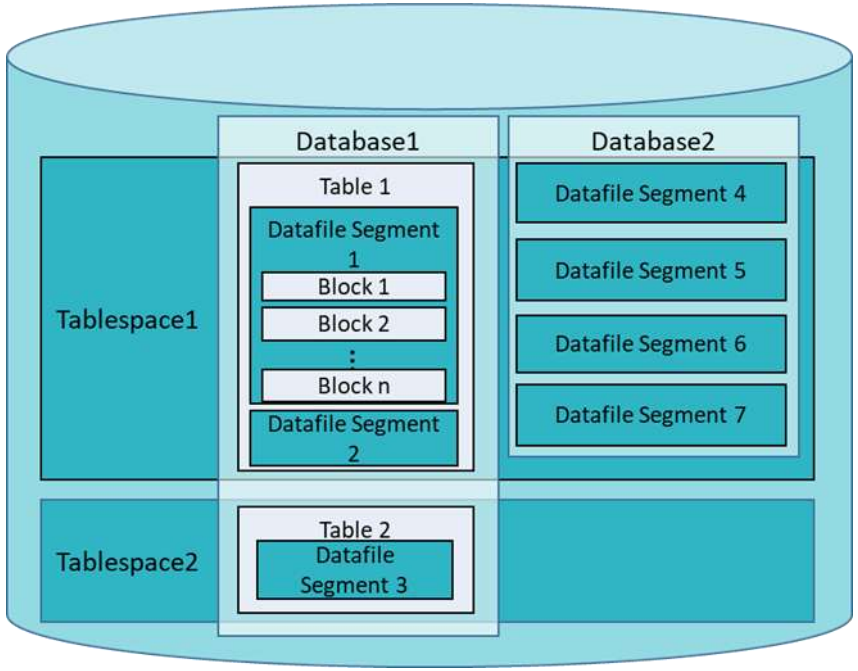
6.1 基础回顾-openGauss物理存储简介

- 每个数据库分成一个或多个表空间，所有表空间的组合存储容量即是数据库的存储容量
- 每个表空间由一个或多个操作系统文件构成，一个操作系统文件只能与一个数据库相联，操作系统文件仅起占位的作用。数据在表空间中可跨文件进行操作。
- 操作系统文件中存储一个表(Table)或多个表，一个表可存储在一个文件中也可能存储在多个文件中。
- 上述为逻辑存储层



6.1 基础回顾-openGauss物理存储简介

- 物理存储层由段(segment)和数据块(data block)构成
- 数据块是最小的IO存储单位 (相当于扇区)
- 段是一组分配了特定数据结构的盘区，又分为数据段、索引段和临时段等
- 一个表的数据可以存放在一个段内，也可以存放在多个段内。一个段可以存放一个表的数据，也可以存放多个表的数据(如聚簇文件)



6.1 基础回顾-总结


数据库物理存储
磁盘块的分配：顺序、 链接、索引、按簇
记录跨块vs. 记录不跨块
定长记录vs. 不定长记录
表
记录
属性值
磁盘块
主文件
索引文件
无序记录文件
有序记录文件
散列记录文件
聚簇记录文件
数据库重组

6.2 数据库索引

➢基本内容

1. 为什么需要索引与什么是索引

2. SQL语言中的索引创建与维护

2. 索引的简单分类

3. B+树索引

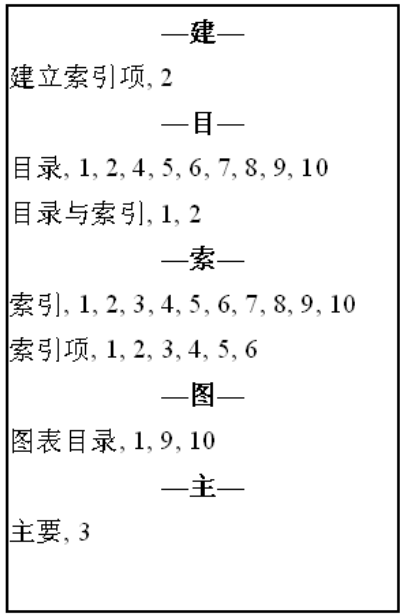
4. 散列索引

6.2 数据库索引-为什么需要索引与什么是索引

➢(1) 索引的概念

➢ 书籍、词典/字典中的词汇表(索引表)

- 词汇表等包含两部分：词条，词条在文件中的页码
- 词汇表等通常按词条进行某种方式的排序
- 目的是通过对“有序的小数据量的词汇表”的快速查找，发现词条在“大数据量书籍中”出现的位置。



—建—

建立索引项, 2

—目—

目录, 1, 2, 4, 5, 6, 7, 8, 9, 10

目录与索引, 1, 2

—索—

索引, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

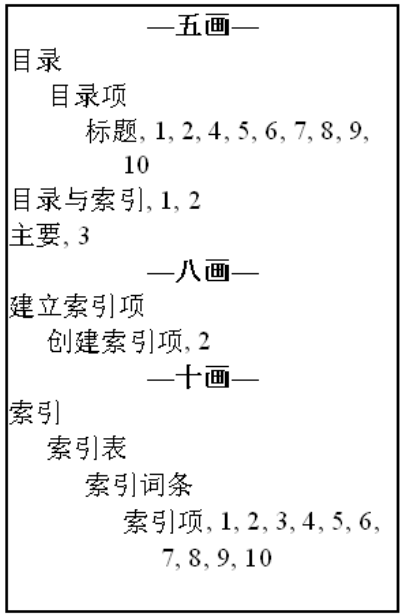
索引项, 1, 2, 3, 4, 5, 6

—图—

图表目录, 1, 9, 10

—主—

主要, 3



—五画—

目录

目录项

标题, 1, 2, 4, 5, 6, 7, 8, 9, 10

目录与索引, 1, 2

主要, 3

—八画—

建立索引项

创建索引项, 2

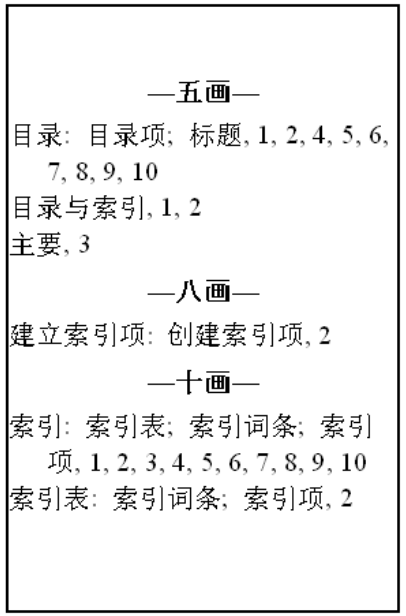
—十画—

索引

索引表

索引词条

索引项, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10



—五画—

目录: 目录项; 标题, 1, 2, 4, 5, 6, 7, 8, 9, 10

目录与索引, 1, 2

主要, 3

—八画—

建立索引项: 创建索引项, 2

—十画—

索引: 索引表; 索引词条; 索引项, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

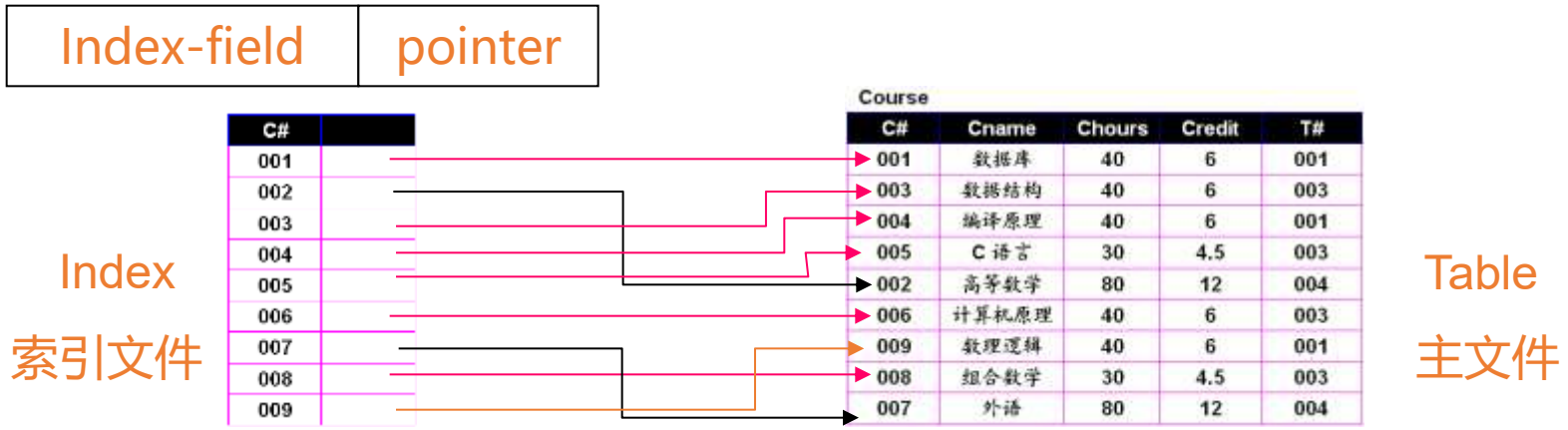
索引表: 索引词条; 索引项, 2

6.2 数据库索引-为什么需要索引与什么是索引

➢(1) 索引的概念

➢索引是定义在存储表(Table)基础之上，有助于无需检查所有记录而快速定位所需记录的一种辅助存储结构，由一系列存储在磁盘上的索引项(index entries)组成，每一索引项又由两部分构成:

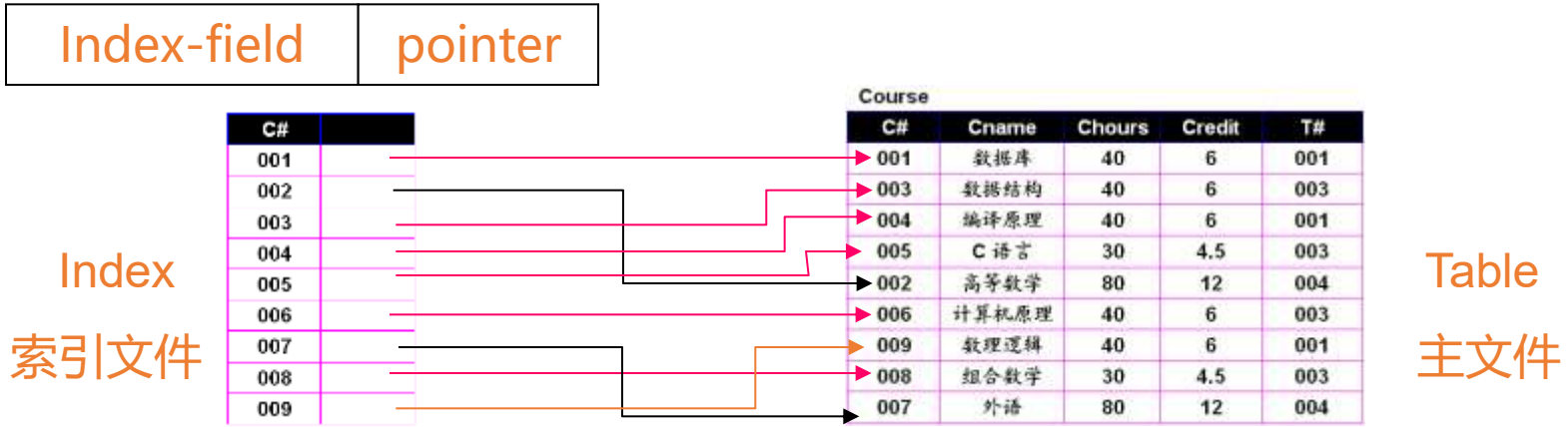
- 索引字段：由Table中某些列(通常是一列)中的值串接而成。索引中通常存储了索引字段的每一个值(也有不是这样的)。索引字段类似于词典中的词条。
- 行指针：指向Table中包含索引字段值的记录在磁盘上的存储位置。行指针类似于词条在书籍、词典中出现的页码。
- 存储索引项的文件为索引文件，相对应，存储表又称为主文件。



6.2 数据库索引-为什么需要索引与什么是索引

➢(2) 索引的一般性特点

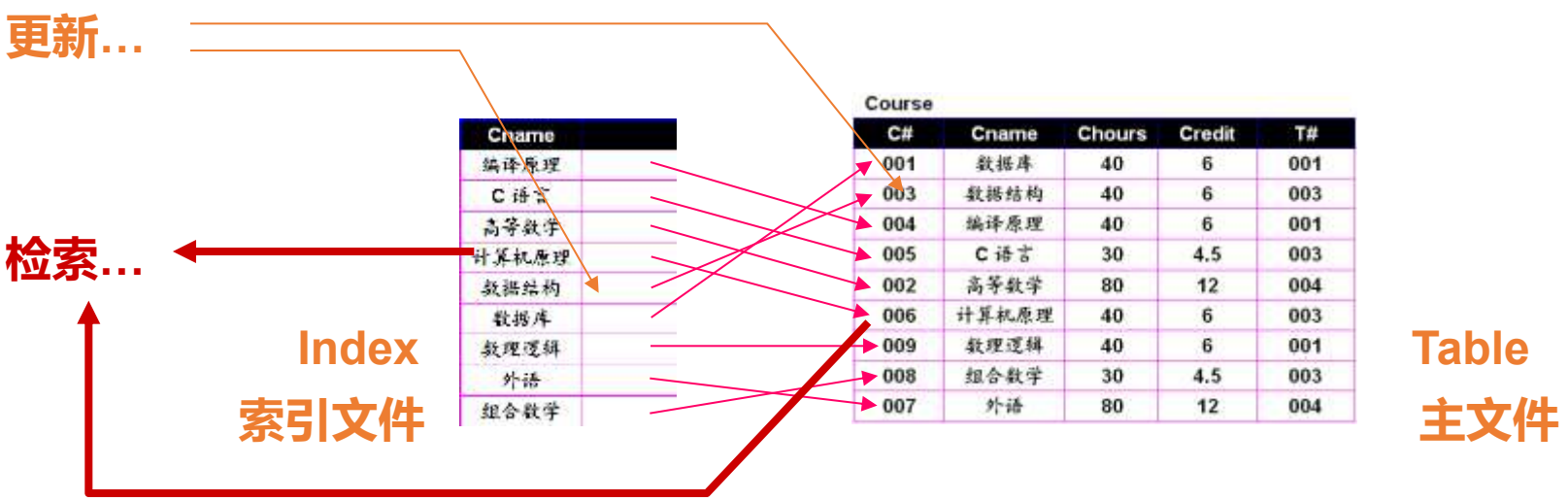
- 索引文件是一种辅助存储结构，其存在与否不改变存储表的物理存储结构；然而其存在，可以明显提高存储表的访问速度。
- 索引文件组织方式有两种：(相对照的，主文件组织有堆文件、排序文件、散列文件、聚簇文件等多种方式)
  - 排序索引文件(Ordered indices)：按索引字段值的某一种顺序组织存储
  - 散列索引文件(Hash indices)：依据索引字段值使用散列函数分配散列桶的方式存储



6.2 数据库索引-为什么需要索引与什么是索引

➢(2) 索引的一般性特点

- 在一个表上可以针对不同的属性或属性组合建立不同的索引文件，可建立多个索引文件。索引字段的值可以是Table中的任何一个属性的值或任何多个属性值的组合值
- 索引文件比主文件小很多。通过检索一个小的索引文件(可全部装载进内存)，快速定位后，再有针对性的读取非常大的主文件中的有关记录
- 有索引时，更新操作必须同步更新索引文件和主文件。否则...



6.2 数据库索引-为什么需要索引与什么是索引

➢(3) 关于索引应用的评价问题

- 关于索引应用的评价
  索引技术应用使检索效率大幅度提高，但同时其也增加了存储空间、使维护负担加重(不仅要维护主文件，而且要维护索引文件)
- 衡量索引性能好坏：
  - ✓访问时间
  - ✓插入开销
  - ✓删除开销
  - ✓空间负载
  - ✓支持存取的有效性, 比如：支持的是属性的限定值(是否符合单一值)，还是支持属性的限定范围的值(是否符合一定范围)

对哪些属性建立索引?

SELECT…FROM…WHERE…GROUP BY…

对经常出现在检索条件、连接条件、分组计算条件中的属性可建立索引

6.2 数据库索引-为什么需要索引与什么是索引

➢(4) 关于索引相关的概念含义的区分

几个概念的澄清

- 字段、排序字段、索引字段
- 码(Key)、主码(Primary Key), 又称为表键(Table Key)---具有唯一性和最小性
- 排序码(Order Key)---对主文件进行排序存储的那些属性或属性组
- 索引码(Index Key)---即索引字段，不一定具有唯一性
- 搜索码(Search Key)---在主文件中查找记录的属性或属性集
- 主文件 vs. 索引文件



6.2 数据库索引-SQL语言中的索引创建与维护

➢(1) 基本知识

SQL语言关于索引的基本知识

- 当定义Table后，如果定义了主键，则系统将自动创建主索引，利用主索引对Table进行快速定位、检索与更新操作;
- 索引可以由用户创建，也可以由用户撤消
- 当索引被创建后，无论是主索引，还是用户创建的索引，DBMS都将自动维护所有的索引，使其与Table保持一致，即：当一条记录被插入到Table中后，所有索引也自动的被更新
- 当Table被删除后(drop table), 定义在该Table上的所有索引将自动被撤消

6.2 数据库索引-SQL语言中的索引创建与维护

➢(2) 创建和维护索引的SQL语句

- 索引可以由用户在任何属性上创建

  X/OPEN SQL中关于索引的语句

➢ 创建索引：

```
CREATE [unique] INDEX indexname
        ON tablename ( colname [asc | desc]
                {, colname [asc | desc] . . .} );
```

- 示例：在student表中创建一个基于Sname的索引

```
create index idxSname on student(sname);
```

- 示例：在student表中创建一个基于Sname和Sclass的索引

```
create index idxSnamcl on student(sname, sclass);
```

- 如某索引不再需要，则可通过撤消命令，撤消用户创建的索引

```
DROP INDEX indexname;
```

6.2 数据库索引-SQL语言中的索引创建与维护

➢(3) 索引应用要注意效果

- 选择哪些属性创建索引，以及如何创建与维护索引，如何利用索引改善数据库的运行性能，是DBA(数据库管理员)的重要职责。
- 是否建立和在哪些属性上建立索引需要考虑：访问时间、插入时间、删除时间与空间负载。既要改善性能，又要控制代价。
- 建立索引还需考虑索引的类型：索引如何支持存取的有效性, 比如：支持的是属性的限定值(是否符合单一值)，还是支持属性的限定范围的值(是否符合一定范围)



对经常出现在检索条件、连接条件、分组计算条件中的属性可建立索引

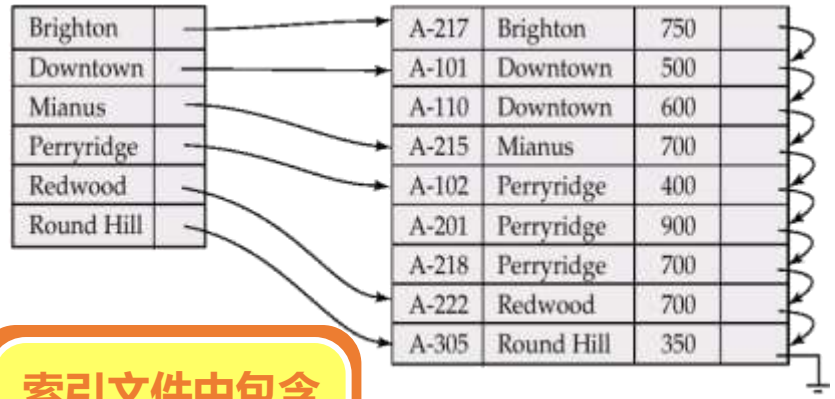
SELECT…FROM…WHERE…GROUP BY…

6.2 数据库索引-稠密索引与稀疏索引

➢(1) 稠密索引与稀疏索引

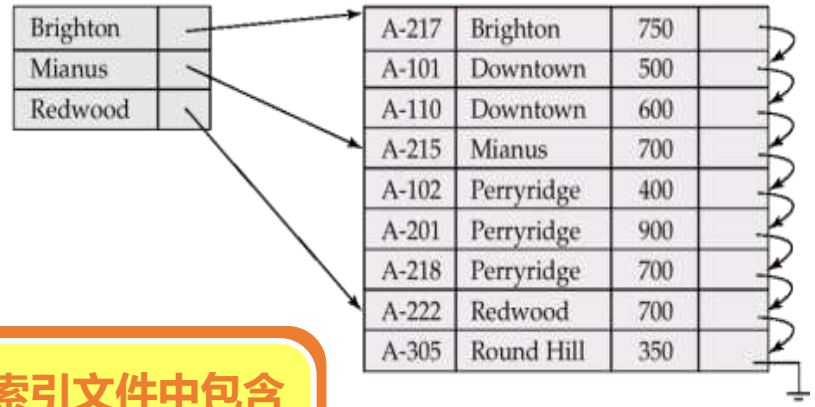
- 对于主文件中每一个记录(形成的每一个索引字段值)，都有一个索引项和它对应，指明该记录所在位置。这样的索引称稠密索引(dense index)
- 对于主文件中部分记录(形成的索引字段值)，有索引项和它对应，这样的索引称非稠密索引(undense index)或稀疏索引(sparse index)

稠密索引



索引文件中包含了主文件对应字段的**所有**不同值

稀疏索引

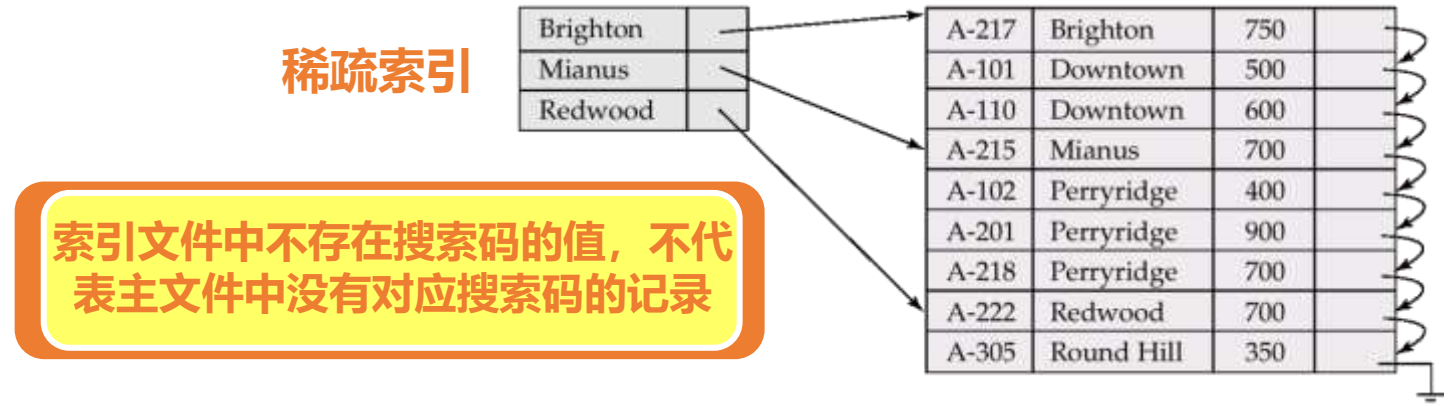


索引文件中包含了主文件对应字段的**部分**不同值

6.2 数据库索引-稠密索引与稀疏索引

➢(2) 稀疏索引如何定位记录

- 定位索引字段值为 K的记录，需要
  - ✓ 首先找相邻的小于K的最大索引字段值所对应的索引项
  - ✓ 从该索引项所对应的记录开始顺序进行Table的检索
- 稀疏索引的使用要求—主文件必须是按对应索引字段属性排序存储
- 相比稠密索引：空间占用更少，维护任务更轻，但速度更慢
- 平衡: 索引项不指向记录指针，而是指向记录所在存储块的指针，即每一存储块有一个索引项，而不是每条记录有一索引项----主索引

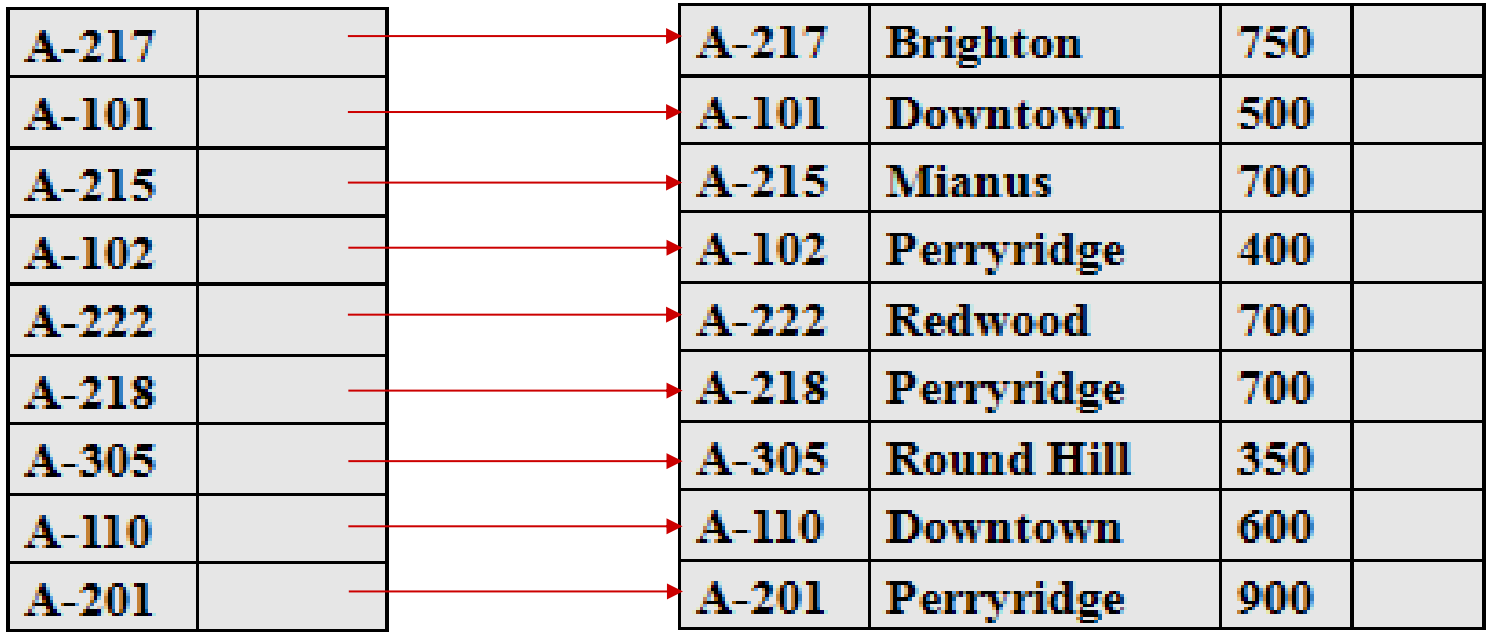


6.2 数据库索引-稠密索引与稀疏索引

➢(3) 稠密索引如何定位记录

候选键属性的稠密索引—先查索引，然后再依据索引读主文件

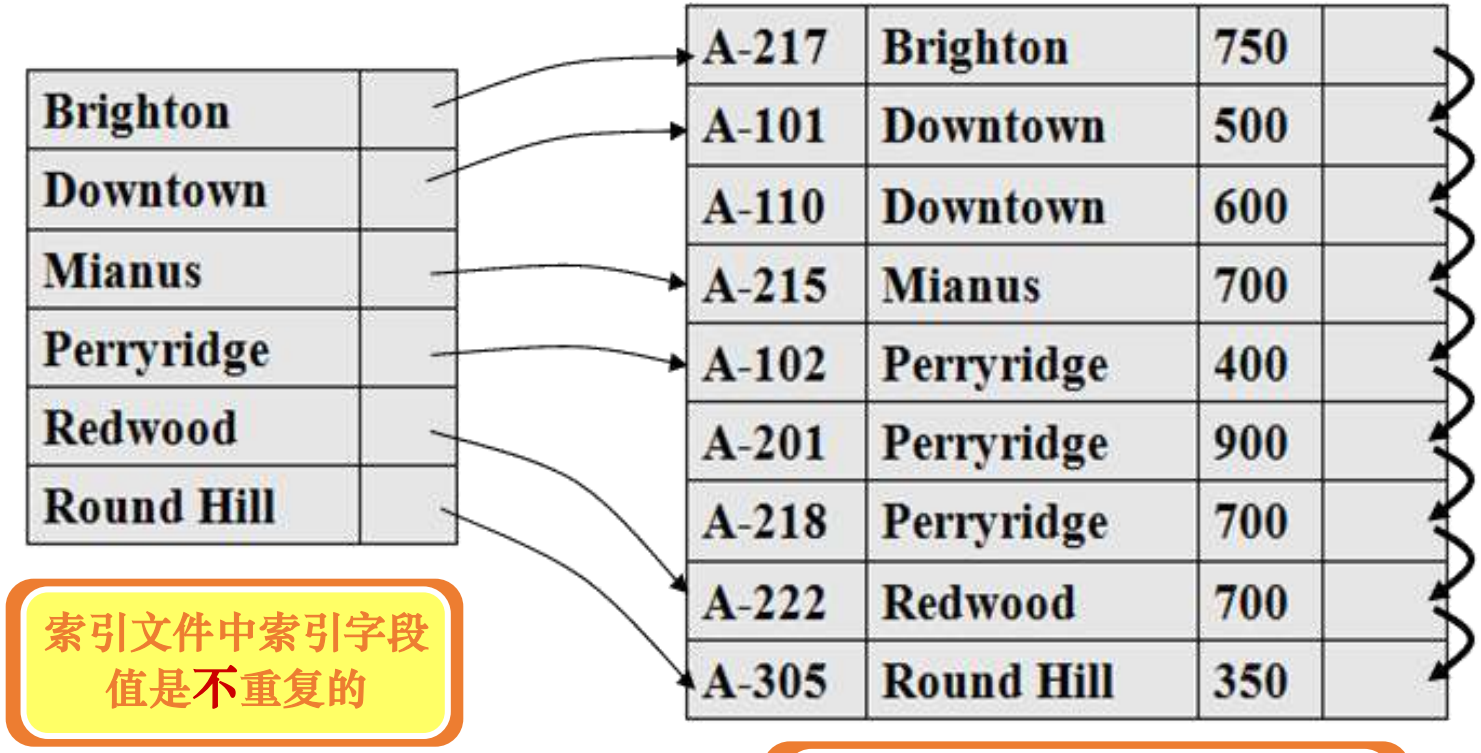
无论是候选键属性的稠密索引，还是非候选键属性的稠密索引：索引文件中不存在搜索码的值，就代表着主文件中没有对应搜索码的记录



6.2 数据库索引-稠密索引与稀疏索引

➢(3) 稠密索引如何定位记录

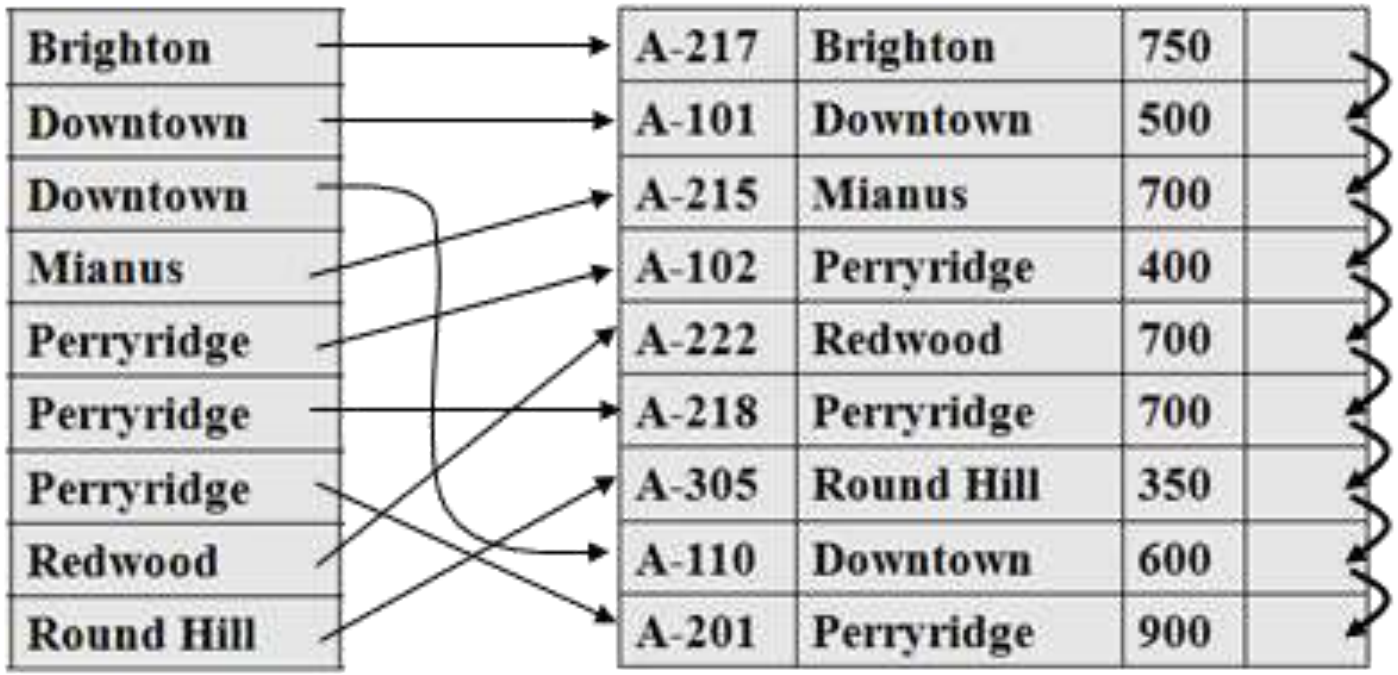
非候选键属性的稠密索引（I）



6.2 数据库索引-稠密索引与稀疏索引

➢(3) 稠密索引如何定位记录

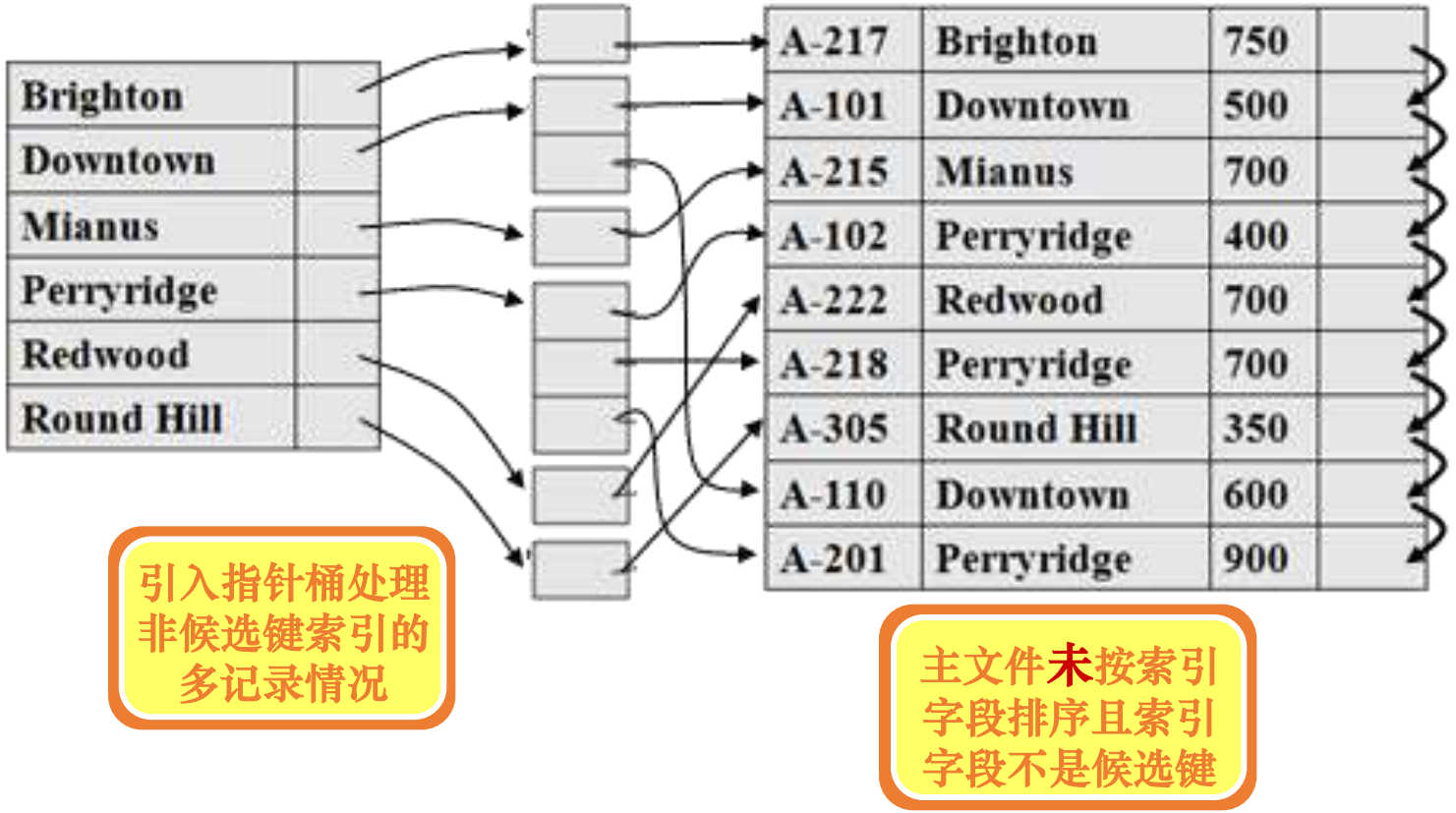
非候选键属性的稠密索引（II）



6.2 数据库索引-稠密索引与稀疏索引

➢(3) 稠密索引如何定位记录

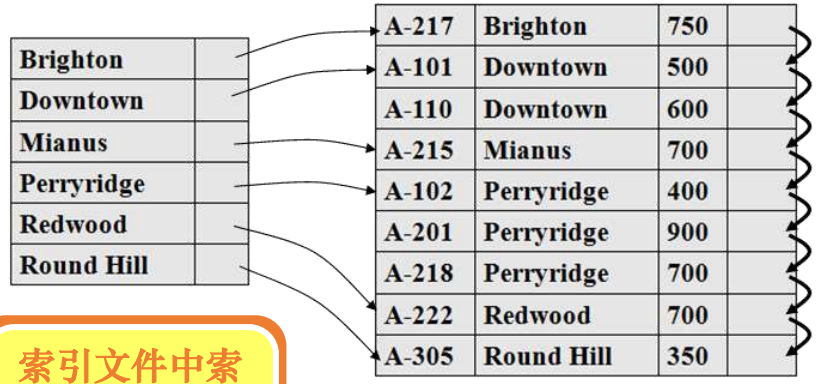
非候选键属性的稠密索引（III）



6.2 数据库索引-稠密索引与稀疏索引

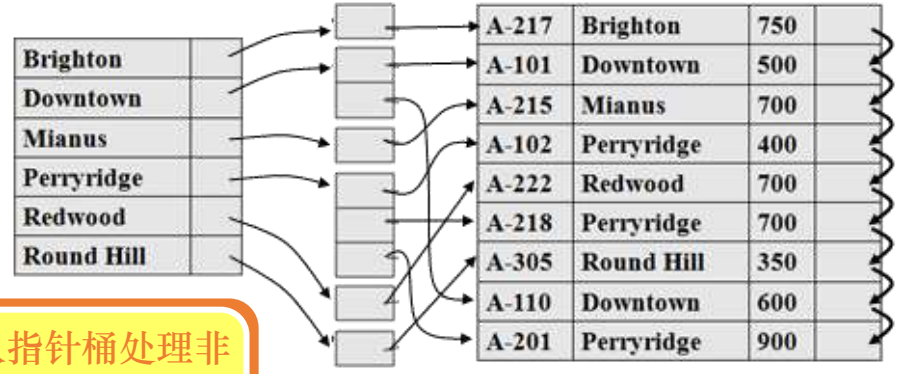
➢(3) 稠密索引如何定位记录

非候选键属性的稠密索引的三种情况之对比



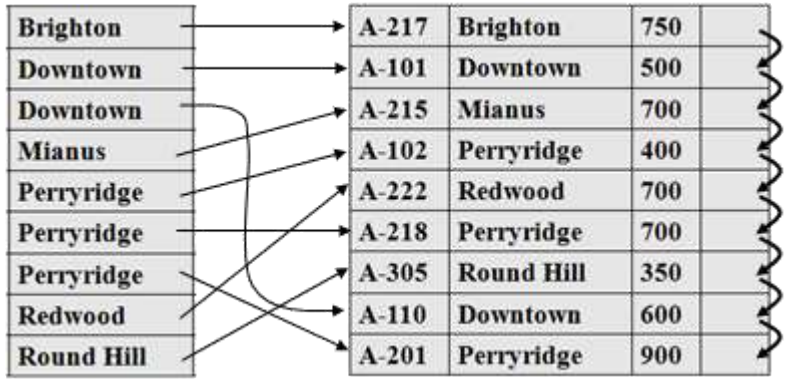
索引文件中索引字段值是不重复的

主文件按索引字段排序且索引字段不是候选键



引入指针桶处理非候选键索引的多记录情况

主文件未按索引字段排序且索引字段不是候选键



索引文件中索引字段值是有重复的

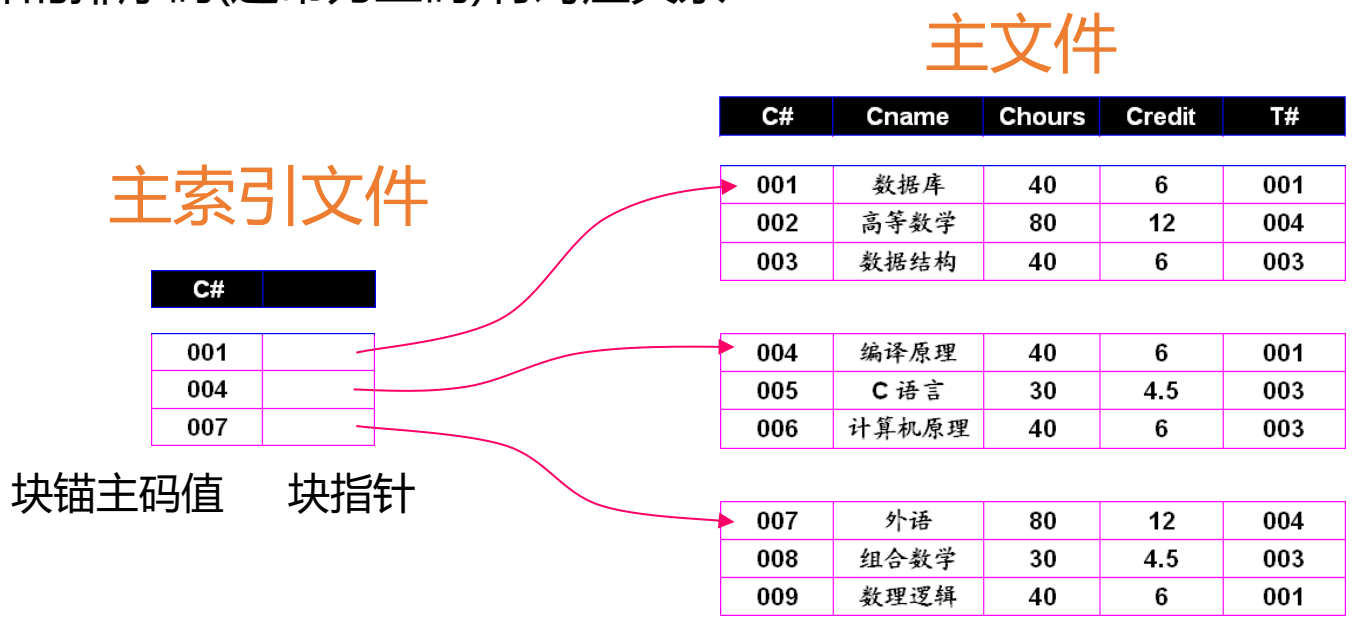
主文件未按索引字段排序且索引字段不是候选键

6.2 数据库索引-主索引与辅助索引

➢(1) 主索引

主索引通常是对每一存储块有一个索引项，索引项的总数和存储表所占的存储块数目相同，存储表的每一存储块的第一条记录，又称为锚记录(anchor record), 或简称为块锚(block anchor)

- 主索引的索引字段值为块锚的索引字段值，而指针指向其所在的存储块。
- 主索引是按索引字段值进行排序的一个有序文件，通常建立在有序主文件的基于主码的排序字段上，即主索引的索引字段与主文件的排序码(通常为主码)有对应关系
- 主索引是稀疏索引。

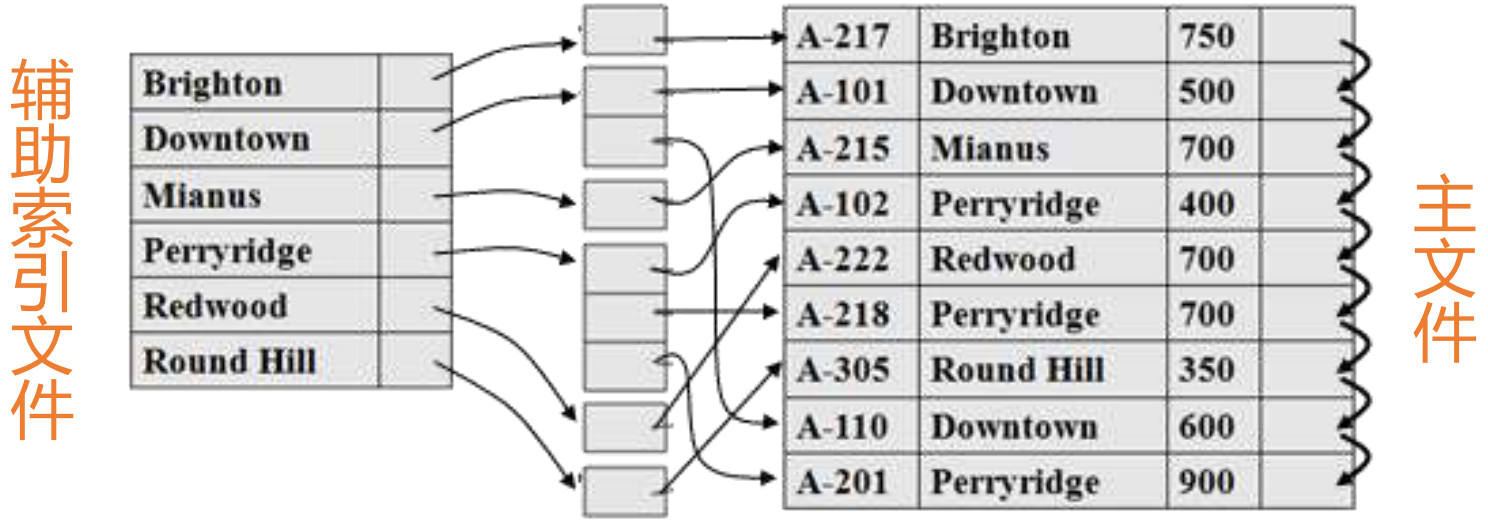


6.2 数据库索引-主索引与辅助索引

➢(2) 辅助索引

辅助索引 是定义在主文件的任一或多个非排序字段上的辅助存储结构。

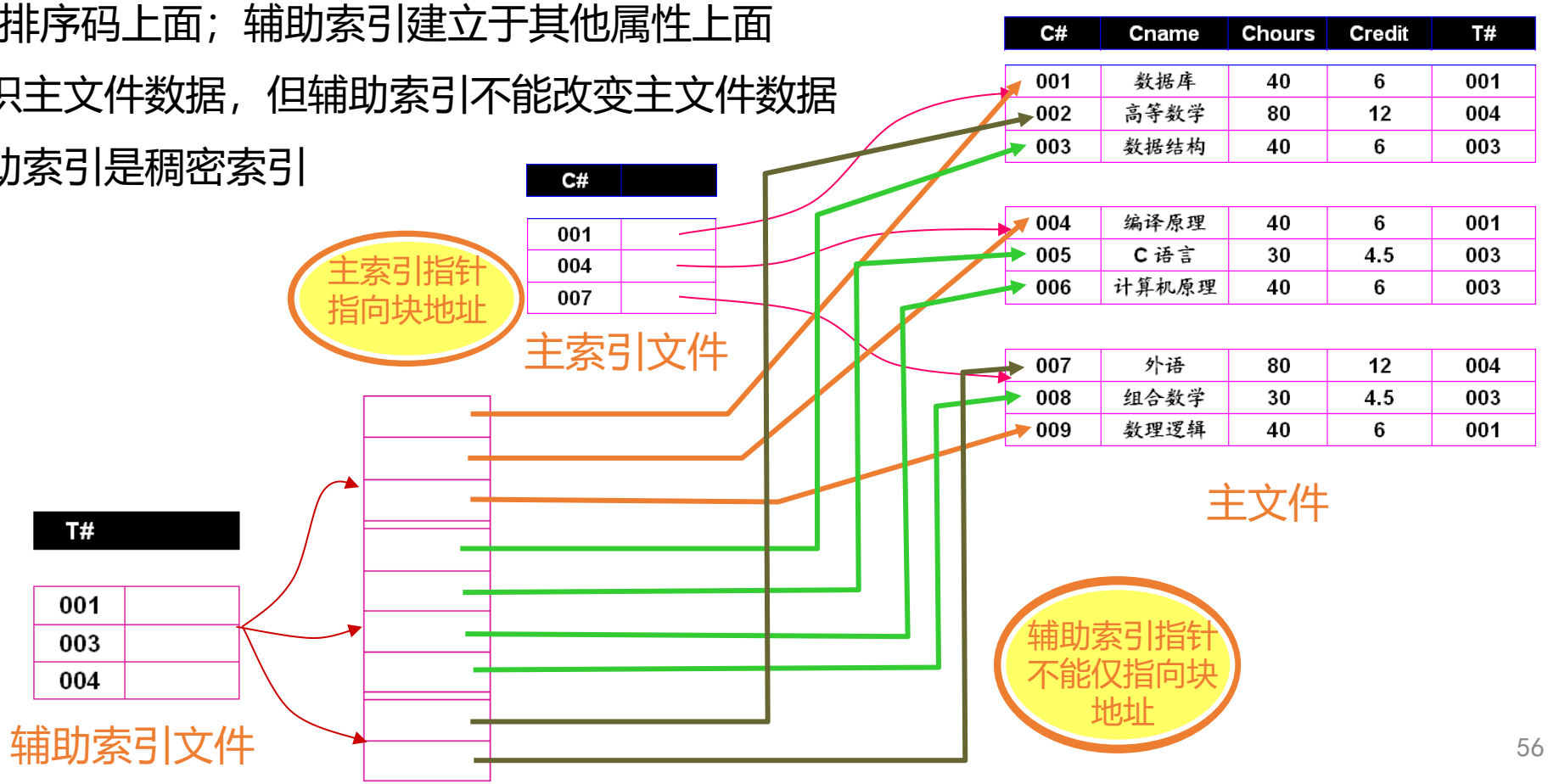
- 辅助索引通常是对某一非排序字段上的每一个不同值有一个索引项：索引字段即是该字段的不同值，而指针则指向包含该记录的块或该记录本身；
- 当非排序字段为索引字段时，如该字段值不唯一，则要采用一个类似链表的结构来保存包含该字段值的所有记录的位置。
- 辅助索引是稠密索引，其检索效率有时相当高。



6.2 数据库索引-主索引与辅助索引

➢(3) 主索引 vs. 辅助索引

- 一个主文件仅可以有一个主索引，但可以有多个辅助索引
- 主索引通常建立于主码/排序码上面；辅助索引建立于其他属性上面
- 可以利用主索引重新组织主文件数据，但辅助索引不能改变主文件数据
- 主索引是稀疏索引，辅助索引是稠密索引



6.2 数据库索引-其他类型的索引

➢(1) 聚簇索引和非聚簇索引

➢聚簇索引是指索引中邻近的记录在主文件中也是临近存储的。

T#是“非主码字段”，有重复值，排序存储

聚簇索引文件

T#	
001	
003	
004	

聚簇字段值　　块指针

主文件

C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
004	编译原理	40	6	001
009	数理逻辑	40	6	001
008	组合数学	30	4.5	003
003	数据结构	40	6	003
005	C 语言	30	4.5	003
006	计算机原理	40	6	003
002	高等数学	80	12	004
007	外语	80	12	004

主文件基于“非主码的字段”T#排序存储

➢非聚簇索引是指索引中邻近的记录在主文件中不一定是邻近存储的。

T#是“非主码字段”，有重复值，排序存储

非聚簇索引文件

T#	
001	
001	
001	
003	
003	
003	
003	
004	
004	

索引字段值　记录指针

主文件

C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
002	高等数学	80	12	004
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C 语言	30	4.5	003
006	计算机原理	40	6	003
007	外语	80	12	004
008	组合数学	30	4.5	003
009	数理逻辑	40	6	001

主文件没有基于“非主码的字段”T#排序存储

6.2 数据库索引-其他类型的索引

➢(1) 聚簇索引和非聚簇索引

聚簇索引—是指索引中邻近的记录在主文件中也是临近存储的；

非聚簇索引—是指索引中邻近的记录在主文件中不一定是邻近存储的。

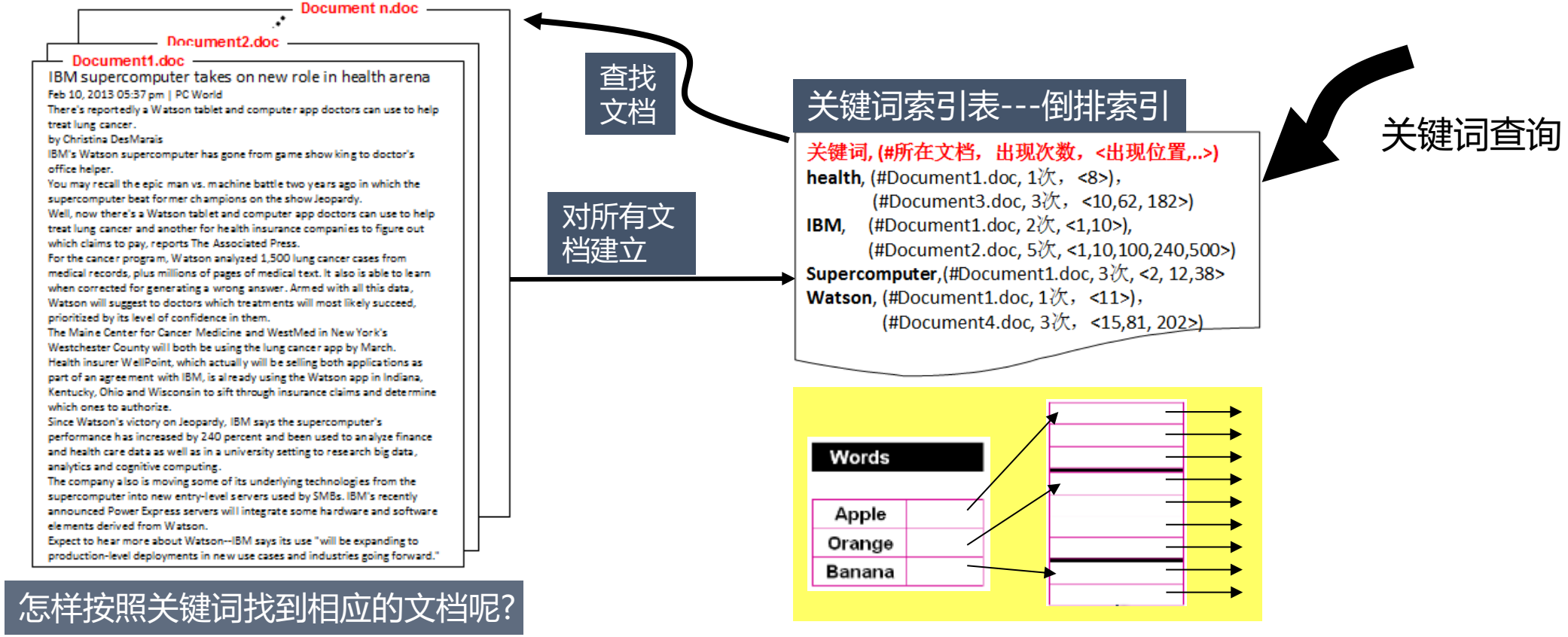
- 如果主文件的某一排序字段不是主码，则该字段上每个记录取值便不唯一，此时该字段被称为聚簇字段；聚簇索引通常是定义在聚簇字段上。
- 聚簇索引通常是对聚簇字段上的每一个不同值有一个索引项（索引项的总数和主文件中聚簇字段上不同值的数目相同），索引字段即是聚簇字段的不同值，由于有相同聚簇字段值的记录可能存储于若干块中，则索引项的指针指向其中的第一个块。
- 一个主文件只能有一个聚簇索引文件，但可以有多个非聚簇索引文件
- 主索引通常是聚簇索引(但其索引项总数不一定和主文件中聚簇字段上不同值的数目相同，其和主文件存储块数目相同)；辅助索引通常是非聚簇索引。
- 主索引/聚簇索引是能够决定记录存储位置的索引；而非聚簇索引则只能用于查询，指出已存储记录的位置。

6.2 数据库索引-其他类型的索引

➢(2) 倒排索引

正排：一个文档包含了哪些词汇？#Doc1, { Word1,Word2,... }

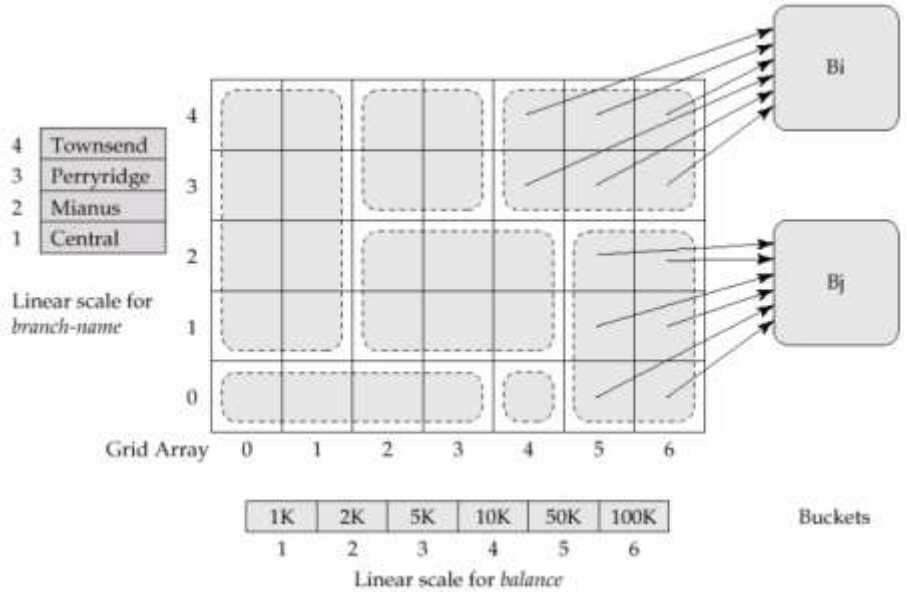
倒排：一个词汇包含在哪些文档中？ Word1, { #Doc1, #Doc2, ... }



6.2 数据库索引-其他类型的索引

➢(3) 其他结构索引

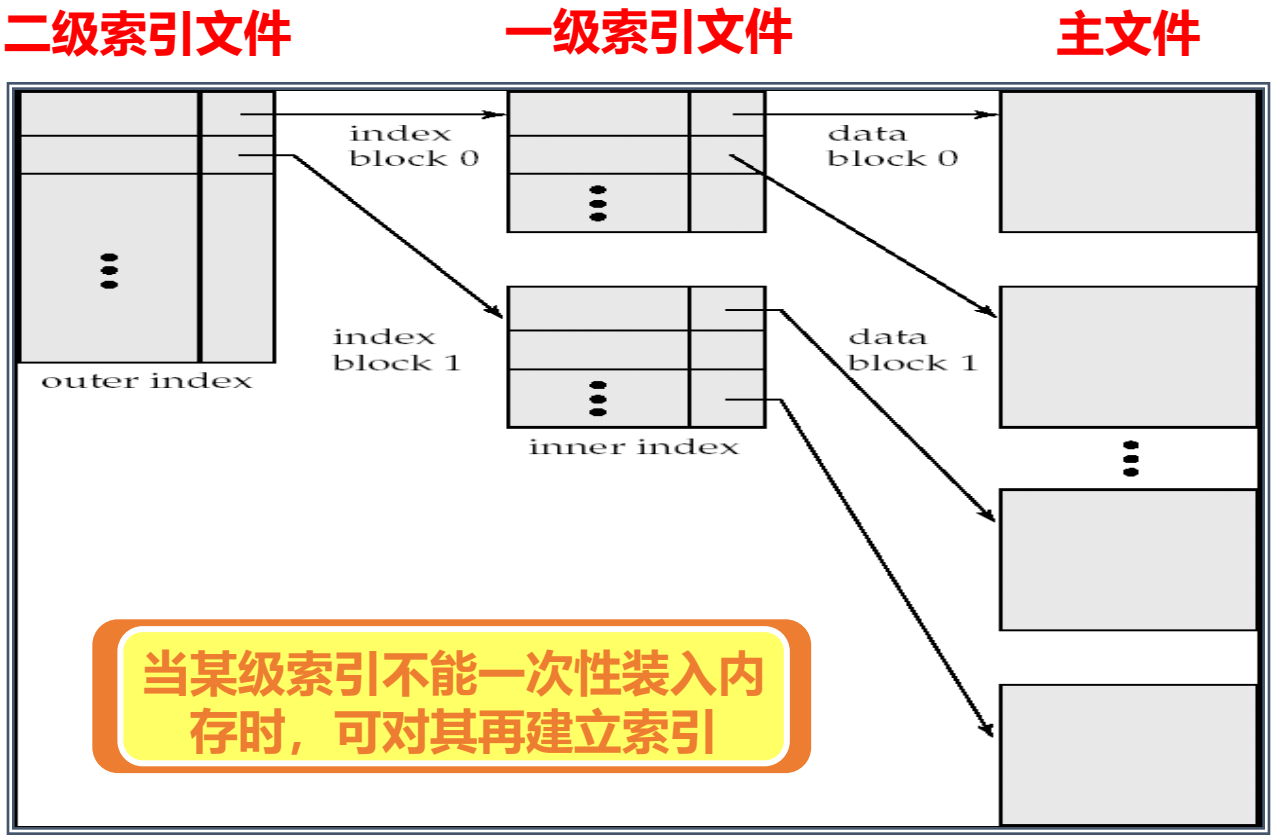
- 多级索引：当索引项比较多时，可以对索引再建立索引，依此类推，形成多级索引
- ✓ 常见的多级索引形式，如B树/B+树索引，以树型数据结构来组织索引项等
- 多属性索引：索引字段由Table的多个属性值组合在一起形成的索引
- 散列索引：使用散列技术组织的索引
- 网格索引(Grid file)：使用多索引字段进行交叉联合定位与检索



6.2 数据库索引-B+树(B+ Tree)

➢(1) 多级索引

多级索引：当索引项比较多时，可以对索引再建立索引，依此类推，形成多级索引。

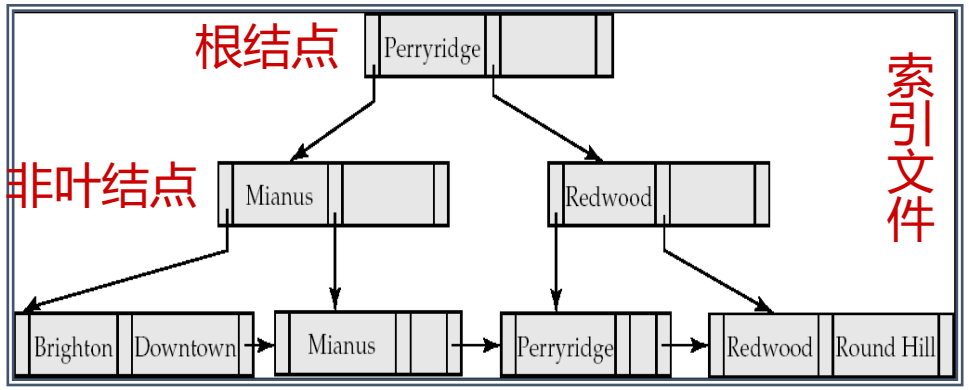


6.2 数据库索引-B+树(B+ Tree)

➢(2) B+树的基本概念

B+树索引：一种以树型数据结构来组织索引项的多级索引

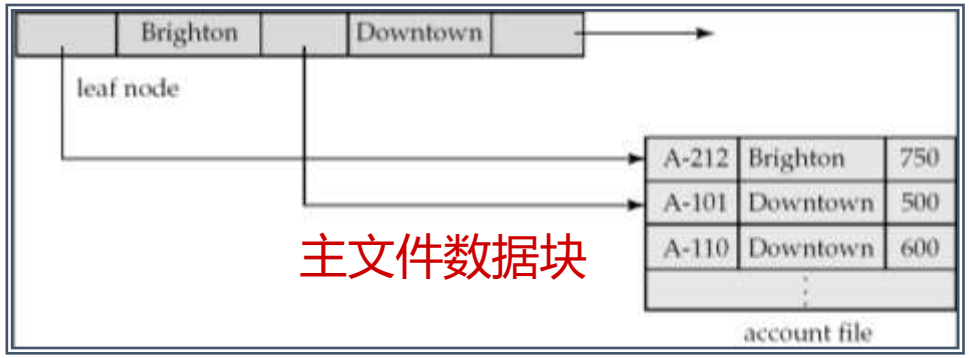




一块中索引项的组织

- K_i—索引字段值
- P_j—指针，指向索引块或数据块或数据块中记录的指针

索引文件的叶子结点



- 能够自动保持与主文件大小相适应的树的层次
- 每个索引块的指针利用率都在50%-100%之间

怎样实现呢?

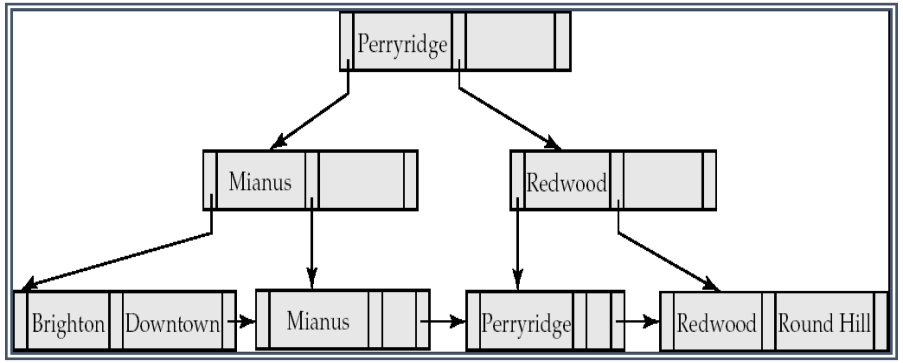
6.2 数据库索引-B+树(B+ Tree)

➢(3) B+树的存储约定

一块中存放多少个索引项(n的大小)?

P_1	K_1	P_2	...	P_{n-1}	K_{n-1}	P_n

- 有n-1个索引项(<索引字段值K_i, 指针P_i >)+1个指针(P_n);
- 索引字段值x在$K_i - 1 \leq X < K_i$的由P_i指向;而$K_i \leq X < K_i + 1$的由$P_i + 1$指向。



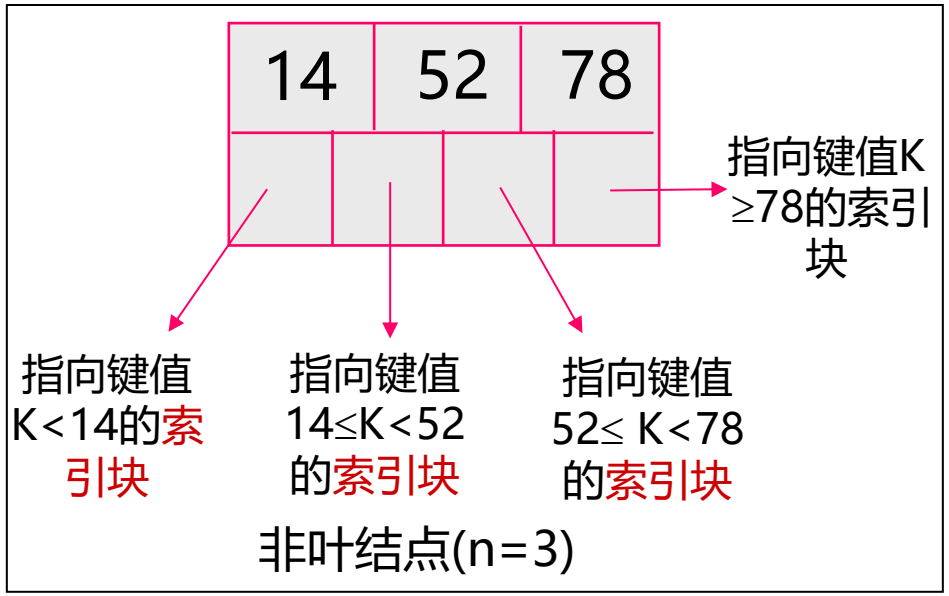
示例 存储块 = 4096 Byte
整数型索引字段值= 4 Byte
指针 = 8 Byte
则：n应满足 4(n-1)+8n<=4096
n取最大值
n=341

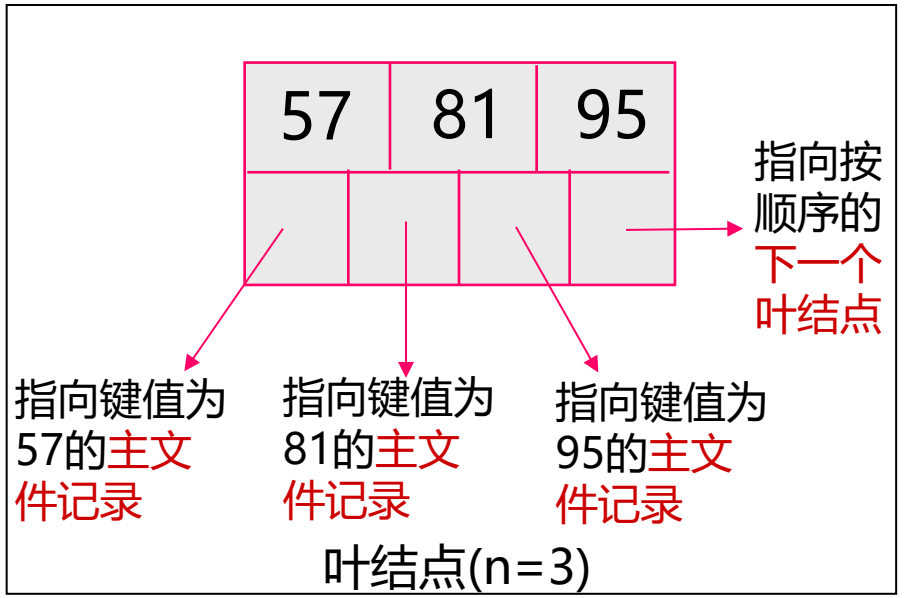
6.2 数据库索引-B+树(B+ Tree)

➢(3) B+树的存储约定

叶结点和非叶节点的指针指向什么?

- 非叶结点指针指向索引块，叶结点指针指向主文件的数据块或数据记录
- 叶结点的最后一个指针可能不同于它的其他指针---始终指向其下一个数据块





•一索引块实际使用的索引指针个数d，满足(根结点除外):

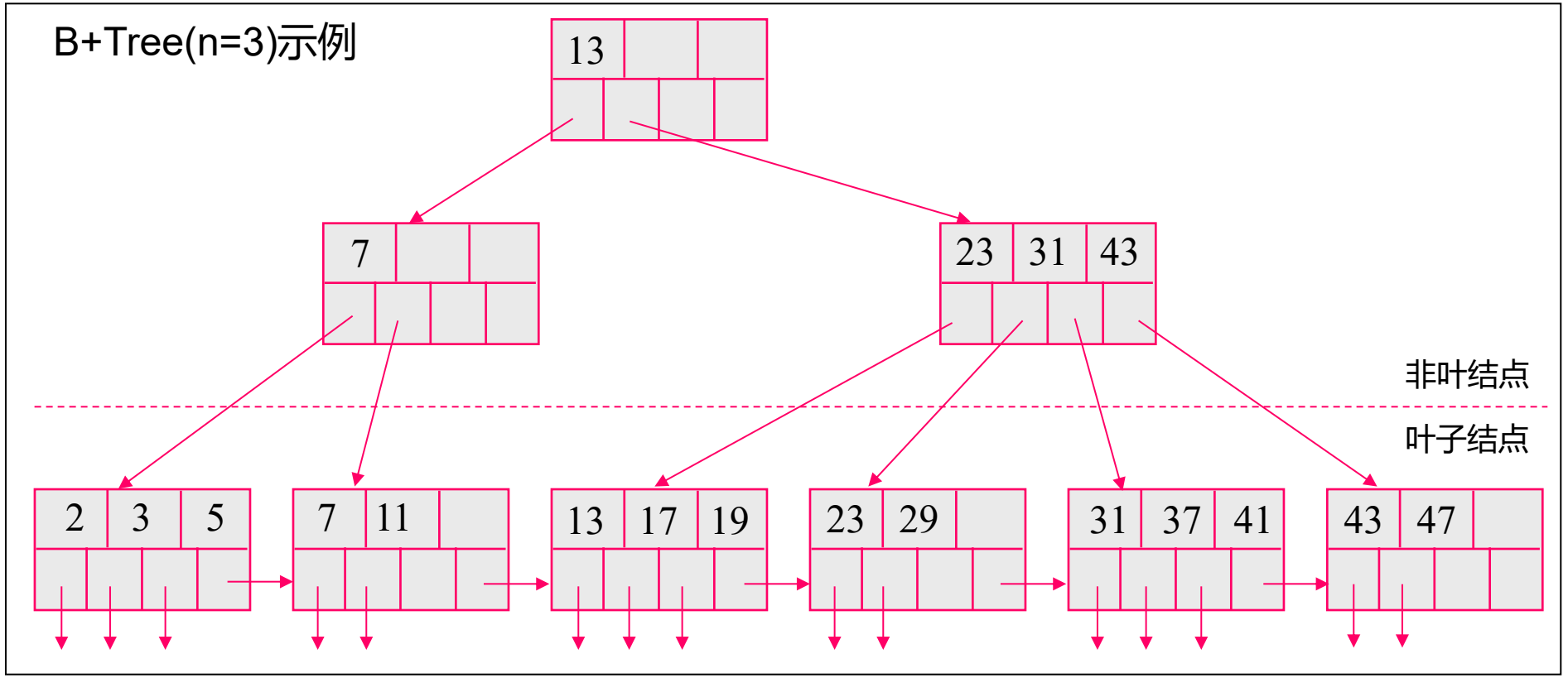
$n/2 \le d \le n$。

•根结点至少2个指针被使用。

6.2 数据库索引-B+树(B+ Tree)

➢(3) B+树的存储约定

索引字段值重复出现于叶结点和非叶结点；



仅叶结点块的集合就是主文件完整的索引，即：最低一级(或一层)索引

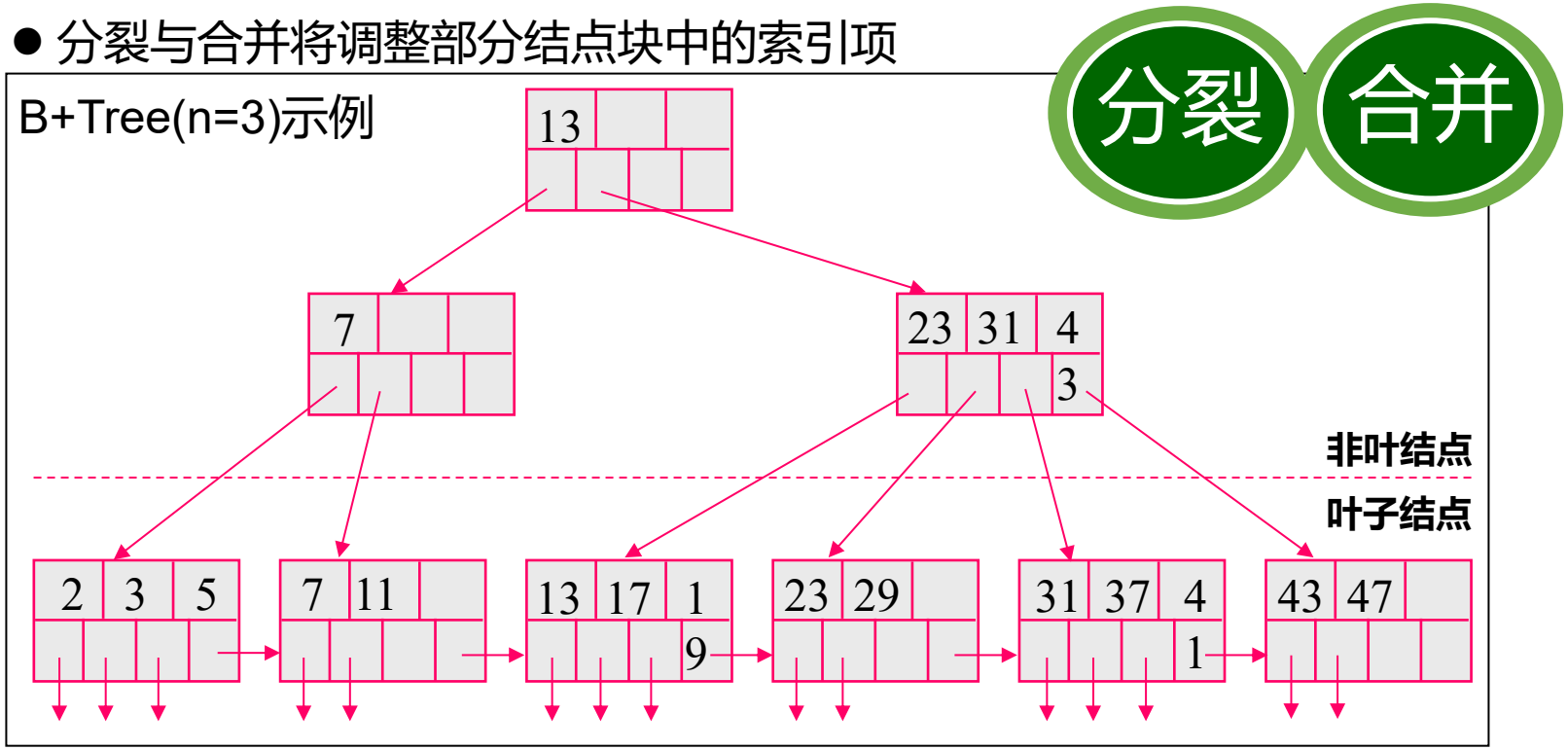
- 指向主文件的指针仅出现于叶结点；
- 所有叶结点即可覆盖所有键值的索引；
- 索引字段值在叶结点中是按顺序排列的；

6.2 数据库索引-B+树(B+ Tree)

➢(3) B+树的存储约定

级数(或层数)相同--平衡。如何保证?

- 插入/删除记录时，伴随着结点的分裂与合并;
- 分裂与合并将调整部分结点块中的索引项



分裂 合并

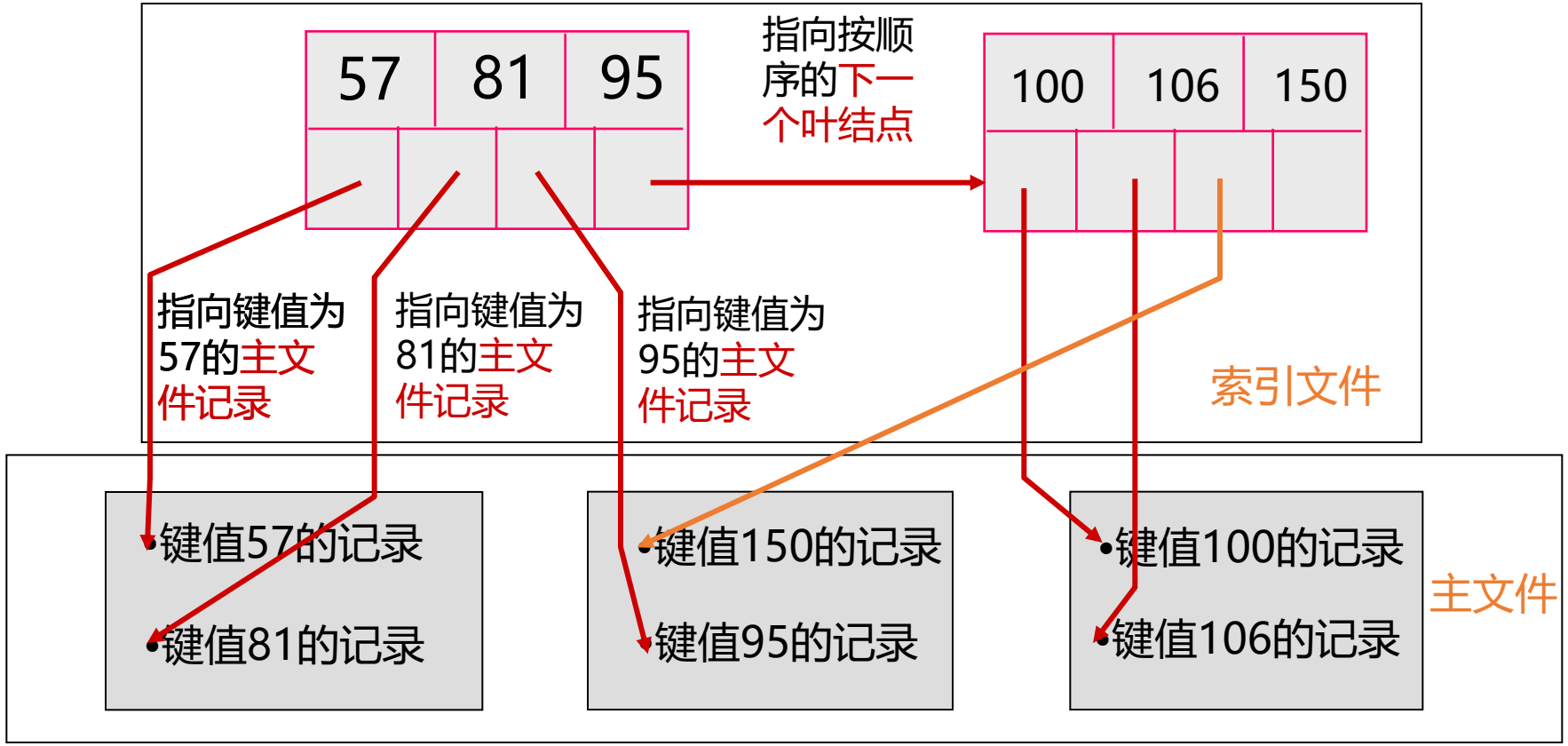
- 能够自动保持与主文件大小相适应的树的层次
- 每个索引块的指针利用率都在50%-100%之间

6.2 数据库索引-B+树(B+ Tree)

➢(4) 用B+树建立不同的索引

I. 用B+树建立键属性稠密索引

- 索引字段是主文件的主键，索引是稠密的。主文件可以按主键排序，也可以不按主键排序。指针指向的是记录。

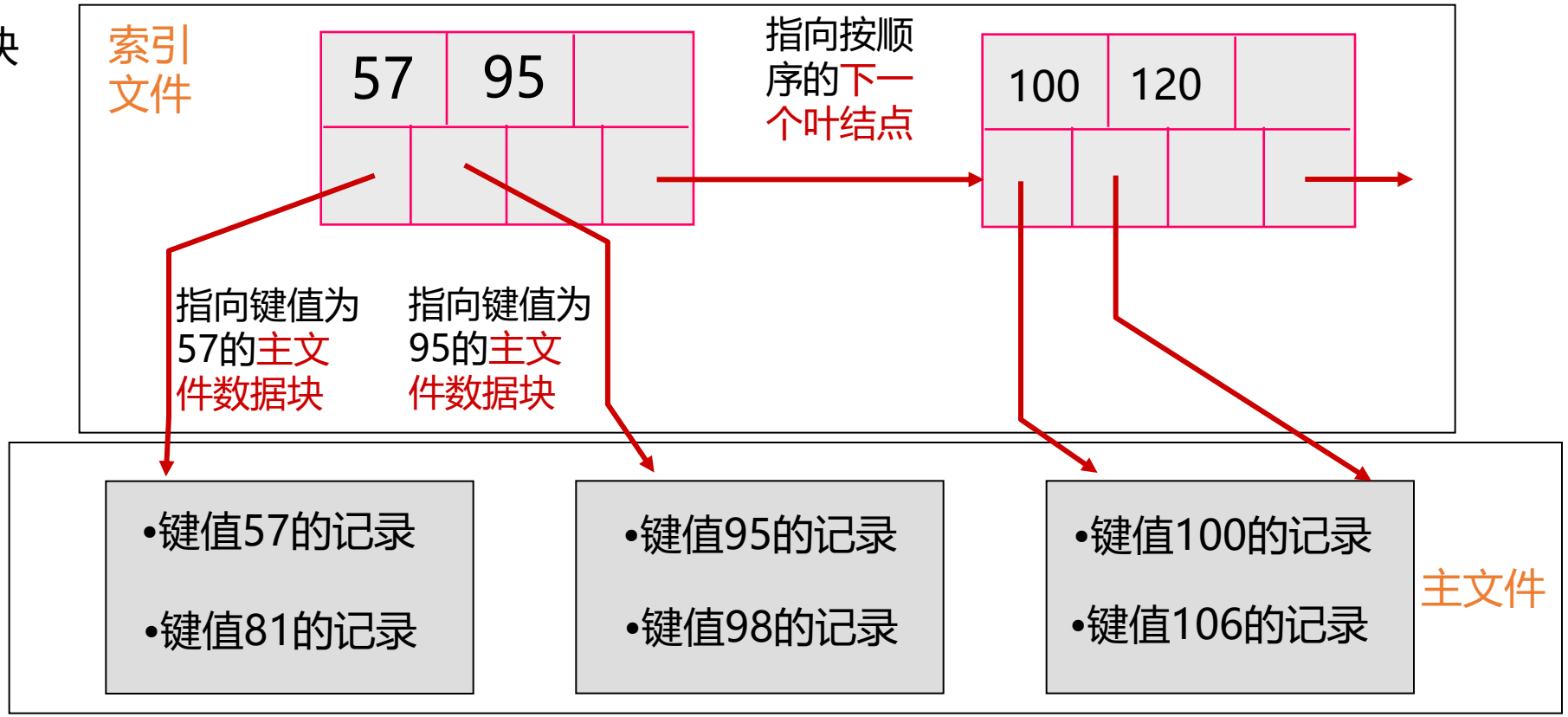


6.2 数据库索引-B+树(B+ Tree)

➢(4) 用B+树建立不同的索引

II. 用B+树建立键属性稠密索引

- 引字段是主文件的主键，索引是稠密的。主文件可以按主键排序，也可以不按主键排序。
- 指针指向的是数据块

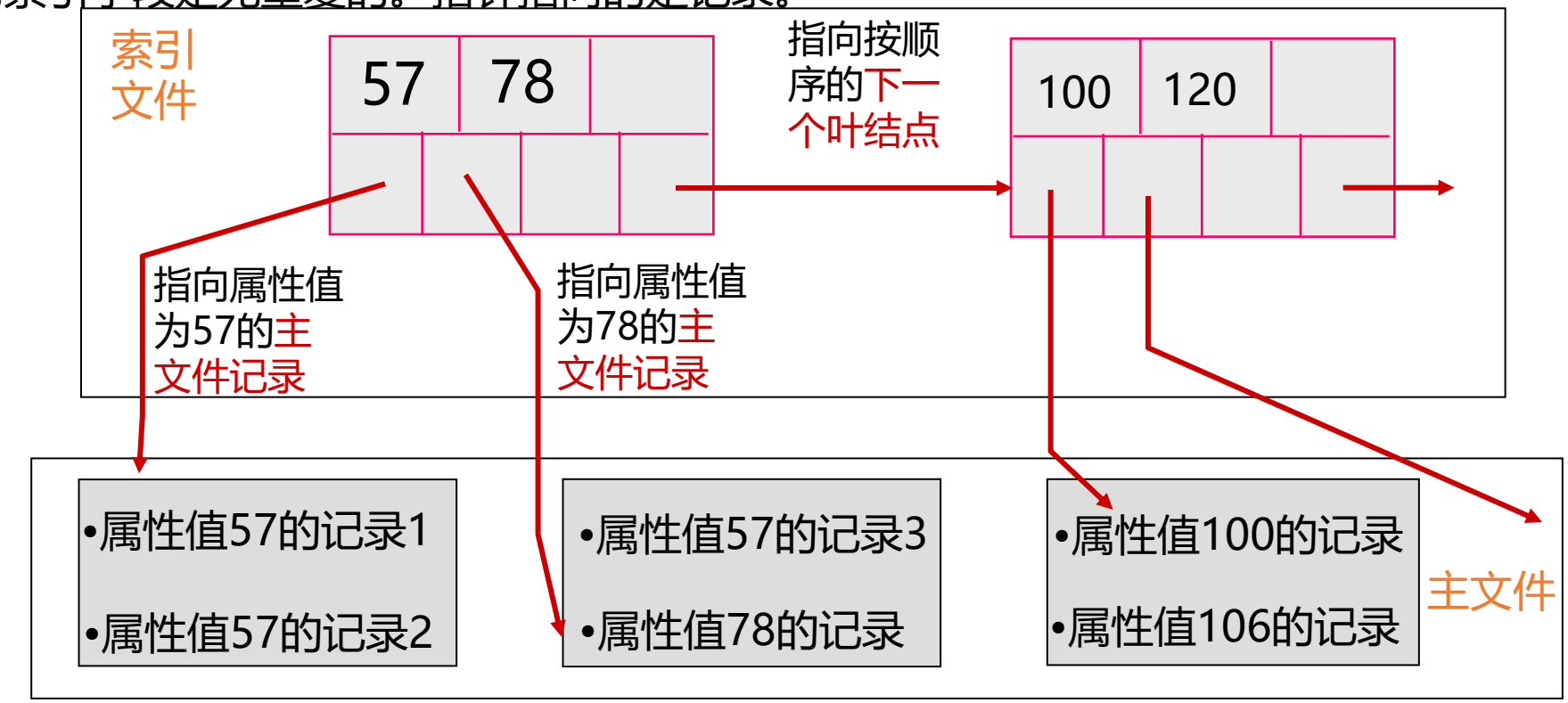


6.2 数据库索引-B+树(B+ Tree)

➢(4) 用B+树建立不同的索引

III. 用B+树建立非键属性稠密索引

- 索引字段是主文件的非键属性，索引是稠密的。主文件按非键属性排序
- 索引文件的索引字段是无重复的。指针指向的是记录。

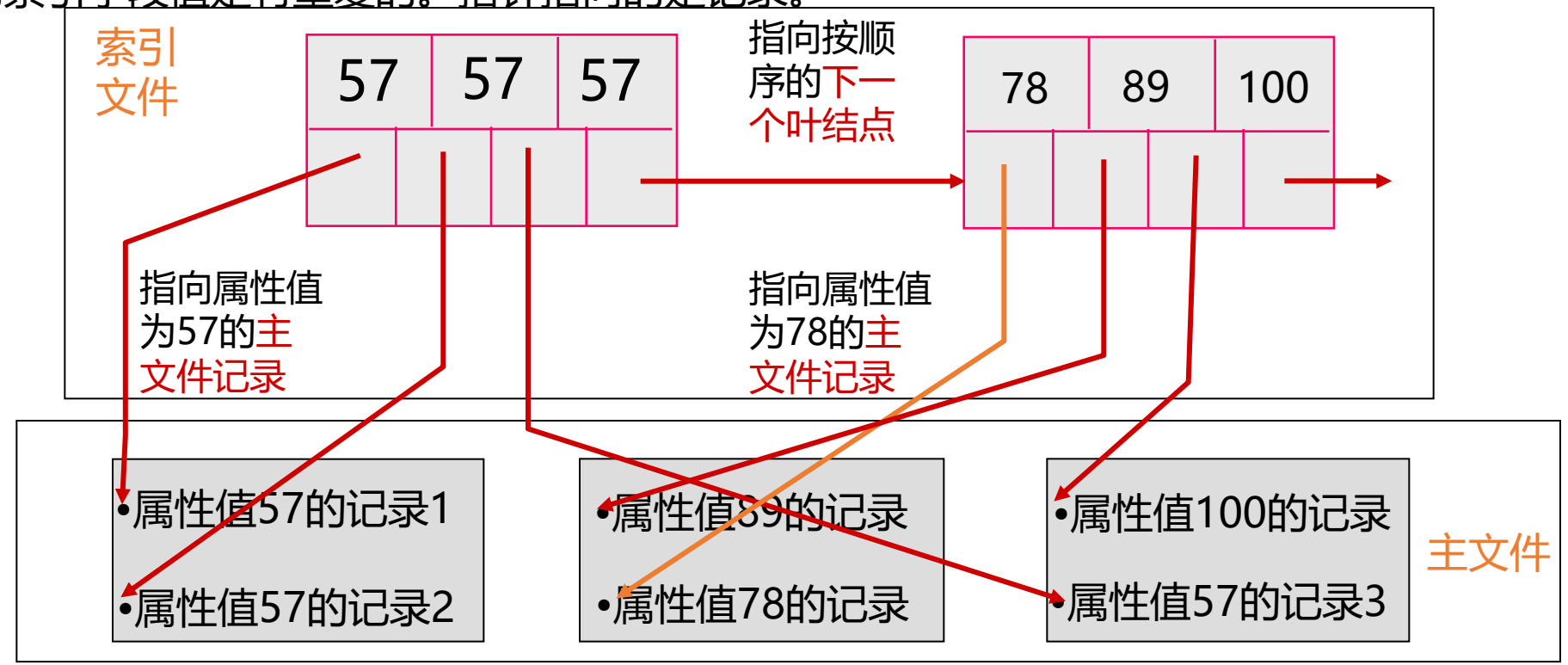


6.2 数据库索引-B+树(B+ Tree)

➢(4) 用B+树建立不同的索引

IV. 用B+树建立非键属性稠密索引

- 索引字段是主文件的非键属性。主文件不按此非键属性排序
- 索引文件的索引字段值是有重复的。指针指向的是记录。

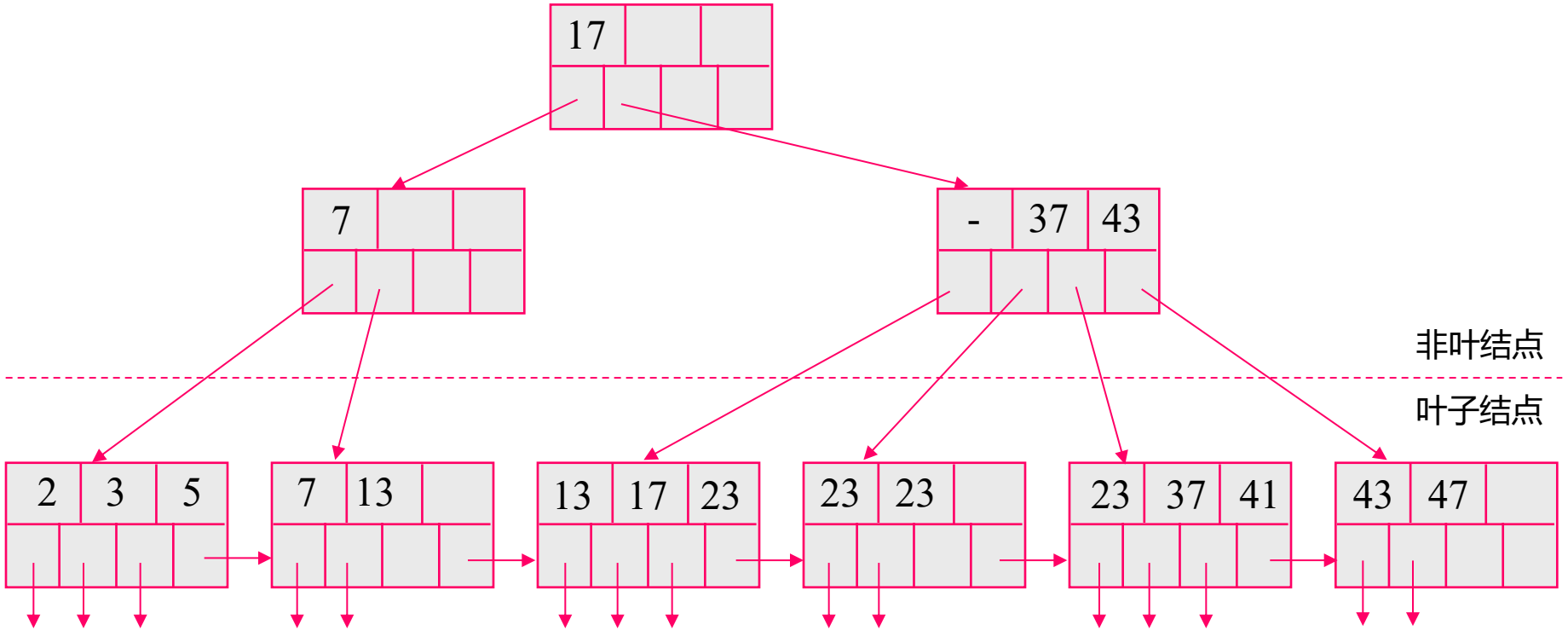


6.2 数据库索引-B+树(B+ Tree)

➢(4) 用B+树建立不同的索引

IV. 用B+树建立非键属性稠密索引

- 索引字段带有重复值的B+树的非叶结点示例

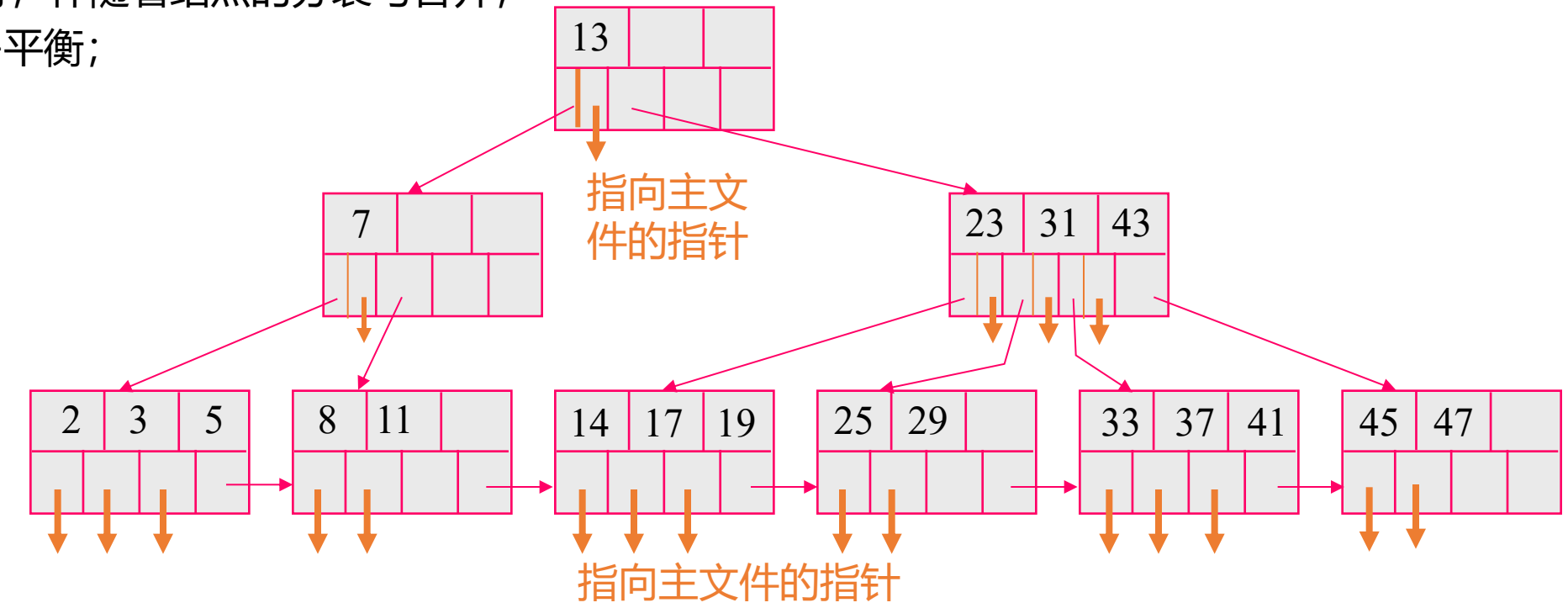


一棵带重复键值的B+Tree

6.2 数据库索引-B+树(B+ Tree)

➢(5) B Tree vs. B+ Tree

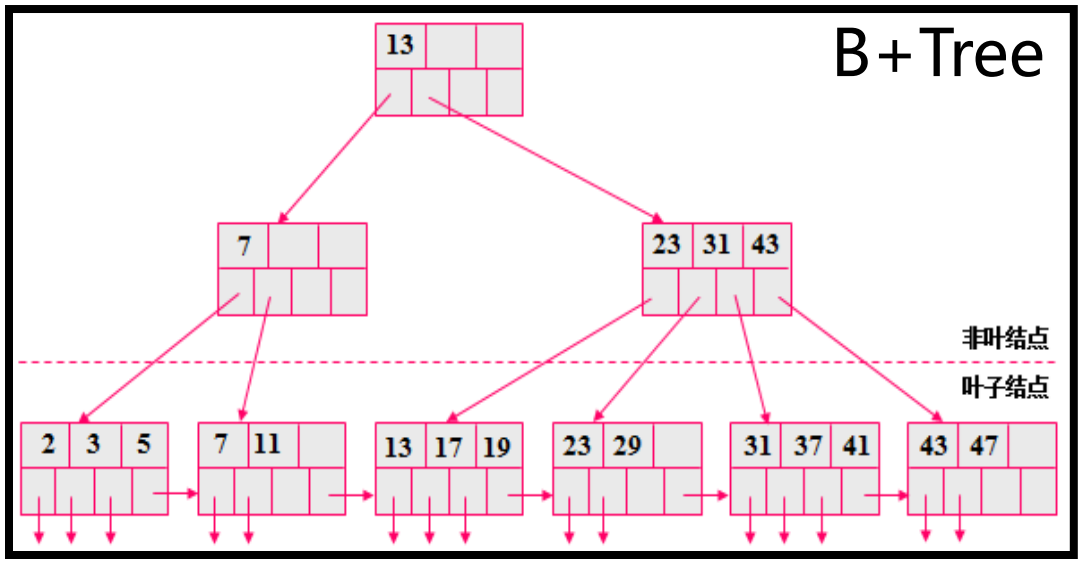
- 索引字段值仅出现一次或者在叶结点或者在非叶结点；
- 指向主文件的指针出现于叶结点或非叶结点；
- 所有结点才能覆盖所有键值的索引。
- 插入记录时，伴随着结点的分裂与合并；
- 层数相同--平衡；

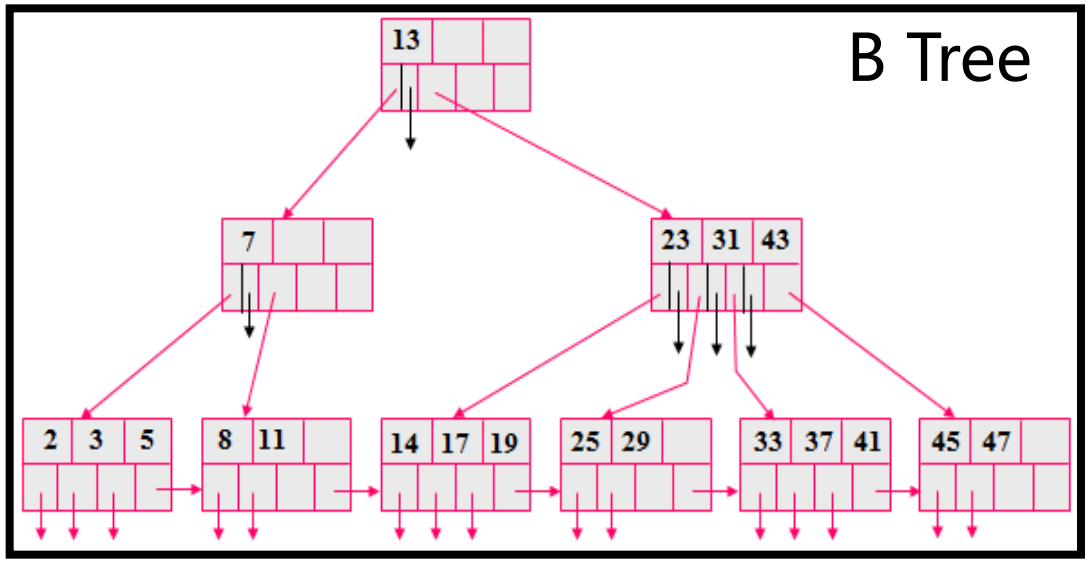


6.2 数据库索引-B+树(B+ Tree)

➢(5) B Tree vs. B+ Tree

- 一块中存放的索引项个数是否相同?
- 索引字段值都出现在哪里?
- 指向主文件的指针存在哪里?
- 分裂与合并的方法是否一致?





6.2 数据库索引-B+树(B+ Tree)

➢(6) B+树还要考虑什么

利用B+树检索的算法

```
function find( value V )
/*假设没有重复码返回叶结点C和索引i使得C.P_i指向第一条搜索码值等于V的记录*/
    置C=根结点
    while C不是叶结点 begin
        令i=满足V≤C.K_i的最小值
        if 没有这样的i then begin
            令P_n=结点中最后一个非空指针
            置C=C.P_n
        end
            else if (V=C.K_i)
                then 置C=C.P_{i+1}
            else C=C.P_i/*V<C.K_i*/
        end
        /*C是叶结点*/
        设i是满足K_i=V的最小值
        if 有这样的i存在
            then 返回(C, i)
            else 返回空/*没有码值等于V的记录存在*/

procedure printAll( value V)
/*打印所有搜索码值等于V的记录*/
    置done=false
    置(L, i)=findFirst(V)
    if((L, i)为空)return
    repeat
        repeat
        打印指针L.P_i指向的记录
        置i=i+1
    until(i>L中码的数目或L.K_i>V)
    if(i>L中码的数目)
        then L=L.P_n
        else 置done=true;
    until (done or L为空)
```

利用B+树增加

```
procedure insert( value K, pointer P)
    if (树为空) 创建一个空叶结点L,
    else 找到应该包含值K的叶结点L
    if (L所含搜索码少于n-1个)
        then insert_in_leaf (L, K, P)
        else begin /*L已经含有n-1个搜索码了，分裂L*/
            创建结点L'
            把L.P_1, …, L.K_{n-1}复制到可以存储n个(指针，搜索码值)对的内存块T中
            insert_in_leaf (T, K, P)
            令L'.P_n=L.P_n; 令L.P_n=L'
            从L中删除L.P_1, …, L.P_{n-1}, L.K_{n-1}, L.P_n
            把T.P_1, …, T.K_{n/2}从T复制到L中，L以L.P_1作为开始
            把T.K_{n/2}, …, T.K_n从T复制到L'中，L'以L'.P_1作为开始
            令K'表示L'中最小搜索码值
proced
    if (K比L.K_1小)
                将P复制到可以存放P以及(K', N')的内存块T中
                创建结点(K', N')插入T中，紧跟在N后面
                删除所有P中所有项；创建结点P'
                把T.P_1, …, T.K_{n/2}复制到P中
令K"=T.K_{n/2}
把T.P_{n/2+1}, …, T.P_{n+1}复制到P'中
insert_in_parent (P, K", P')
    end
```

算法可参见教材

利用B+树删除记录的算法

```
procedure delete (value K, pointer P)
    找到包含(K, P)的叶结点L
    delete_entry(L, K, P)

procedure delete_entry(node N, value K, pointer P)
    从N中删除(K, P)
    if (N是根结点 and N只剩下一个子结点)
    then 使N的子结点成为该树的新的根结点并删除N
    else if (N中值/指针太少) then begin
        令N'为parent(N)的前一子女或后一子女
        令K'为parent(N)中指针N和N'之间的值
        if (N和N'中的项能放在一个结点中)
        then begin /*合并结点*/
            if (N是N'的前一个结点) then swap_variables(N, N')
            if (N不是叶结点)
                then 将K'以及N中所有指针和值附加到N'中
                else 将N中所有(K_i, P_i)对附加到N'中; 令N'.P_n=N.P_n
            delete_entry (parent(N), K', N); 删除结点N
        in /*重新分布：从N'借一个索引项*/
            N'是N的前一个结点) then begin
                点) then begin
                N'.P_n是N'的最后一个指针
                除(N'.K_{n-1}, N'.P_n)
                _n, K') 并通过将其他指针和值右移使之成为
                N中的第一个指针和值
            用N'.K_{n-1}替换parent(N)中的K'
        end
        else begin
            令m满足：(N'.P_m, N'.K_m)是N'中的最后一个指针/值对
            从N'中去除(N'.P_m, N'.K_m)
            插入(N'.P_m, N'.K_m)并通过将其他指针和值右移使之成为
                N中的第一个指针和值
            用N'.K_m替换parent(N)中的K'
        end
    end
    else …与then的情况对称…
    end
end procedure
```

如何利用B+树进行检索?

如何利用B+树进行增加和删除操作?

B+树的增加和删除操作时如何进行分裂和合并?

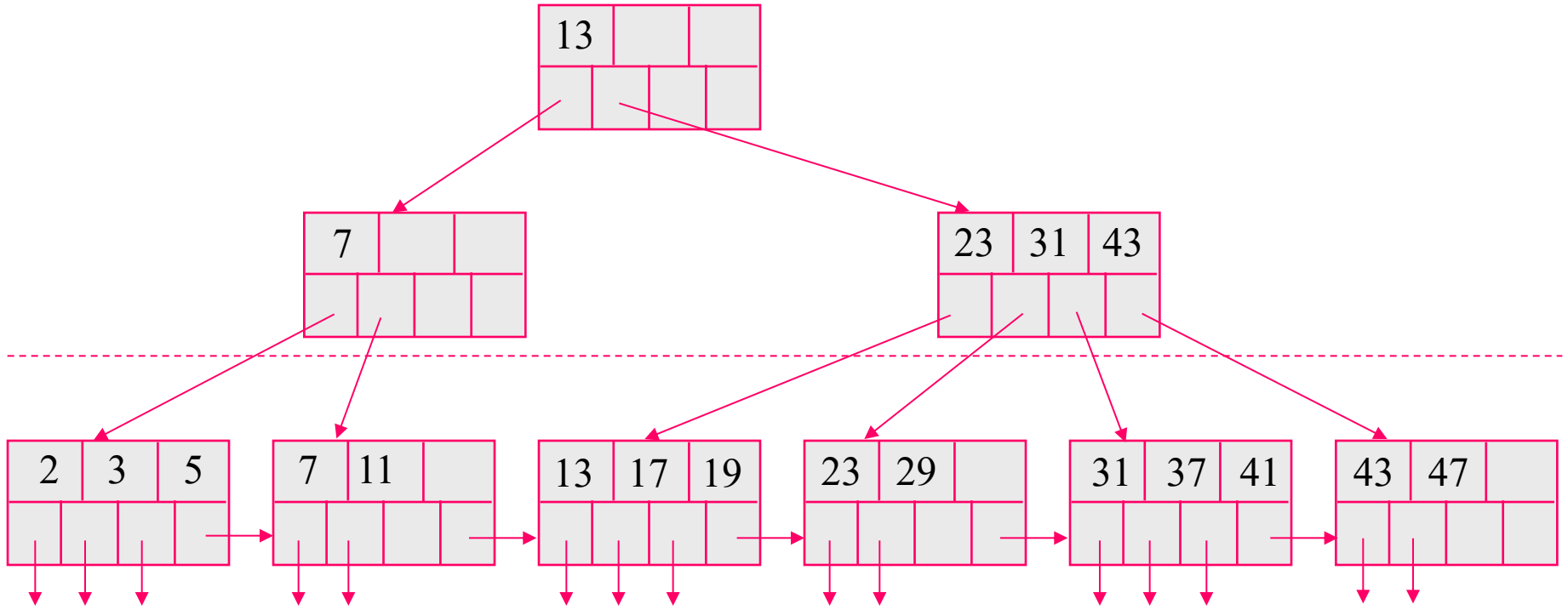
什么条件下分裂与合并?

分裂与合并时如何调整B+树的指针?

6.2 数据库索引-B+树键值插入与结点分裂过程示意

➢(1) 问题

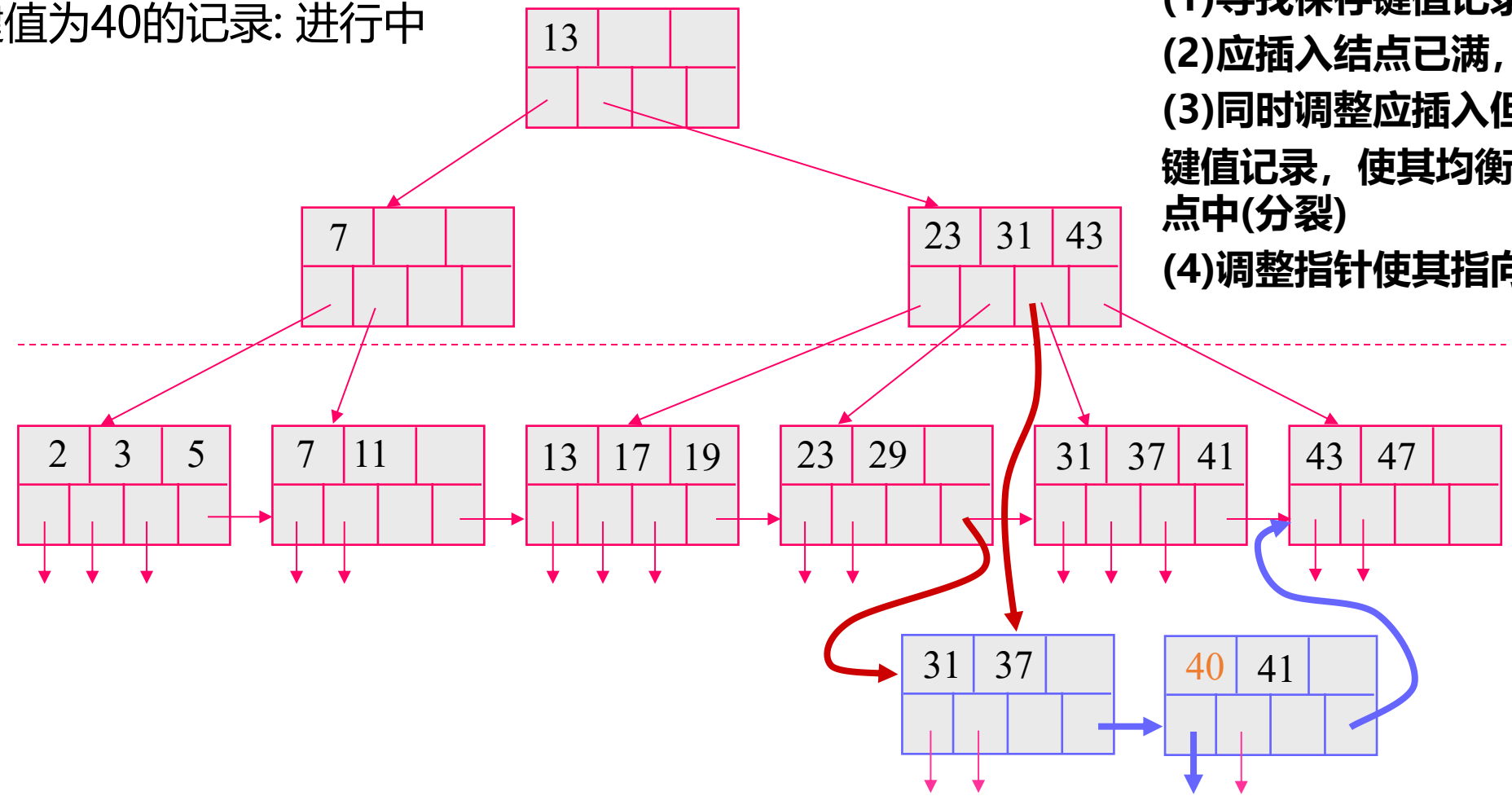
问题1：在下列B+树中插入键值为40的记录



6.2 数据库索引-B+树键值插入与结点分裂过程示意

➢(2) 插入过程及分裂过程示意

插入键值为40的记录: 进行中

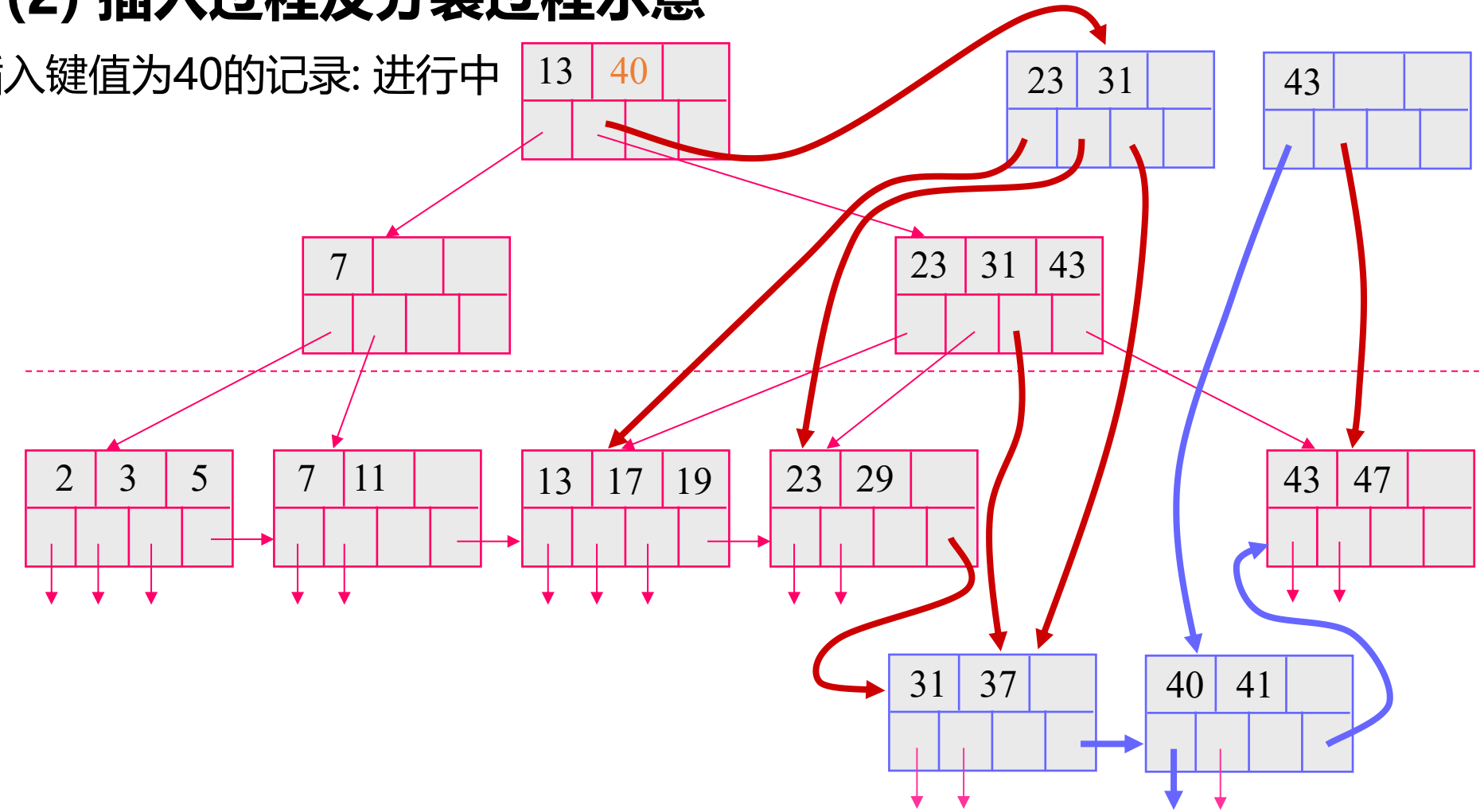


(1)寻找保存键值记录的叶子结点
(2)应插入结点已满，则申请新结点
(3)同时调整应插入但未插入结点中的键值记录，使其均衡存放于两个叶结点中(分裂)
(4)调整指针使其指向新叶子结点

6.2 数据库索引-B+树键值插入与结点分裂过程示意

➢(2) 插入过程及分裂过程示意

插入键值为40的记录: 进行中



(5)寻找指向新叶子结点的非叶结点

(6)应插入结点已满，则申请新结点

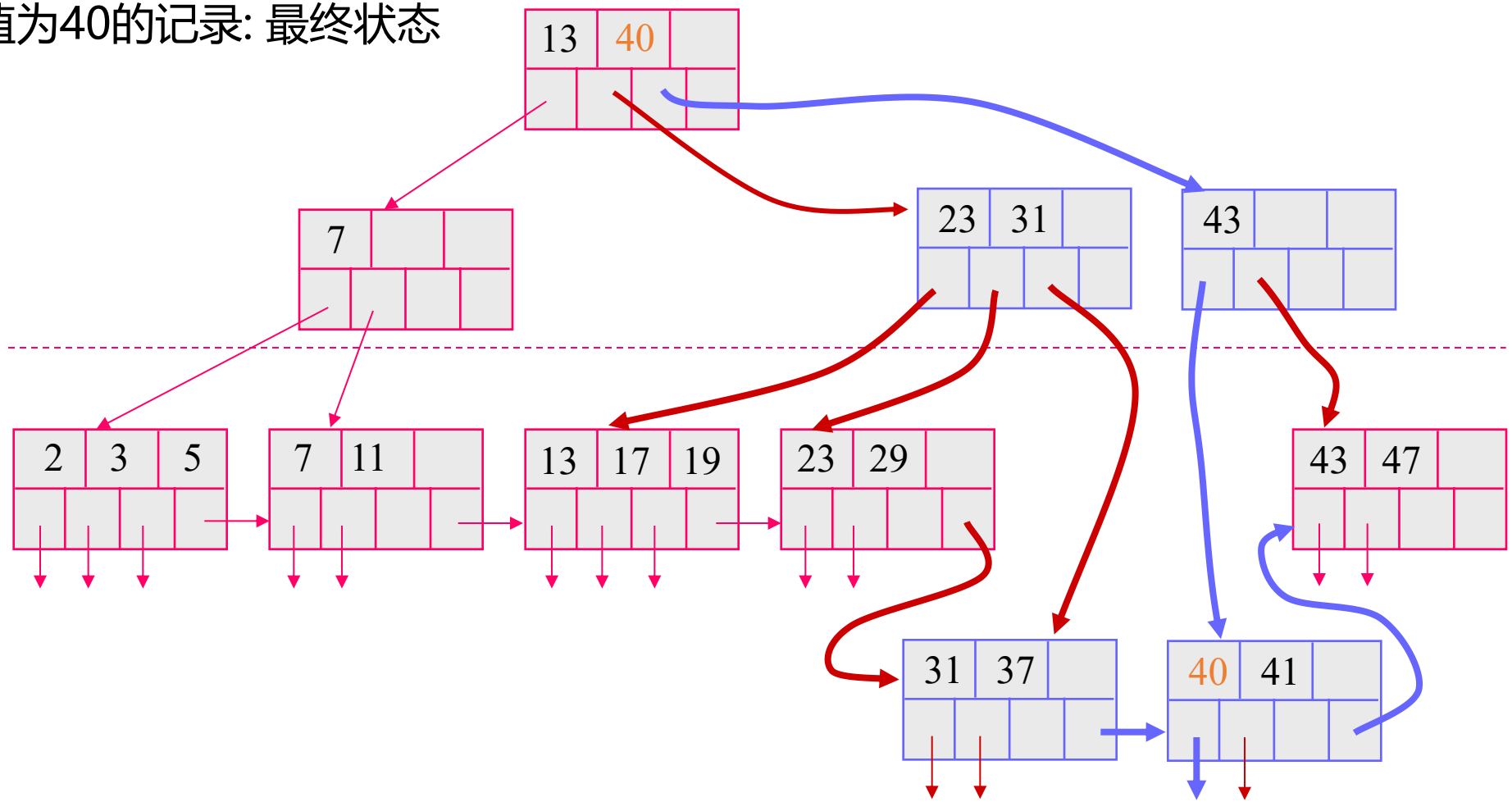
(7)同时调整应插入但未插入结点的键值记录，使其均衡存放于两个非叶结点中(分裂)

(8)调整各结点指针使其指向正确

6.2 数据库索引-B+树键值插入与结点分裂过程示意

➢(3) 插入及分裂完成后

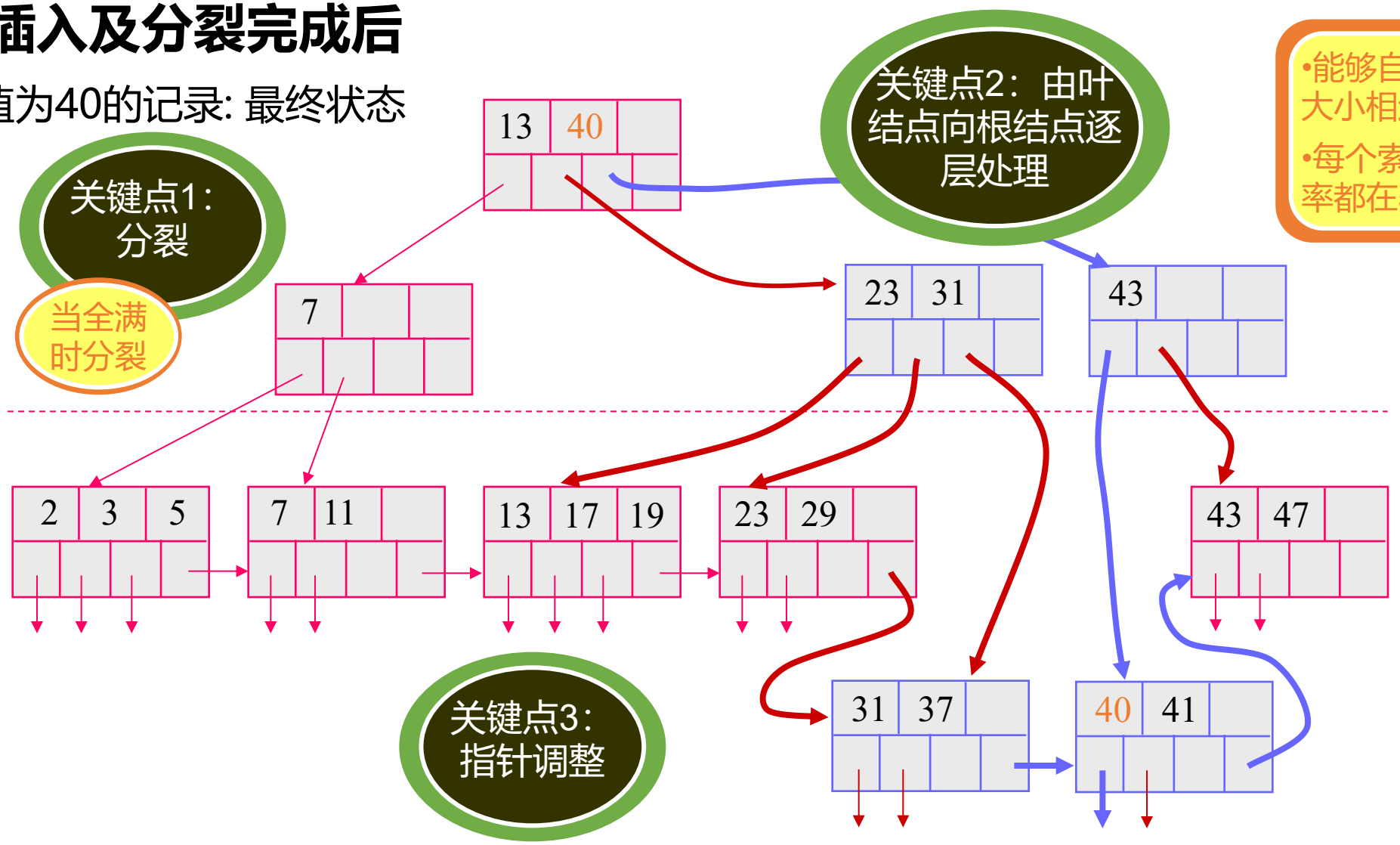
插入键值为40的记录: 最终状态



6.2 数据库索引-B+树键值插入与结点分裂过程示意

➢(3) 插入及分裂完成后

插入键值为40的记录: 最终状态



- 能够自动保持与主文件大小相适应的树的层次
- 每个索引块的指针利用率都在50%-100%之间

6.2 数据库索引-B+树键值插入与结点分裂过程示意



➢(4) 相关的算法表达

```
procedure insert(value K, pointer P)
    if (树为空) 创建一个空叶结点 L, 同时它也是根结点
    else 找到应该包含值 K 的叶结点 L
    if (L 所含搜索码少于 n-1 个)
        then insert_in_leaf (L, K, P)
        else begin  /* L 已经含有 n-1 个搜索码了, 分裂 L */
            创建结点 L'
            把 L.P1, …, L.K(n-1) 复制到可以存储 n 个(指针, 搜索码值)对的内存块 T 中
            insert_in_leaf (T, K, P)
            令 L'.Pn = L.Pn; 令 L.Pn = L'
            从 L 中删除 L.P1, …, L.P(n-1), L.K(n-1), L.Pn
            把 T.P1, …, T.K(⌈n/2⌉) 从 T 复制到 L 中, L 以 L.P1 作为开始
            把 T.K(⌈n/2⌉), …, T.Kn 从 T 复制到 L'中, L'以 L'.P1 作为开始
            令 K'表示 L'中最小搜索码值
            insert_in_parent (L, K', L')
        end

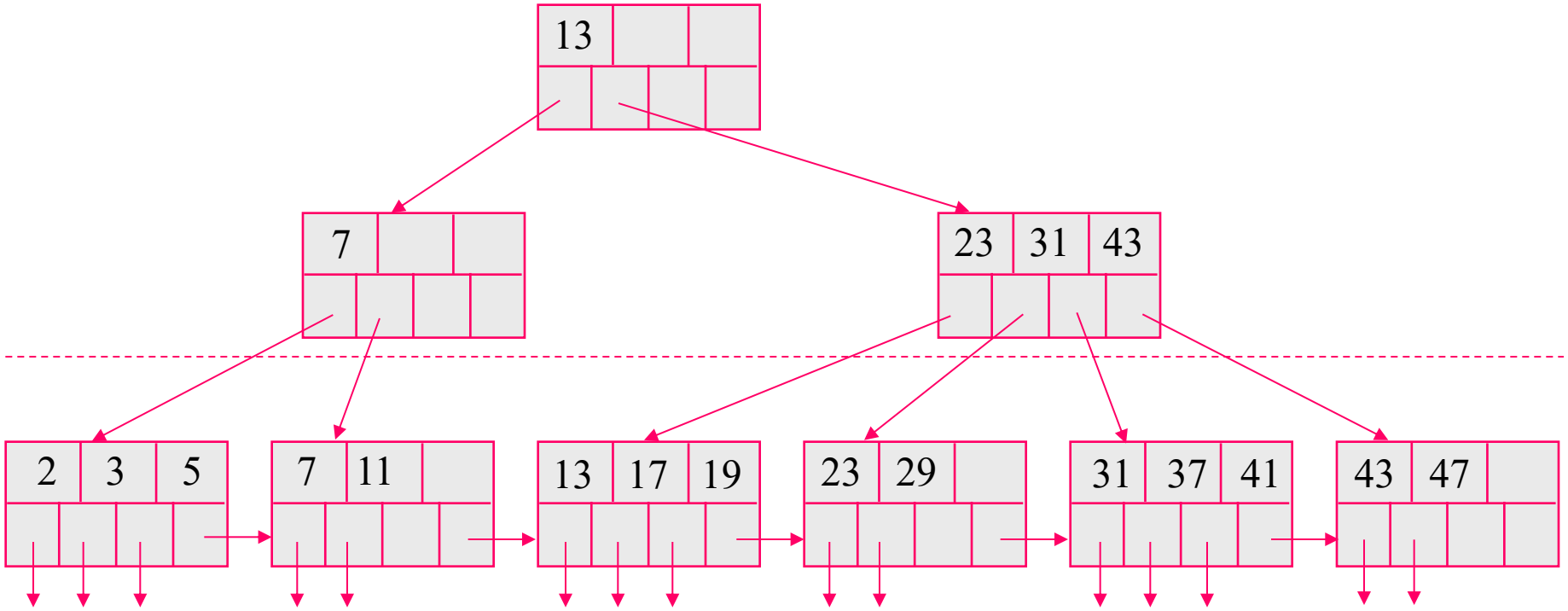
procedure insert_in_leaf (node L, value K, pointer P)
    if (K 比 L.K1 小)
        then 把 P、K 插入 L 中, 紧接在 L.P1 前面
        else begin
            令 Ki 表示 L 中小于等于 K 的最大搜索码值
            把 P、K 插入 L 中, 紧跟在 L.Ki 前面
        end
procedure insert_in_parent (node N, value K', pointer N')
    if (N 是树的根结点)
        then 在 L 中插入(V, P)
        else begin
            创建结点 R 包含 N、K'、N'  /* N 和 N'都是指针 */
            令 R 成为树的根结点
            return
        end
令 P=N 的父结点
if (P 包含的指针少于 n 个)
    then 将(K', N')插到 N 后面
    else begin /* 分裂 P */
            将 P 复制到可以存放 P 以及(K', N')的内存块 T 中
            创建结点(K', N')插入 T 中, 紧跟在 N 后面
            删除所有 P 中所有项; 创建结点 P'
            把 T.P1, …, T.K(⌈n/2⌉) 复制到 P 中
令 K"= T.K(⌈n/2⌉)
把 T.P(⌈n/2⌉+1), …, T.P(n+1)复制到 P'中
insert_in_parent (P, K", P')
    end
```

此算法可参见教材

6.2 数据库索引-B+树键值删除与结点合并过程示意

➢(1) 问题

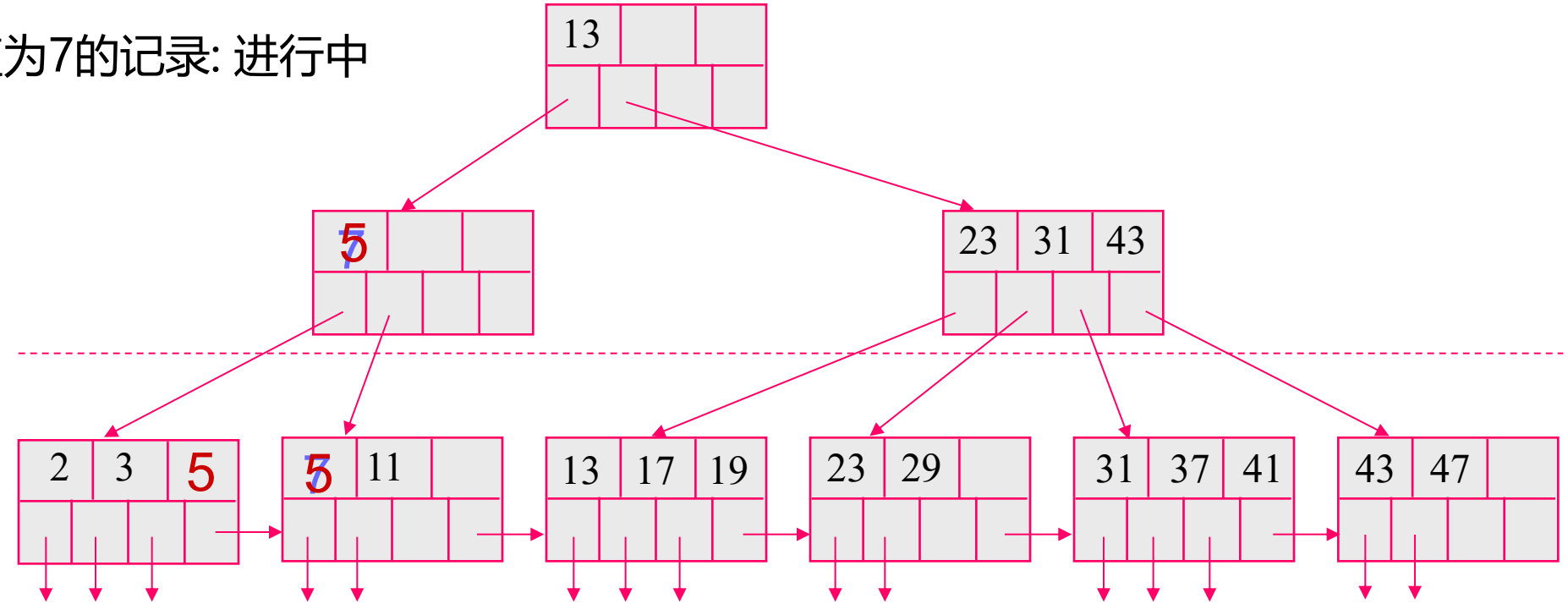
问题1：在下列B+树中删除键值为7的记录



6.2 数据库索引-B+树键值删除与结点合并过程示意

➢(2) 删除过程与合并过程示意

删除键值为7的记录: 进行中

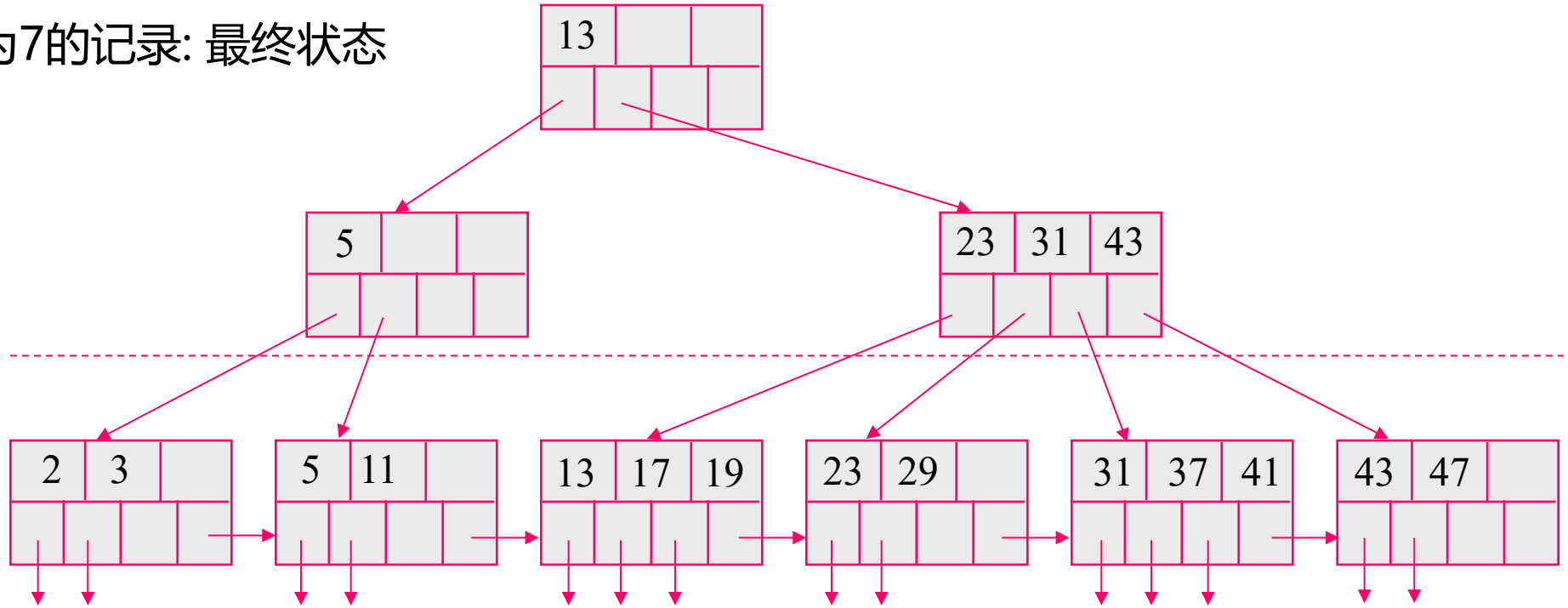


(1)叶子结点中寻找等于键值的记录，删除相应的指针及主文件中对应的记录
(2)调整其左侧(或右侧)结点及本结点中的键值记录, 使其均衡存放于两个叶结点中
(3)如有调整，则进一步调整其上层非叶结点，重新确定其键值，以满足大于等于键值的记录都在其右侧

6.2 数据库索引-B+树键值删除与结点合并过程示意

➢(2) 删除过程与合并过程示意

删除键值为7的记录: 最终状态

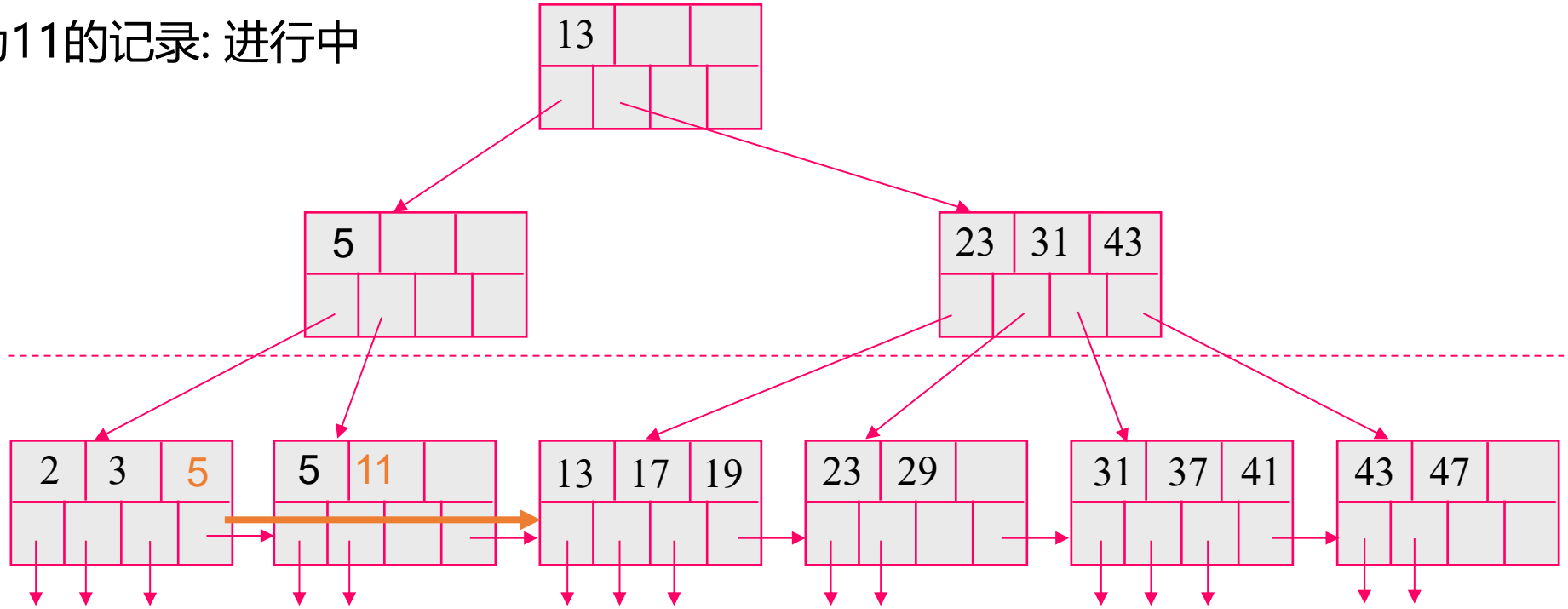


问题2：在此B+树中删除键值为11的记录

6.2 数据库索引-B+树键值删除与结点合并过程示意

➢(2) 删除过程与合并过程示意

删除键值为11的记录: 进行中

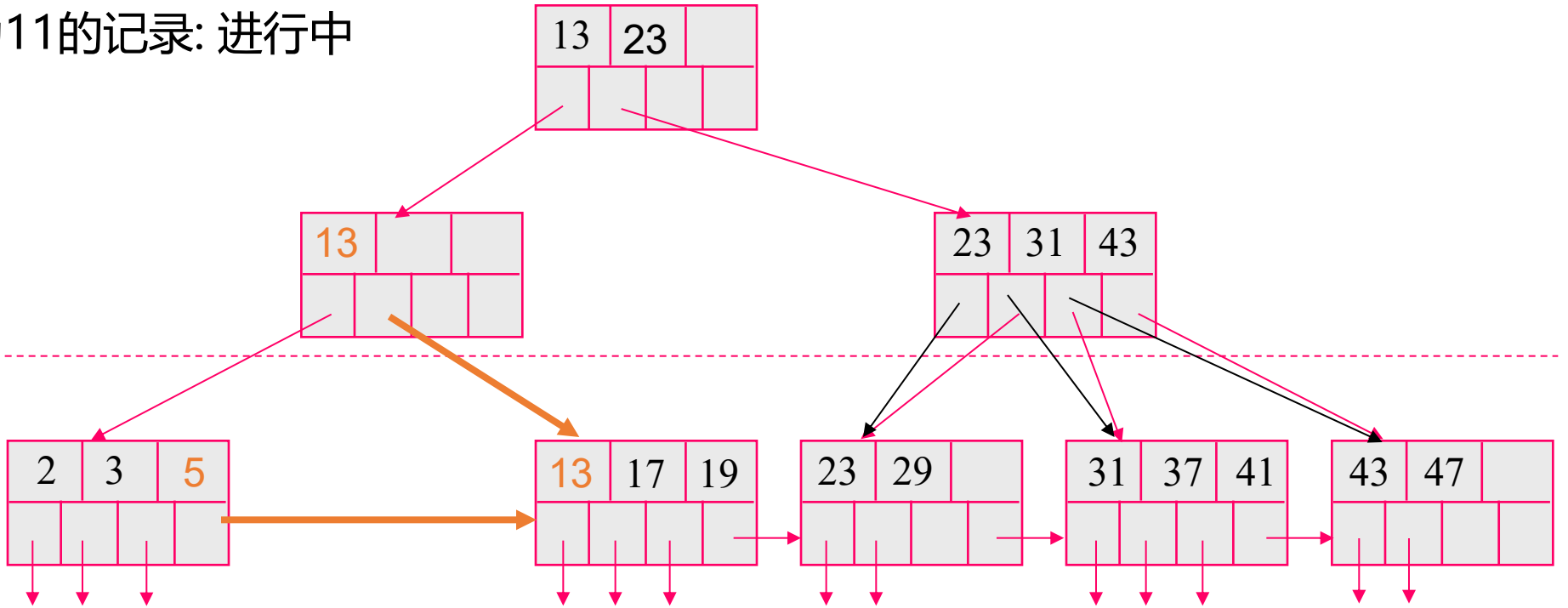


(1)叶子结点中寻找等于键值的记录，删除相应的指针及主文件中对应的记录
(2)将本结点的剩余键值, 移动到左侧(或右侧)结点。此空结点将被删除。调整叶结点指针

6.2 数据库索引-B+树键值删除与结点合并过程示意

➢(2) 删除过程与合并过程示意

删除键值为11的记录: 进行中

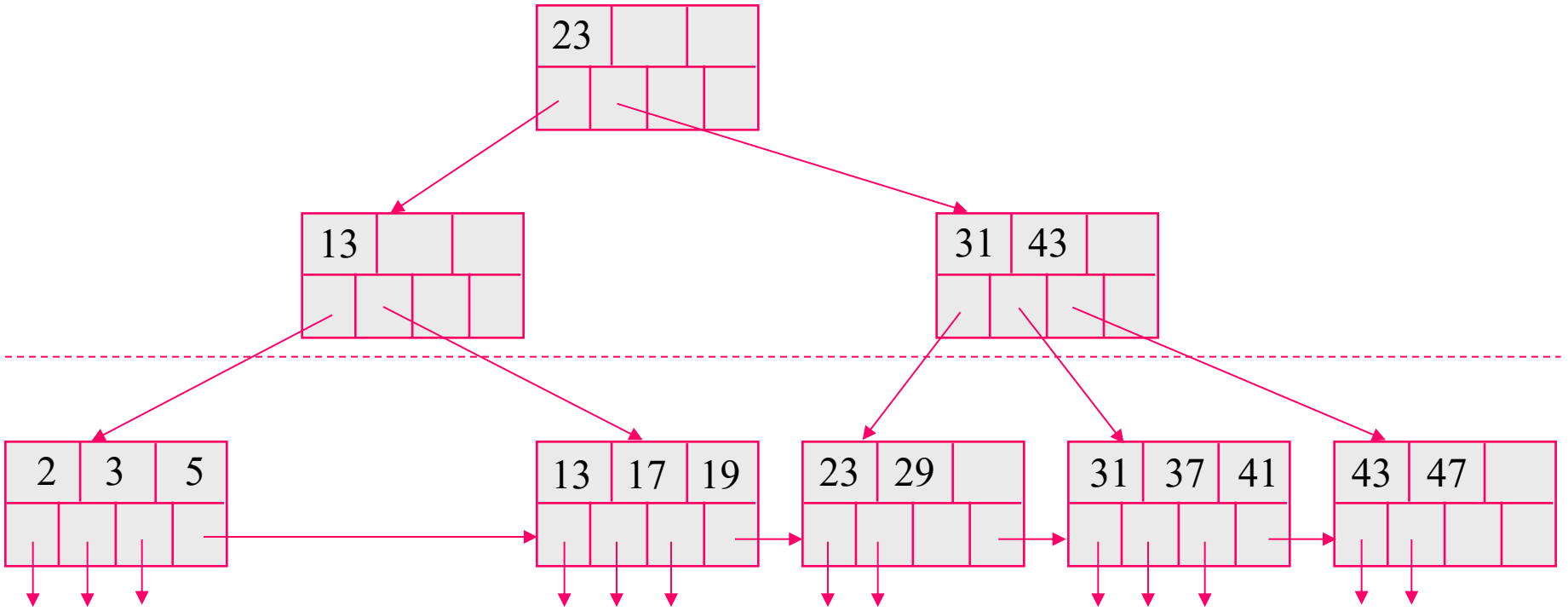


(3)如有调整，则进一步调整其上层非叶结点，重新确定其键值，以满足大于等于键值的记录都在其右侧。依次向上调整，直到根结点。

6.2 数据库索引-B+树键值删除与结点合并过程示意

➢(3) 删除过程与合并过程完成之后

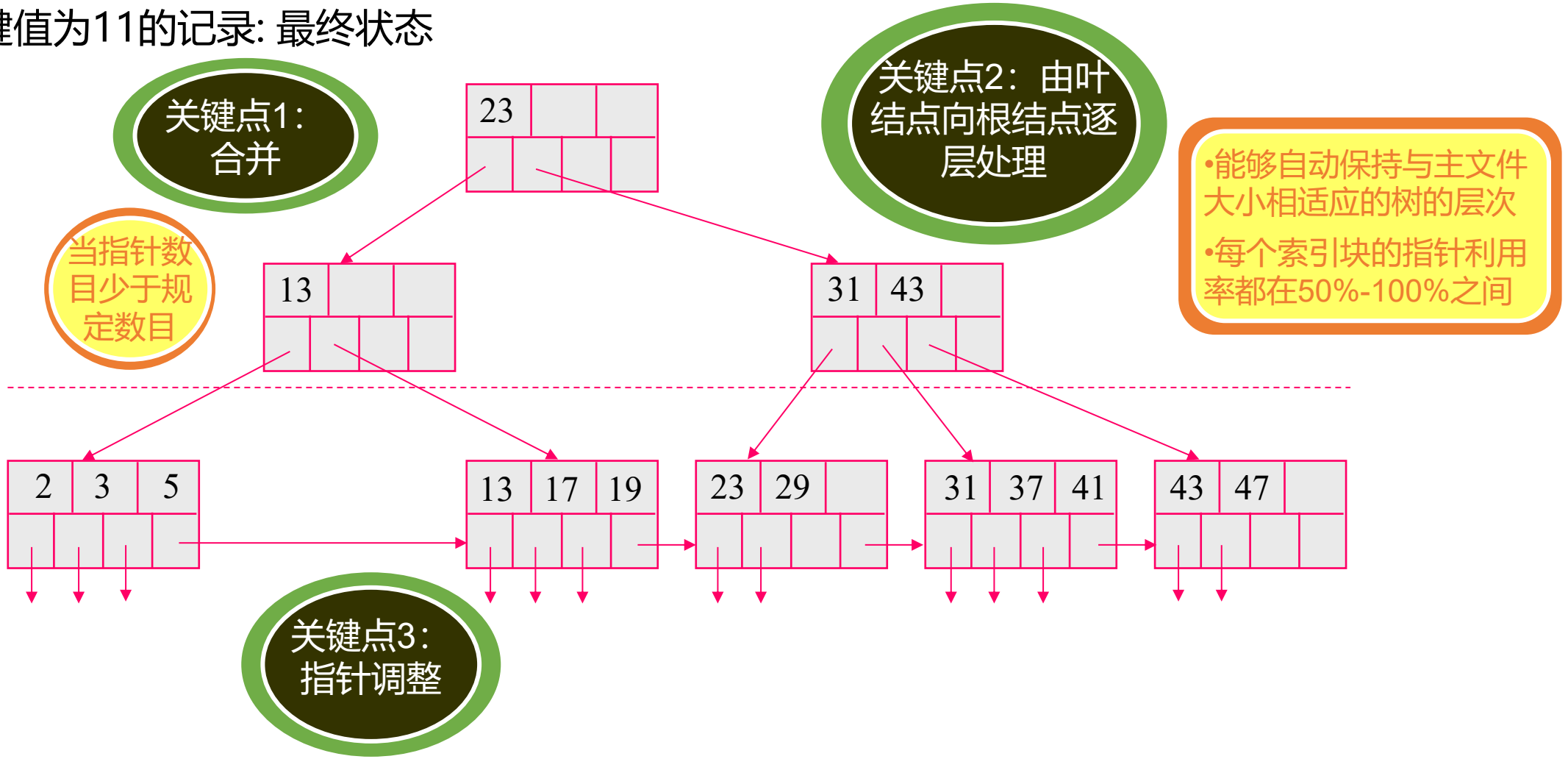
删除键值为11的记录: 最终状态



6.2 数据库索引-B+树键值插入与结点分裂过程示意

➢(3) 删除过程与合并过程完成之后

删除键值为11的记录: 最终状态



6.2 数据库索引-B+树键值插入与结点分裂过程示意



➢(4) 相关的算法表达

```
procedure delete (value K, pointer P)
    找到包含( K, P)的叶结点 L
    delete_entry( L, K, P)

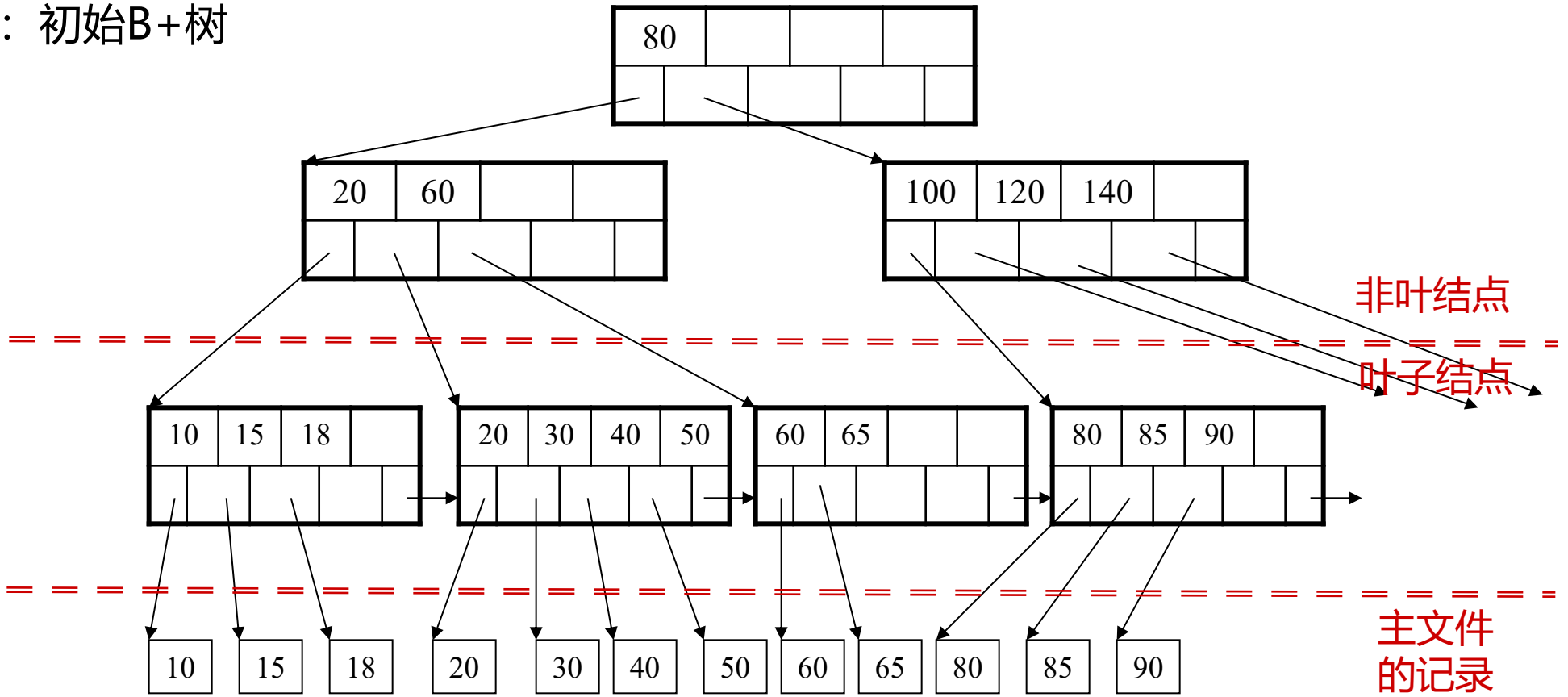
procedure delete_entry(node N, value K, pointer P)
    从 N 中删除(K, P)
    if (N 是根结点 and N 只剩下一个子结点)
    then 使 N 的子结点成为该树的新的根结点并删除 N
    else if (N 中值/指针太少)  then begin
        令 N'为 parent(N)的前一子女或后一子女
        令 K'为 parent(N)中指针 N 和 N'之间的值
        if (N 和 N'中的项能放在一个结点中)
        then begin  /* 合并结点 */
            if  (N 是 N'的前一个结点) then swap_variables(N, N')
            if  (N 不是叶结点)
                then 将 K'以及 N 中所有指针和值附加到 N'中
                else 将 N 中所有( K_i, P_i)对附加到 N'中; 令 N'.P_n = N.P_n
            delete_entry (parent(N), K', N); 删除结点 N
        end
    else begin /* 重新分布: 从 N'借一个索引项 */
        if ( N'是 N 的前一个结点) then begin
            if (N 是非叶结点) then begin
                令 m 满足: N'.P_m 是 N'的最后一个指针
                从 N'中去除(N'.K_m-1, N'.P_m)
                插入(N'.P_m, K')并通过将其他指针和值右移使之成为
                    N 中的第一个指针和值
                用 N'.K_m-1 替换 parent(N)中的 K'
            end
            else begin
                令 m 满足: (N'.P_m, N'.K_m)是 N'中的最后一个指针/值对
                从 N'中去除(N'.P_m, N'.K_m)
                插入( N'.P_m, N'.K_m) 并通过将其他指针和值右移使之成为
                    N 中的第一个指针和值
                用 N'.K_m 替换 parent (N)中的 K'
            end
        end
        else  …与 then 的情况对称…
    end
  end
end procedure
```

此算法可参见教材

6.2 数据库索引-B+树分裂与合并过程再示例

➢(1) 背景

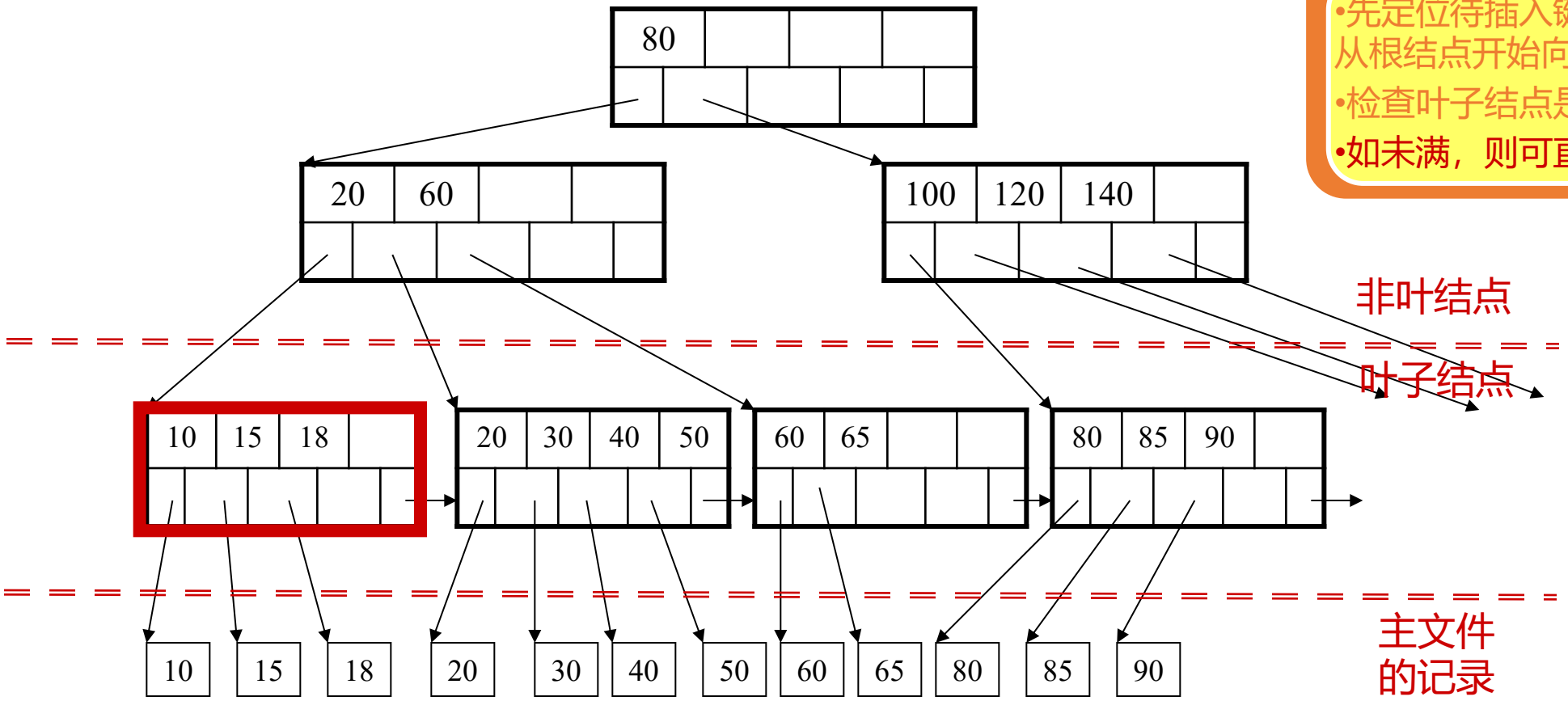
示例：初始B+树



6.2 数据库索引-B+树分裂与合并过程再示例

➢(2) 简单插入

问题：插入键值为19的记录

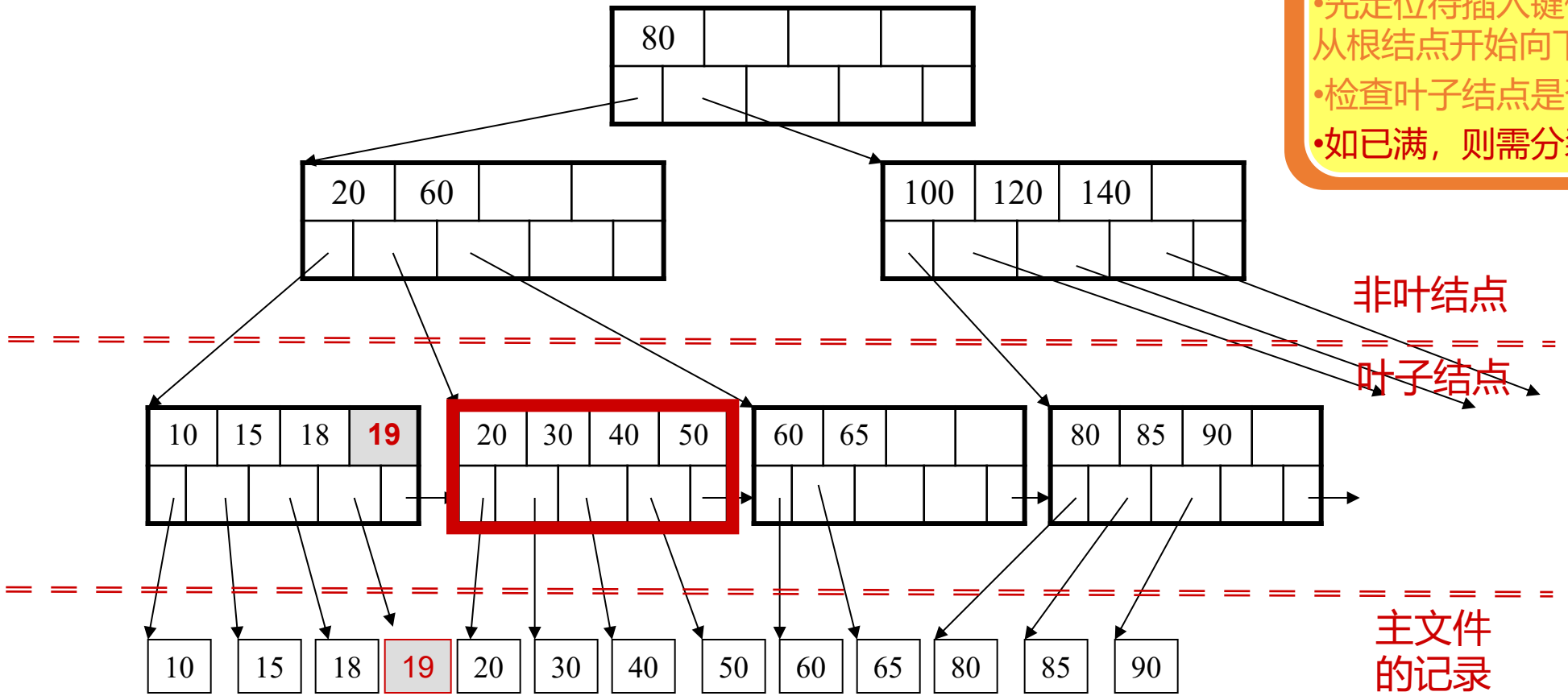


- 先定位待插入键值的叶子结点：从根结点开始向下；
- 检查叶子结点是否已满？
- 如未满，则可直接插入

6.2 数据库索引-B+树分裂与合并过程再示例

➢(3) 复杂插入

键值为19的记录插入后，问题：再插入键值为25的记录



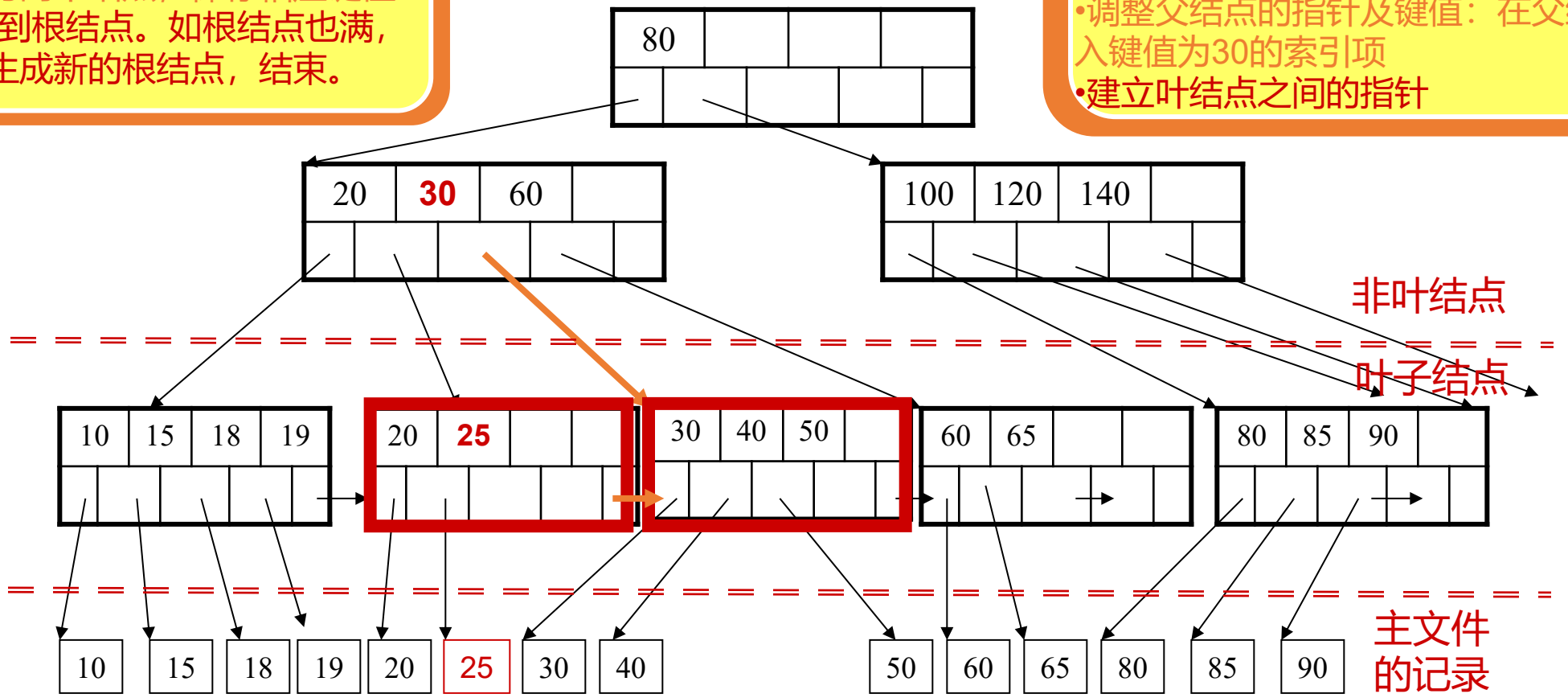
- 先定位待插入键值的叶子结点：从根结点开始向下；
- 检查叶子结点是否已满？
- 如已满，则需分裂该结点为两个

6.2 数据库索引-B+树分裂与合并过程再示例

➢(4) 分裂条件与分裂过程

•如父结点已满，则如此继续将其分裂为两个结点，保存相应键值
•一直到根结点。如根结点也满，则再生成新的根结点，结束。

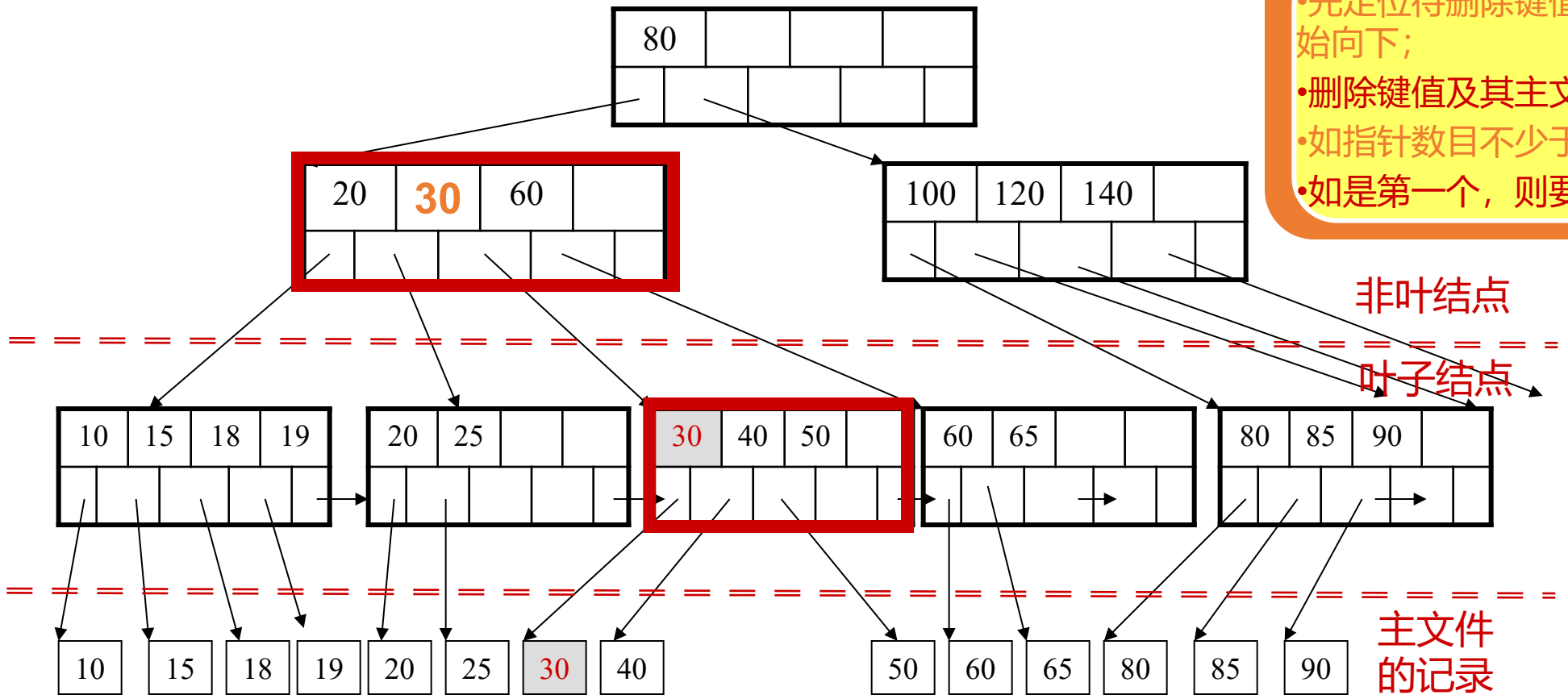
•分裂结点，将所涉及的键值 准均分为两个结点中存储(包含待插入的)
•插入键值为25的键值并指向主文件记录
•调整父结点的指针及键值：在父结点中插入键值为30的索引项
•建立叶结点之间的指针



6.2 数据库索引-B+树分裂与合并过程再示例

➢(5) 简单删除

键值为25的记录插入后，问题：删除键值为30的记录

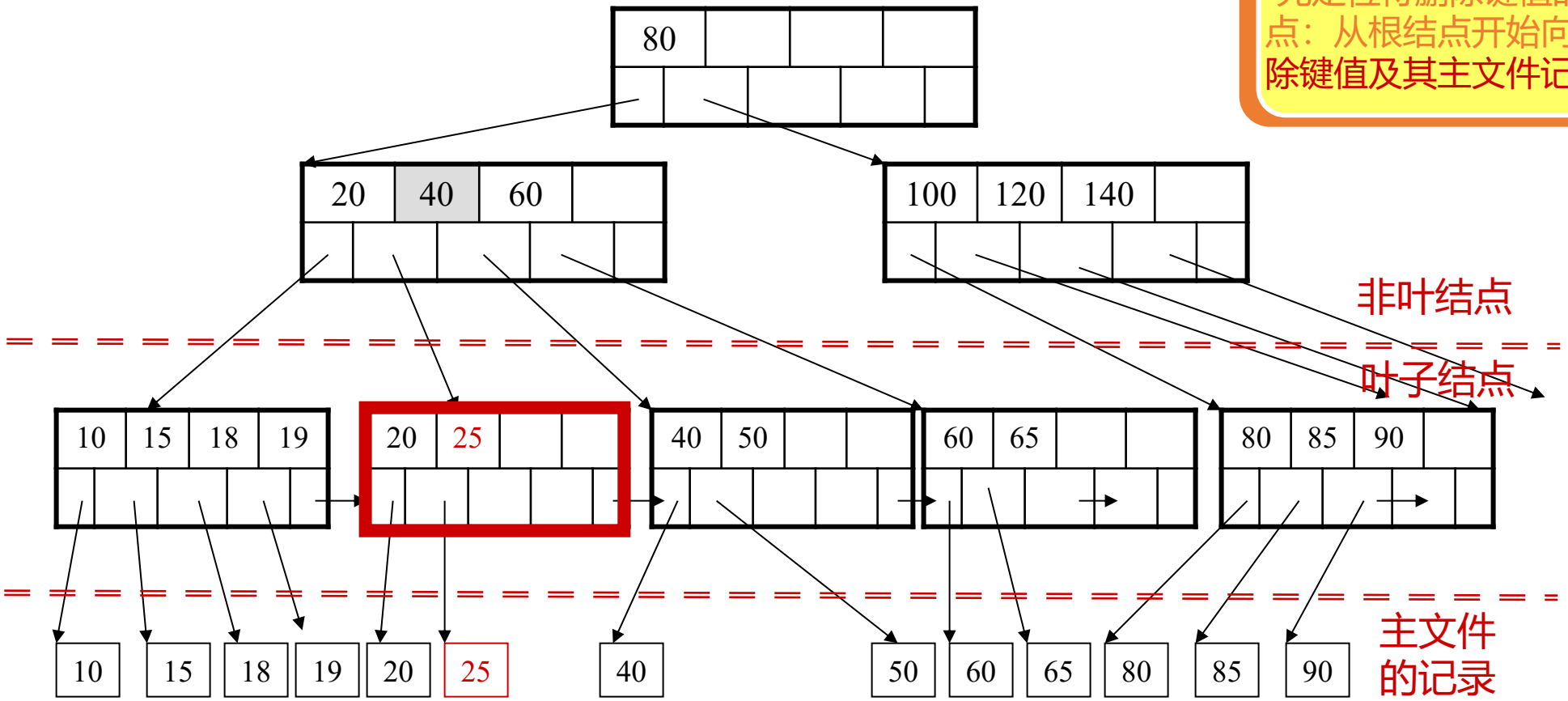


- 先定位待删除键值的叶子结点：从根结点开始向下；
- 删除键值及其主文件记录？
- 如指针数目不少于规定数目，则可以结束
- 如是第一个，则要更新父结点的相应键值

6.2 数据库索引-B+树分裂与合并过程再示例

➢(6) 复杂删除

键值为30的记录删除后，问题：删除键值为25的记录



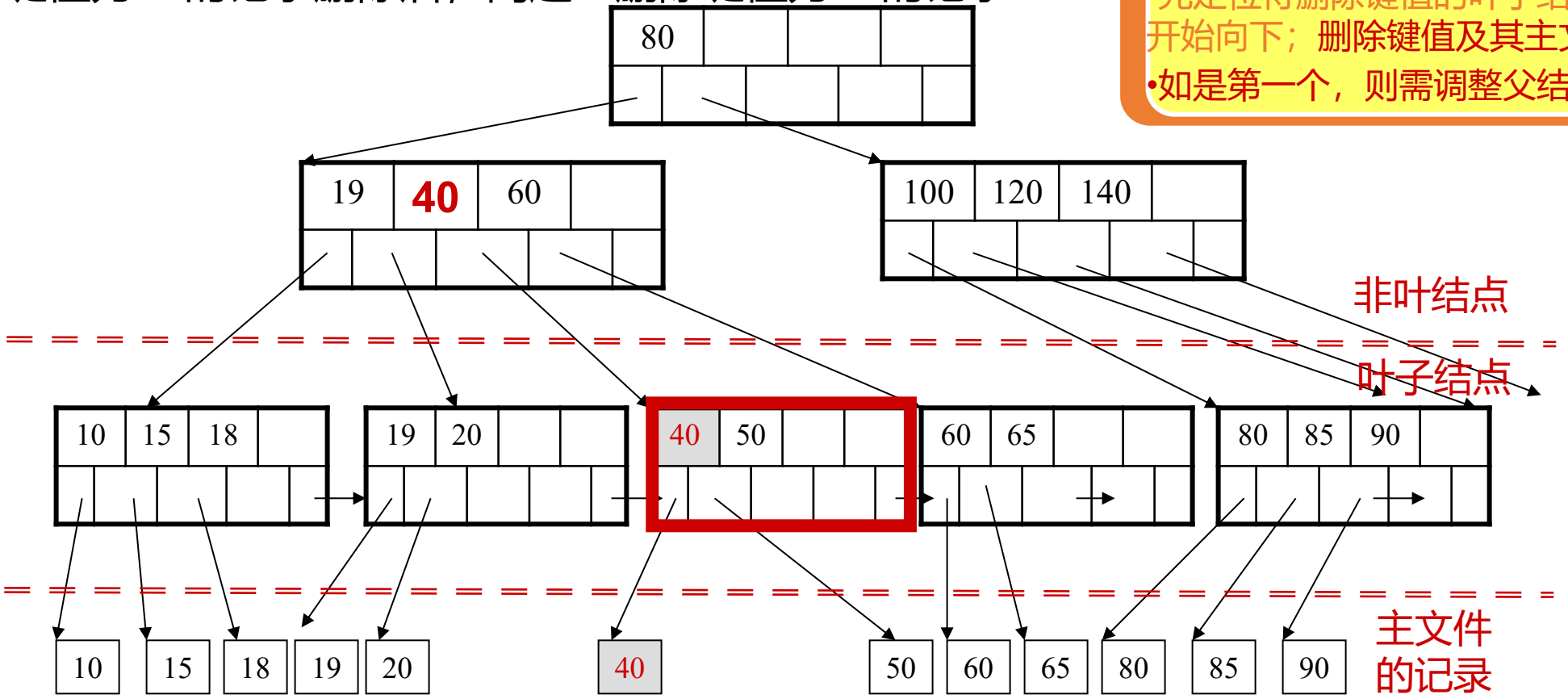
•先定位待删除键值的叶子结点：从根结点开始向下；删除键值及其主文件记录?

6.2 数据库索引-B+树分裂与合并过程再示例

➢(7) 键值之合并

键值为25的记录删除后，问题：删除键值为40的记录

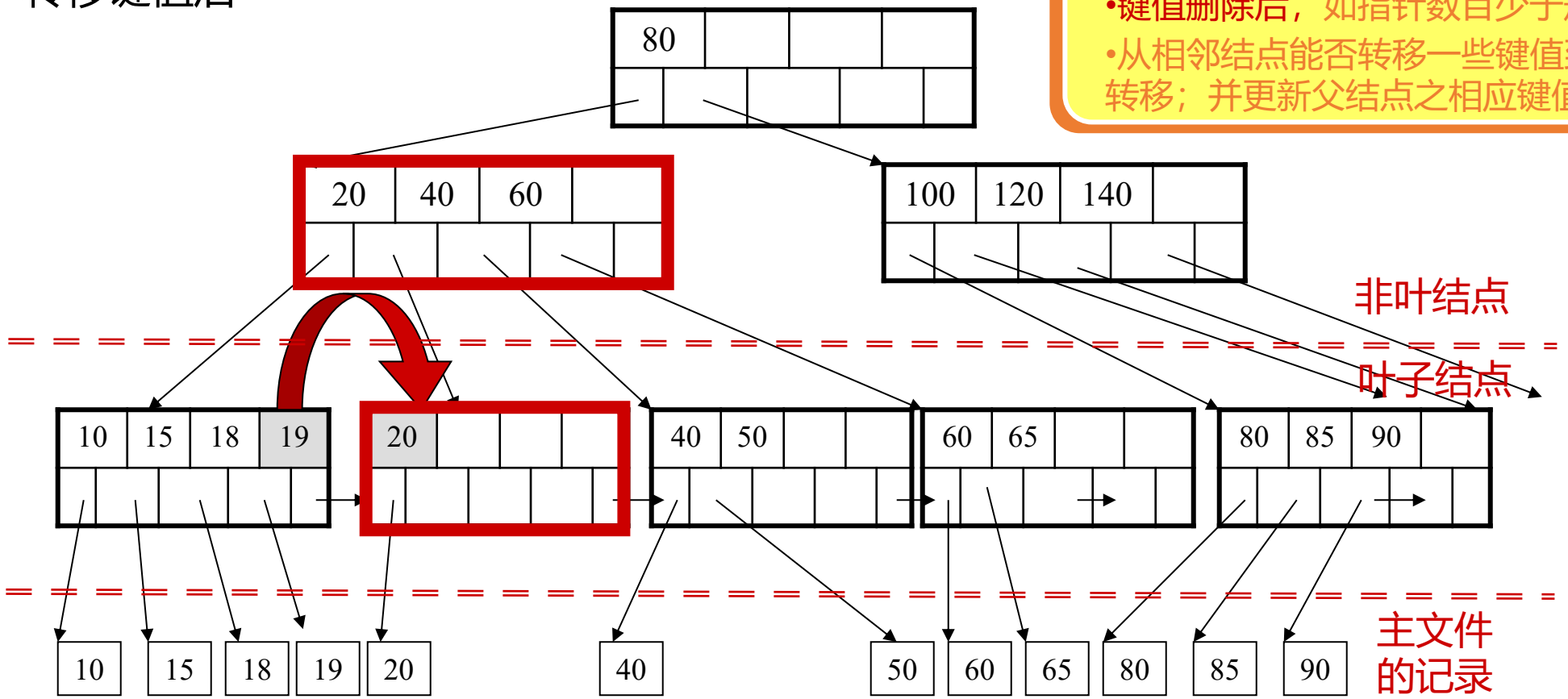
•先定位待删除键值的叶子结点：从根结点开始向下；删除键值及其主文件记录？
•如是第一个，则需调整父结点的键值



6.2 数据库索引-B+树分裂与合并过程再示例

➢(8) 更复杂之删除

转移键值后



•键值删除后，如指针数目少于规定数目，则需要合并

•从相邻结点能否转移一些键值到该结点，如可以，则转移；并更新父结点之相应键值

6.2 数据库索引-B+树分裂与合并过程再示例

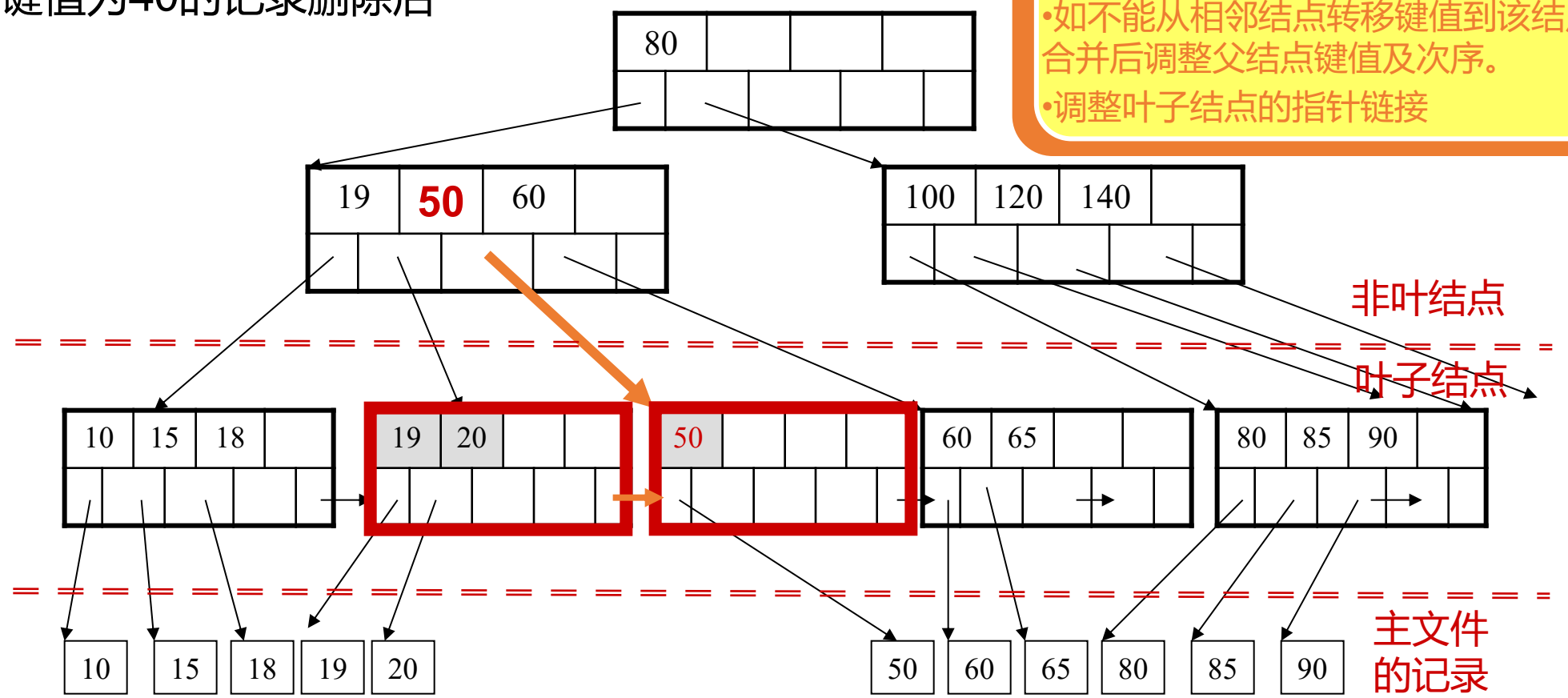
➢(9) 结点之合并

键值为40的记录删除后

•键值删除后，如指针数目少于规定数目，则需要合并

•如不能从相邻结点转移键值到该结点，则考虑结点合并。合并后调整父结点键值及次序。

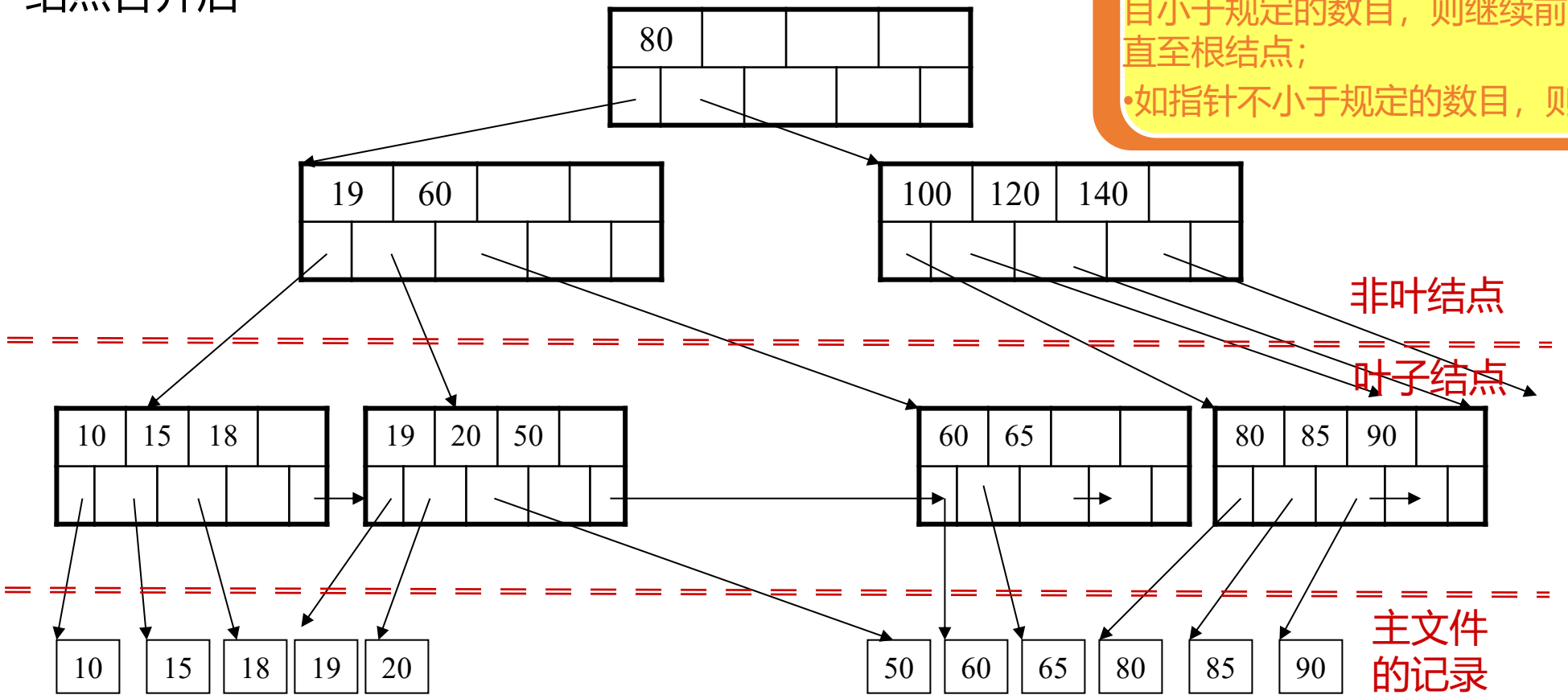
•调整叶子结点的指针链接



6.2 数据库索引-B+树分裂与合并过程再示例

➢(10) 合并之连锁反应

结点合并后



•父结点，如在删除掉索引项及指针后，指针数目小于规定的数目，则继续前面的步骤处理，直至根结点；

•如指针不小于规定的数目，则结束。

6.2 数据库索引-B+树分裂与合并过程再示例

➢(11) 要点概括

插入记录时：

- 可能需要分裂：当索引块充满时，需要分裂。
- 分裂可能引发连锁反应，由叶结点直至根结点。
- 分裂后需要仔细调整索引键值及指针。
- 注意叶子结点之间链接关系的调整

删除记录时：

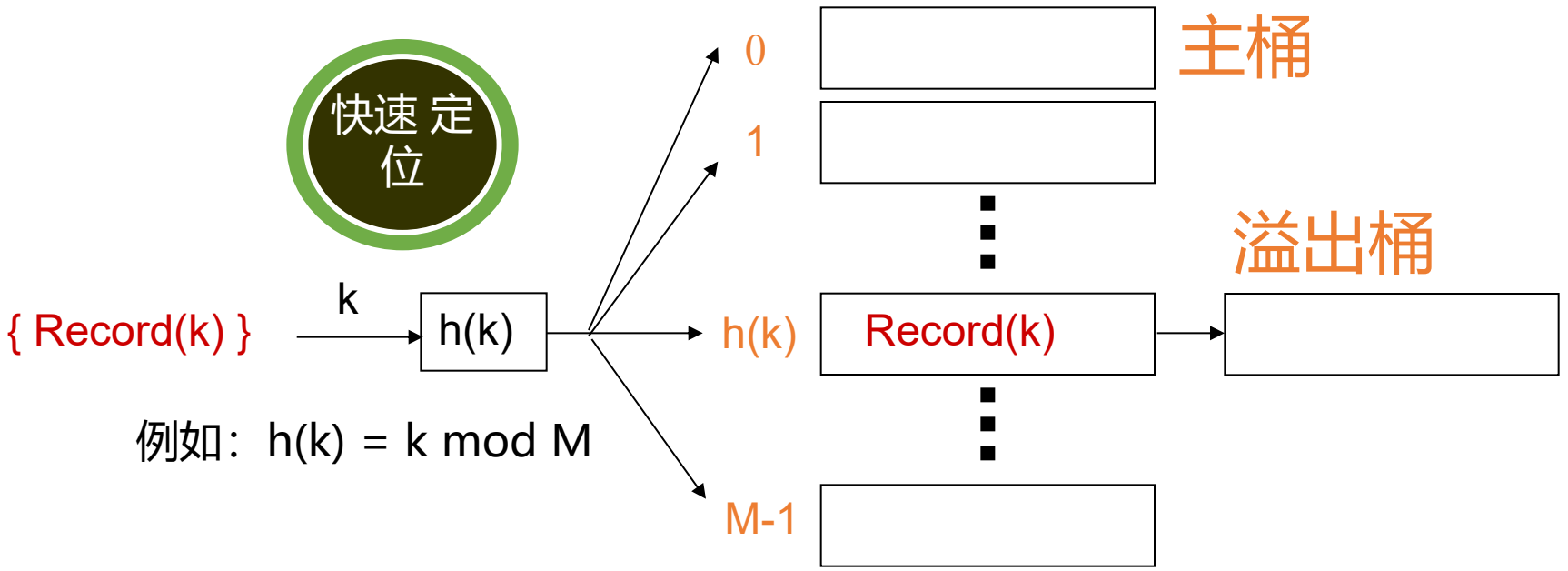
- 可能需要合并：合并记录—从相邻结点中转移索引项（仍旧是两个结点块）
- 可能需要合并：两个结点块合并成一个结点块。
- 合并可能引发连锁反应，由叶结点直至根结点。
- 合并后需要仔细调整索引键值及指针。
- 注意叶子结点之间链接关系的调整

6.2 数据库索引-散列索引

➢(1) 散列的基本概念

散列（参见《数据结构》课程）

- 有M个桶，每个桶是有相同容量的存储地(可以是内存页，也可以是磁盘块)
- 散列函数 h(k)，可以将键值k映射到 { 0, 1, ..., M-1 }中的某一个值
- 将具有键值k的记录Record(k) 存储在对应h(k)编号的桶中



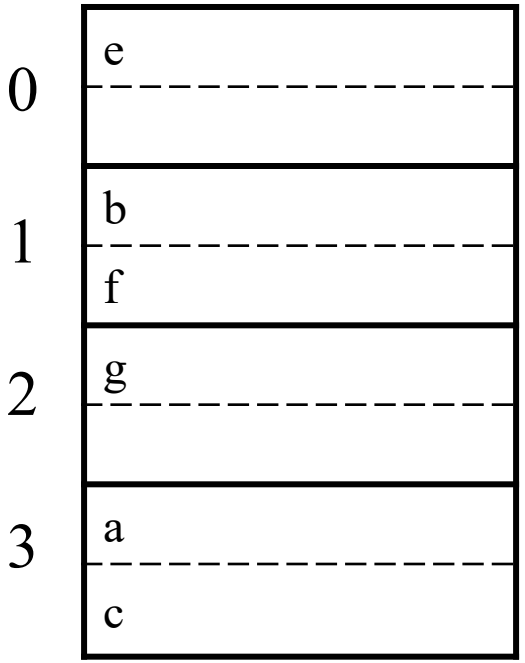
目标：选择一个合适的散列函数，将一个Record集合(每个Record都包含一个关键字k) 均匀地映射到M个桶中。即：对于集合中任一个关键字，经散列函数映射到地址集合中任何一个地址的概率是近乎相等的。

6.2 数据库索引-散列索引

➢(2) 散列索引

散列散列索引

- 内存数据可采用散列确定存储页，主文件可采用散列确定存储块，索引亦可采用散列确定索引项的存储块
- M个桶。一个桶可以是一个存储块，亦可是若干个连续的存储块。



示例：假设 1存储块可存放2个键值及其指针

M=4; 1个桶为 1 个存储块

h(x)满足:

✓h(e)=0 ； h(b)=h(f)=h(s)=1

✓h(g)=2 ； h(a)=h(c)=3

●问：如何查询键值 a的索引项?

✓计算 h(a)=3

✓读取 3号桶，获得键值a的索引项.

需要1个磁盘块读取

6.2 数据库索引-散列索引

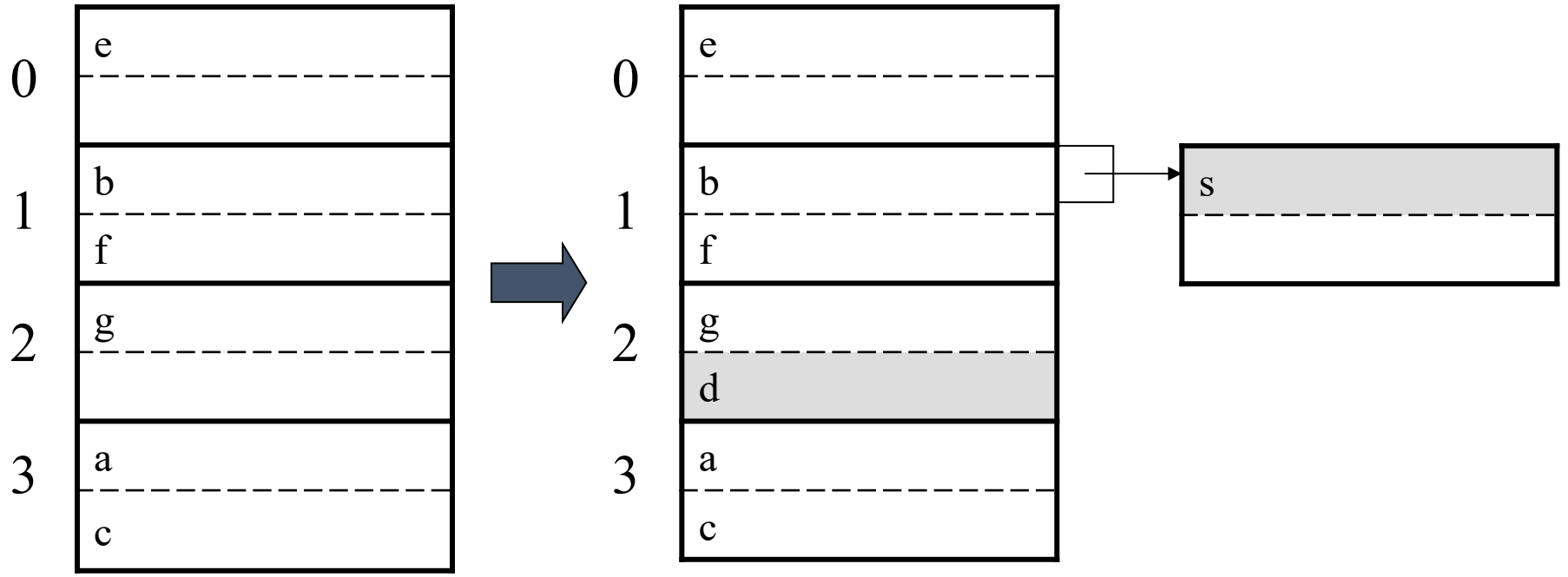
➢(3) 散列索引的插入和删除

问：如何插入键值d的索引项?

- 计算h(d)=2
- 如2号桶有空间，则将索引项d插入2号桶中

问：如何插入键值s的索引项?

- 计算 h(s)=1
- 1号桶无空间，则申请一溢出桶，插入s

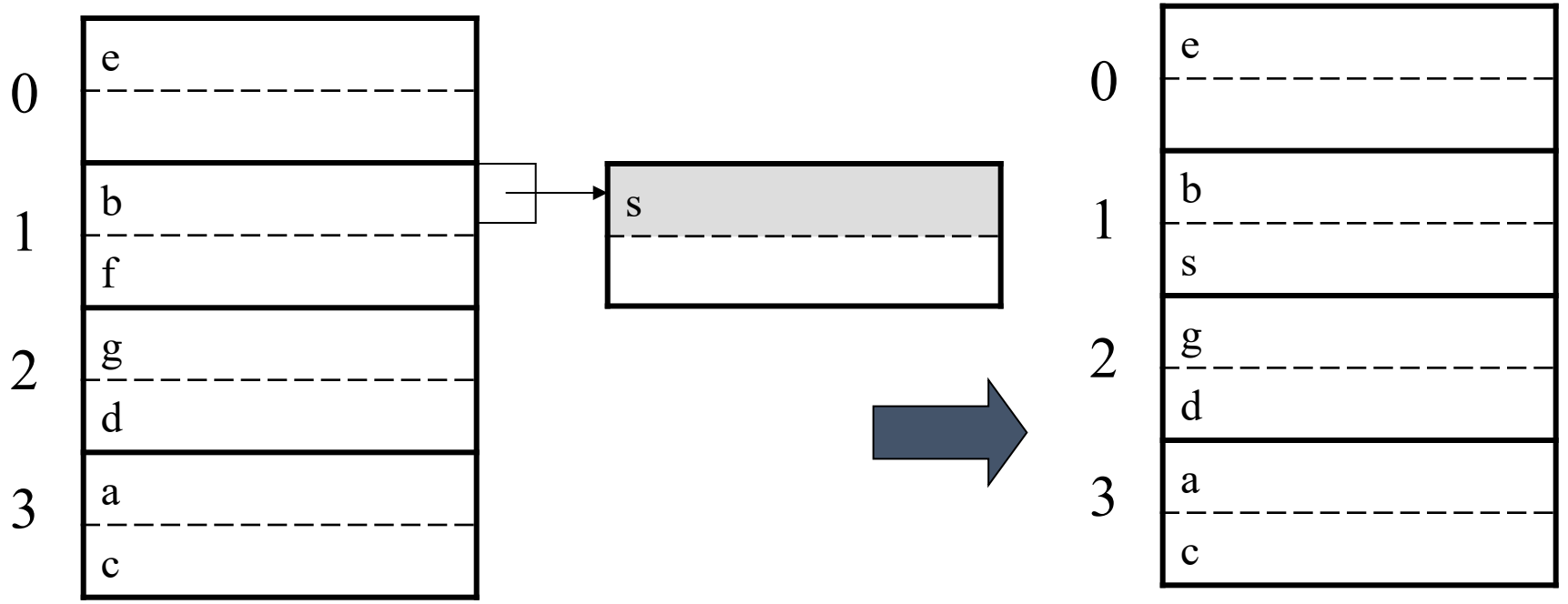


6.2 数据库索引-散列索引

➢(3) 散列索引的插入和删除

问：如何删除键值f.

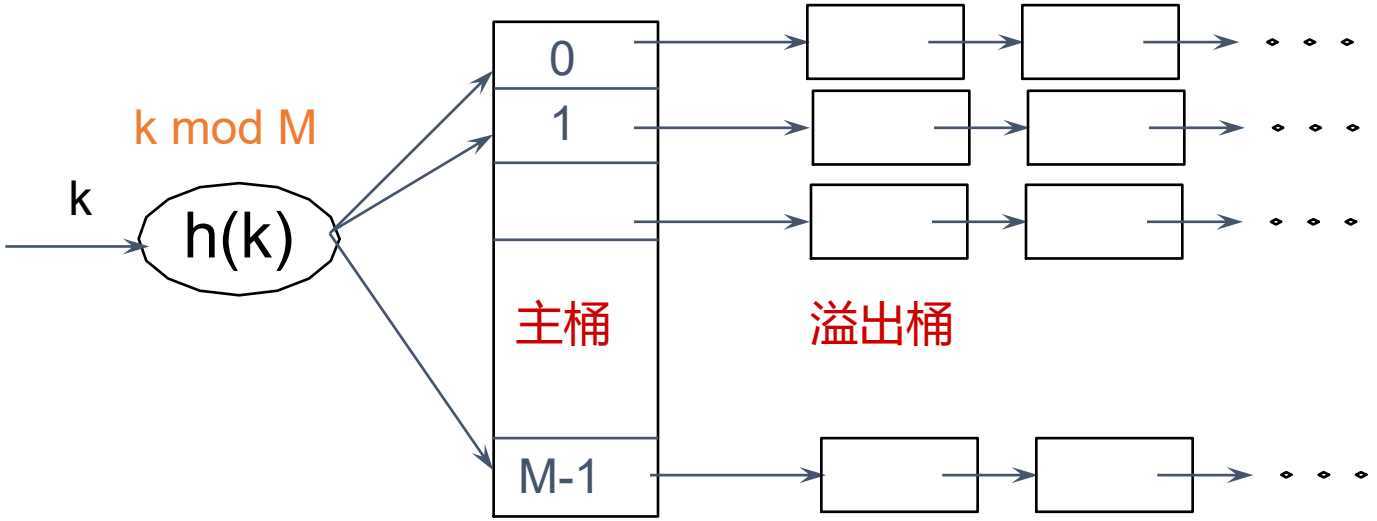
- 计算h(f)=1
- 删除1号桶中的键值f
- 将溢出桶中的s合并到主桶中，删除溢出桶



6.2 数据库索引-散列索引

➢(4) 散列的问题

- 散列索引的目标：最好是没有溢出桶，每一个散列值仅有一个桶。读写每一个键值都只读写一个存储块。
- 均匀分布如何做到？：期望将所有数据分布均匀地存储于M个桶中，使每一个桶的数据成为具有某种特征值h(k)的数据集合。---散列函数的选择。
- 桶的数目M如何确定？：在键值几倍于桶的数目时，每个散列值都可能多于一个桶，形成一个主桶和多个溢出桶的列表，此时需要二次检索：先散列找到主桶号，再依据链表逐一找到每个溢出桶。---桶的数目的确定。



6.2 数据库索引-散列索引

➢(4) 散列的问题

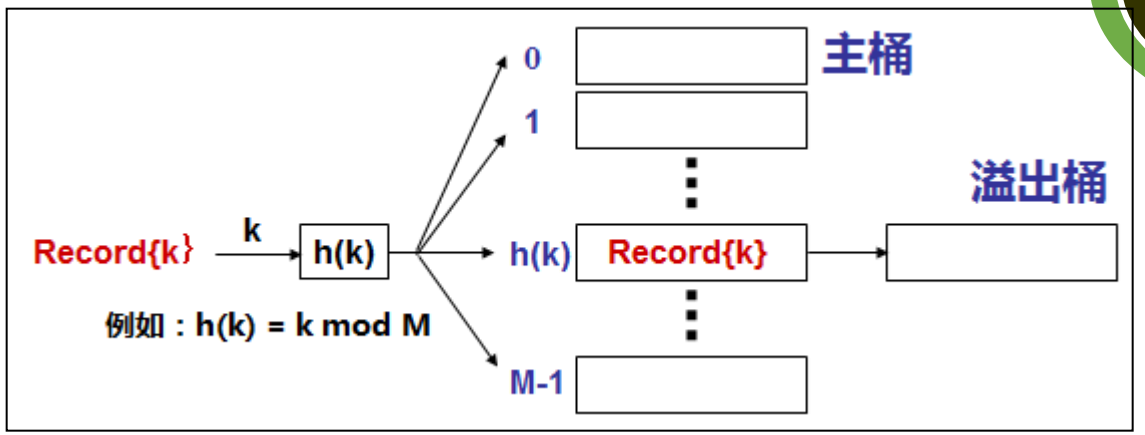
桶的数目M是固定值----静态散列索引

- 如果桶的数目M不变：M过大，则浪费；M过小，将产生更多的溢出桶，增加散列索引检索的时间。

桶的数目随键值增多，动态增加----动态散列索引

- h(k)是和桶的数目M相关的。M的变化会否影响原来存储的内容呢?
- 是否需要将原来已经散列-存储的数据按新的桶数重新进行散列-存储呢?

这个如何处理呢?



6.2 数据库索引-可扩展散列索引

➢(1) 拟解决的问题

桶的数目M是固定值----静态散列索引

- 如果桶的数目M不变：M过大，则浪费；M过小，将产生更多的溢出桶，增加散列索引检索的时间。

桶的数目随键值增多，动态增加----动态散列索引

- h(k)是和桶的数目M相关的。M的变化会否影响原来存储的内容呢?
- 是否需要将原来已经散列-存储的数据按新的桶数重新进行散列-存储呢?

■可扩展散列索引

■线性散列索引

6.2 数据库索引-可扩展散列索引

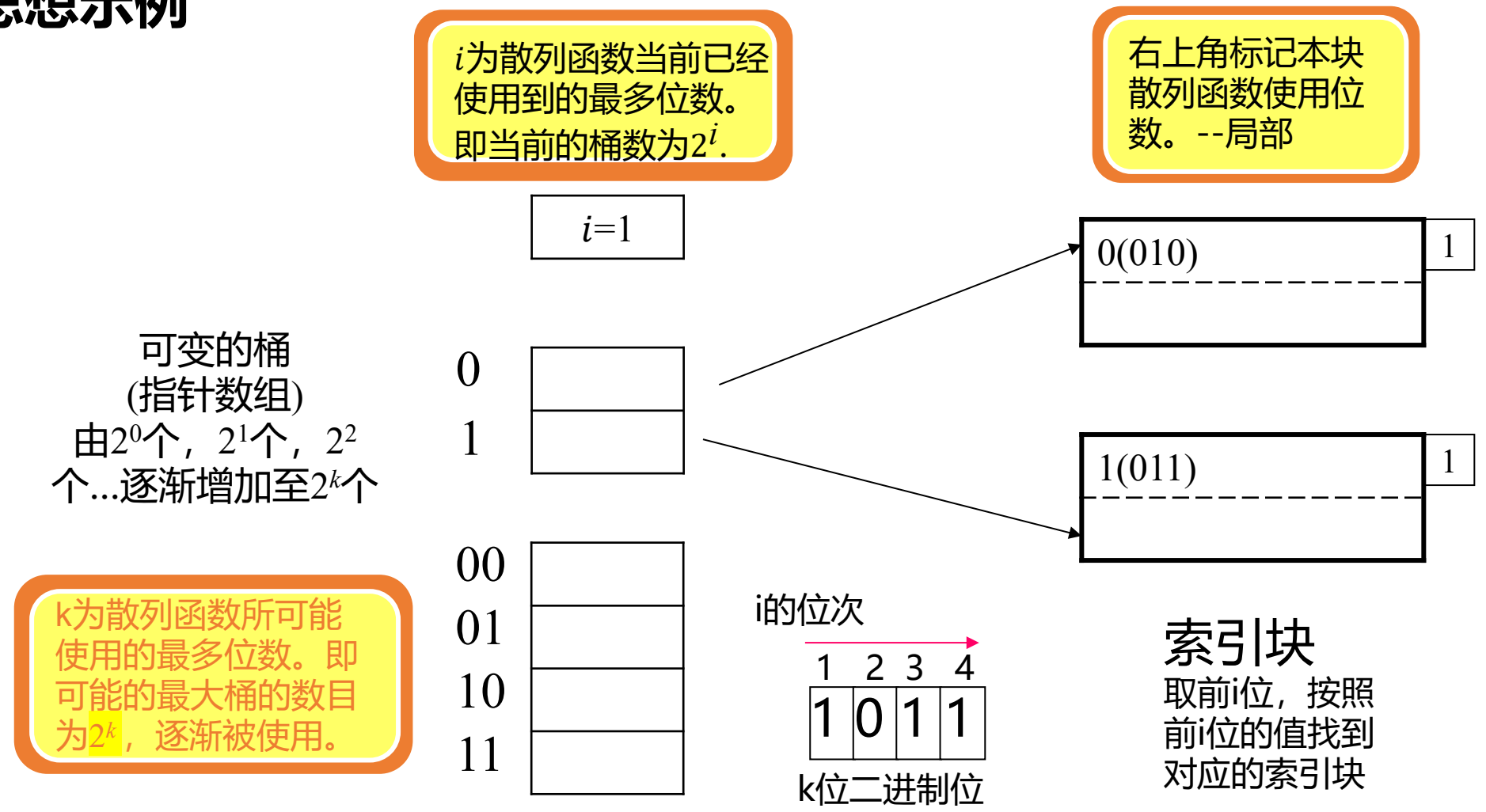
➢(2) 基本思想

- 为桶引入一间接层，即用一个指向块的指针数组来表示桶，而不是用数据块本身组成的数组来表示桶
- 指针数组能增长，其长度总是2的幂。因而数组每增长一次，桶的数目就翻倍。不过，并非每个桶都有一个数据块；如果某些桶中的所有记录可以放在一个块中，则这些桶可能共享一个块。
- 散列函数h为每个键计算出一个K位二进制序列，该K足够大，比如32。但是桶的数目总是使用从序列第一位或最后一位算起的若干位，此位数小于K,比如说i位。也就是说，当i是使用的位数时，桶数组将有2^i个项。

6.2 数据库索引-可扩展散列索引

➢(3) 基本思想示例

- 参数k
- 参数i
- 参数n=2^i



6.2 数据库索引-可扩展散列索引

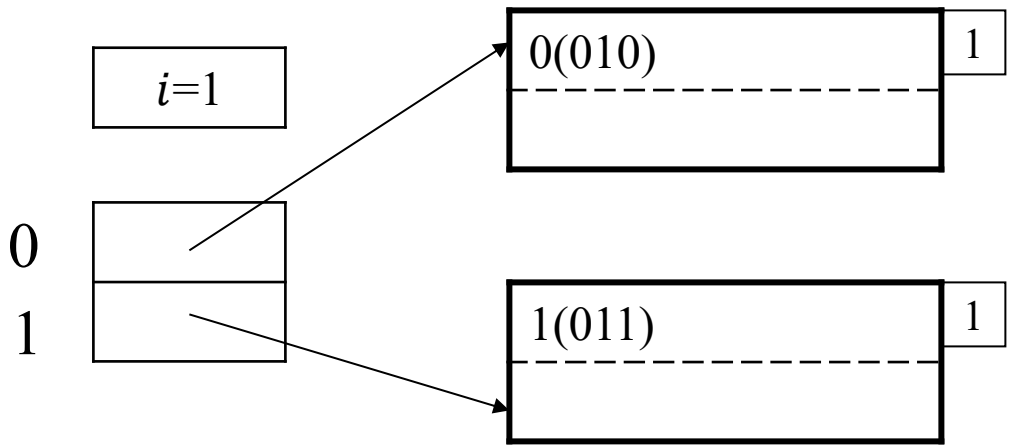
➢(4) 操作示例

插入 1110

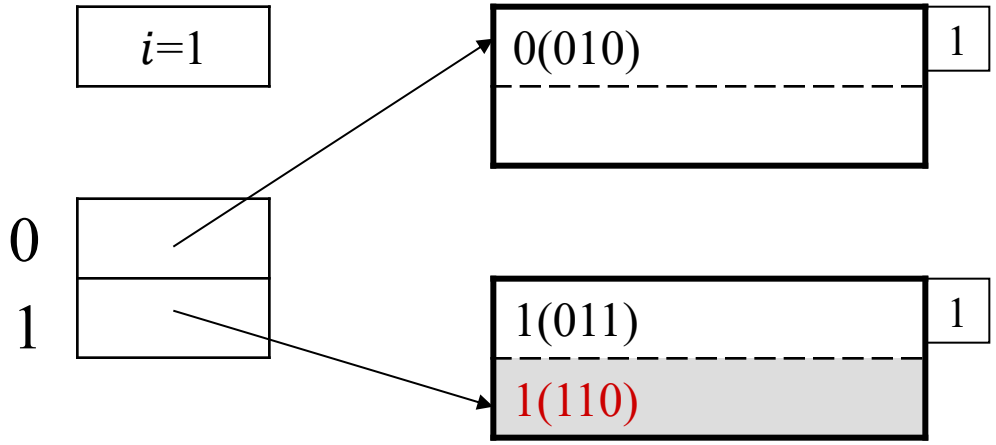
k=4

i =1

n=2^i =2



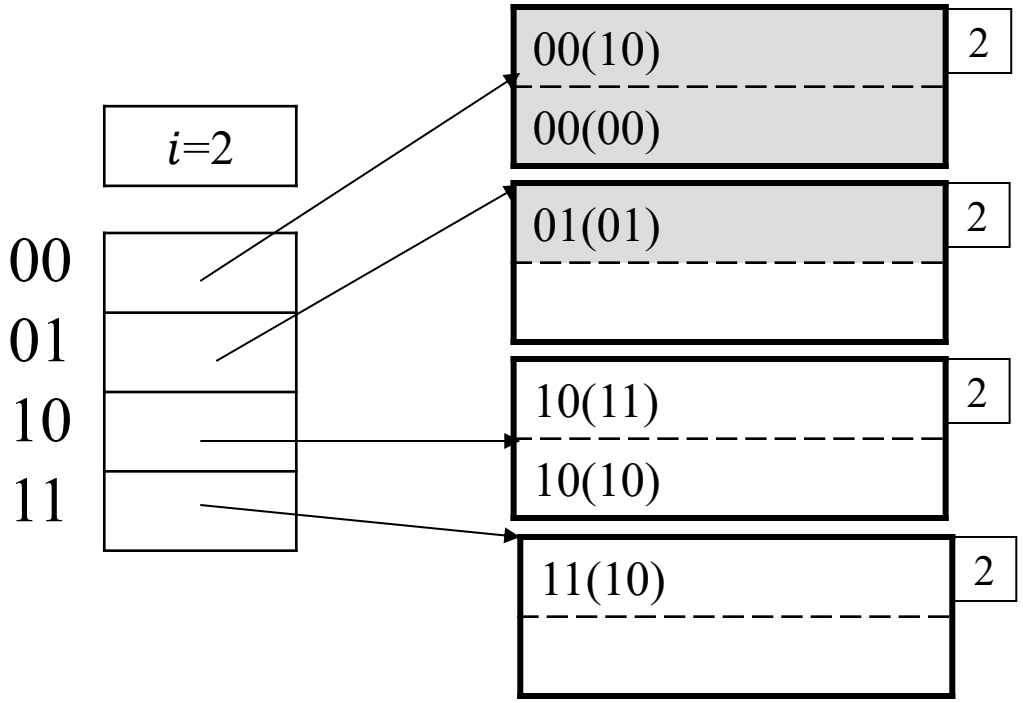
•取前i位，确定索引块。
•如有空间，则存储

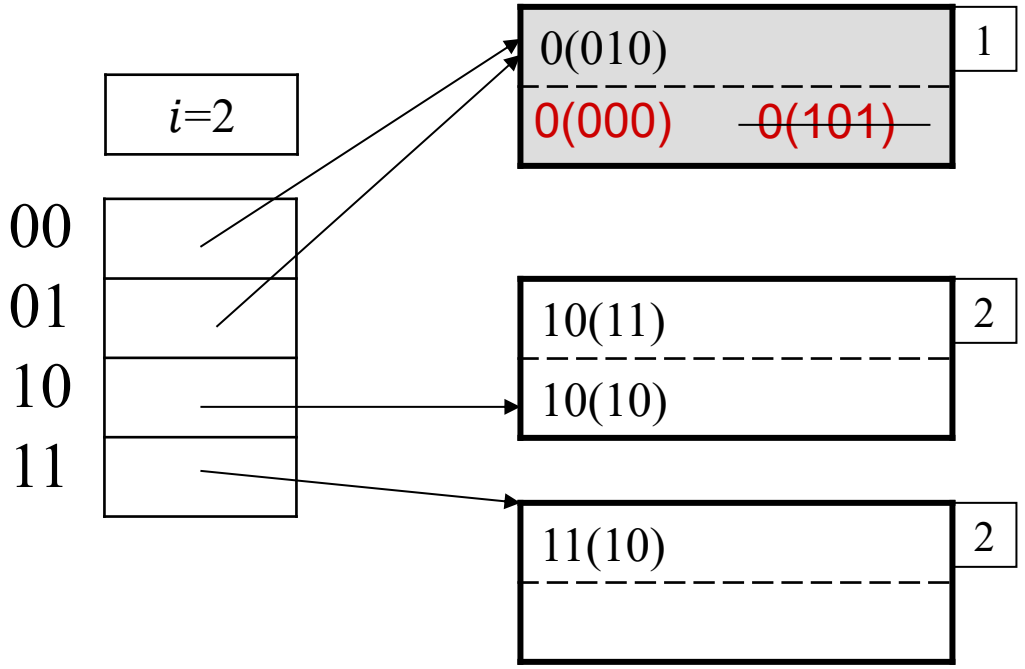


6.2 数据库索引-可扩展散列索引

➢(4) 操作示例

插入0000，再插入0101



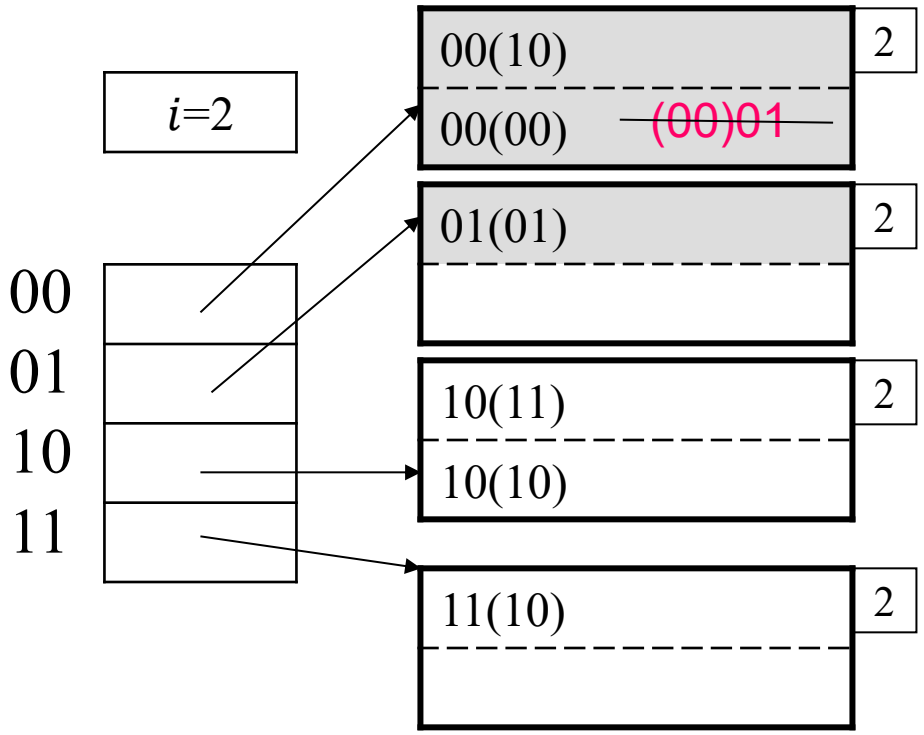


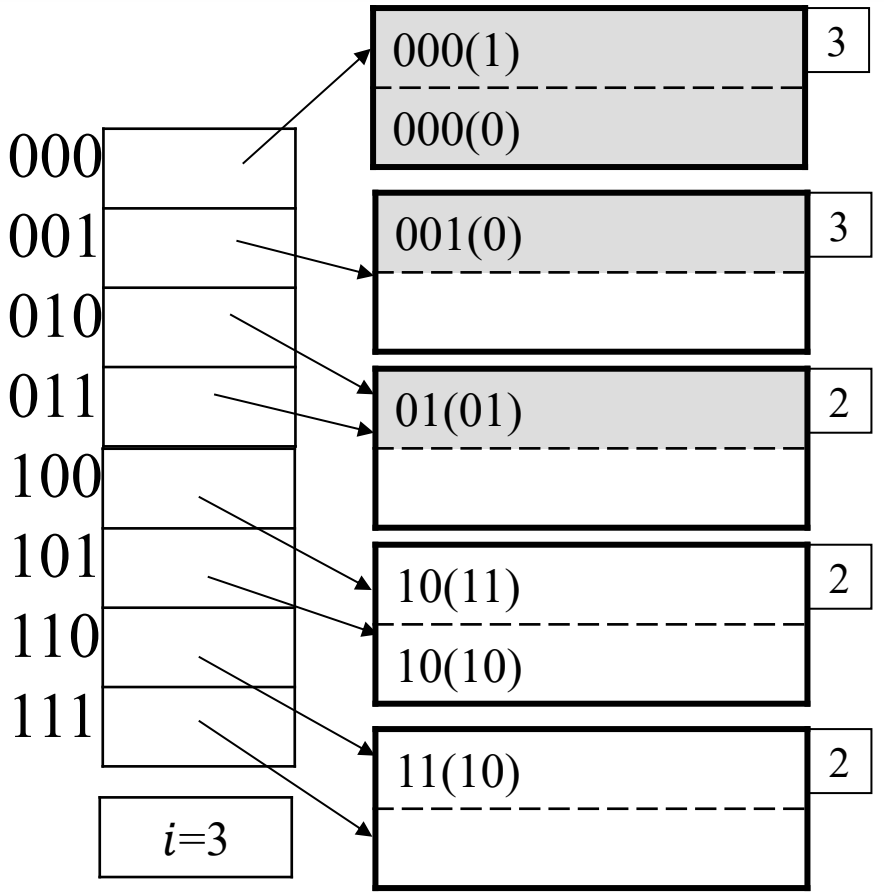
- 取i前位，确定索引块。
- 如未满，则存入。
- 如已满，则需要扩展散列桶，进行分裂：
  - ✓暂时i可能无需增加
  - ✓重新散列该块的数据到两个块中。其他不变

6.2 数据库索引-可扩展散列索引

➢(4) 操作示例

插入 0001





- 取前i位，确定索引块。
- 如未满，则存入。
- 如已满，则需要扩展散列桶，进行分裂：此时i需增加1，重新散列该块的数据到两个块中。其他不变

6.2 数据库索引-可扩展散列索引

➢(4) 存在的问题

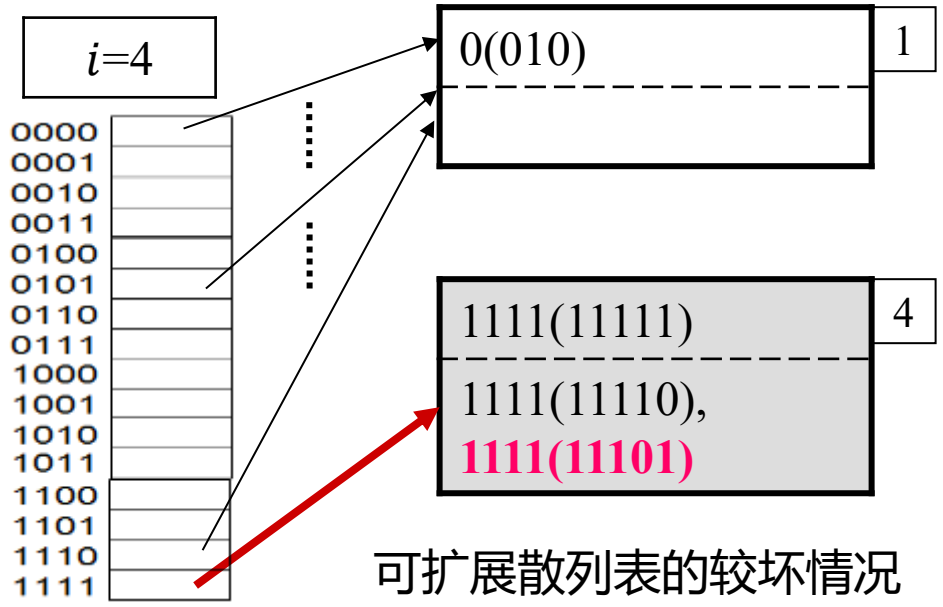
- 当桶数组需要翻倍时，要做大量的工作(当i很大时)；
- 当桶数翻倍后，其在主存中可能就装不下了，或者要占用更大的空间
- 如果每块的记录数很少，那么很有可能某一块的分裂比在逻辑上需要的分裂时间提前很多。例如：块中存放2个记录，即使记录总数远小于220, 但也可能出现三个记录的前20位二进制位序列一样，在这种情况下，将不得不使用i=20和100万个桶数组。

i=1时3个数组在一个桶中；要分裂；则2个桶

i=2时三个数组仍旧在一个桶中；要分裂，则2^2个桶

i=3时三个数组仍旧在一个桶中；要分裂，则2^3个桶

… …



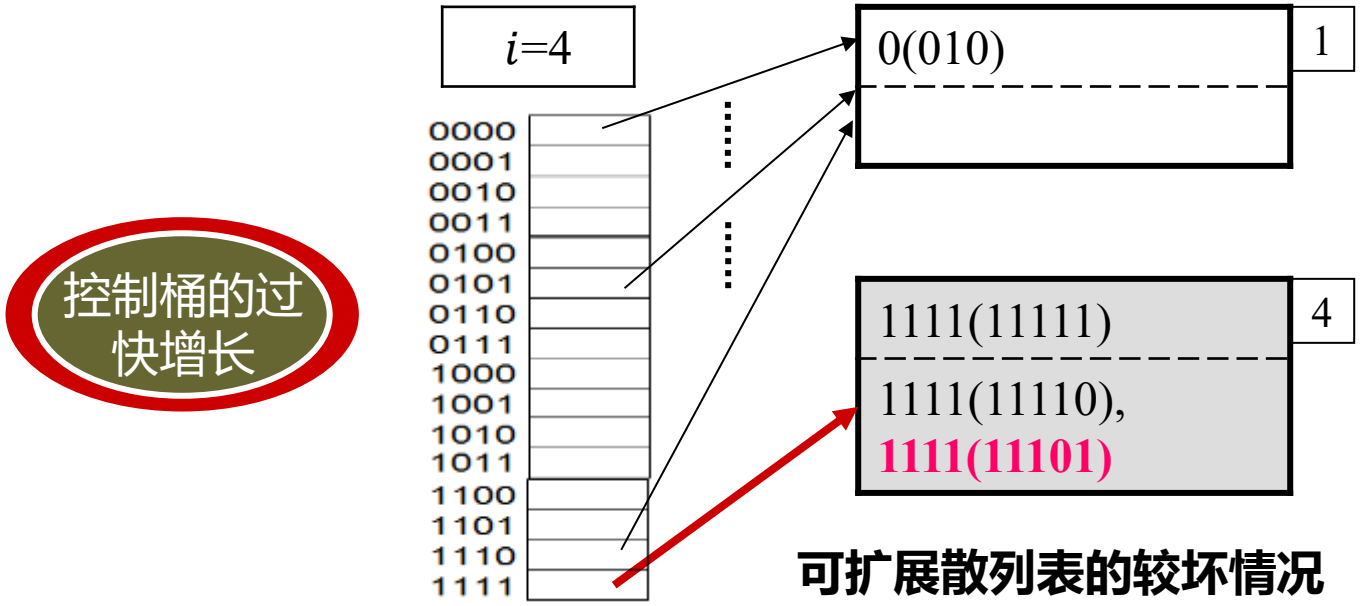
可扩展散列表的较坏情况

如何解决?

6.2 数据库索引-线性散列索引

➢(1) 拟解决的问题

- 桶的数目过快增长，但其利用率不足
- 例如：块中存放2个记录，即使记录总数远小于220, 但也可能出现三个记录的前20位二进制位序列一样，在这种情况下，将不得不使用i=20和100万个桶数组。但大部分桶都空闲未用。



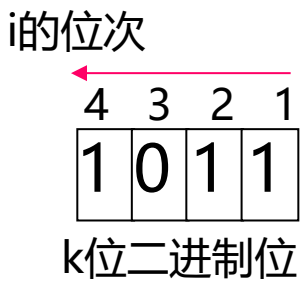
可扩展散列表的较坏情况

6.2 数据库索引-线性散列索引

➢(2) 基本思想

线性散列索引

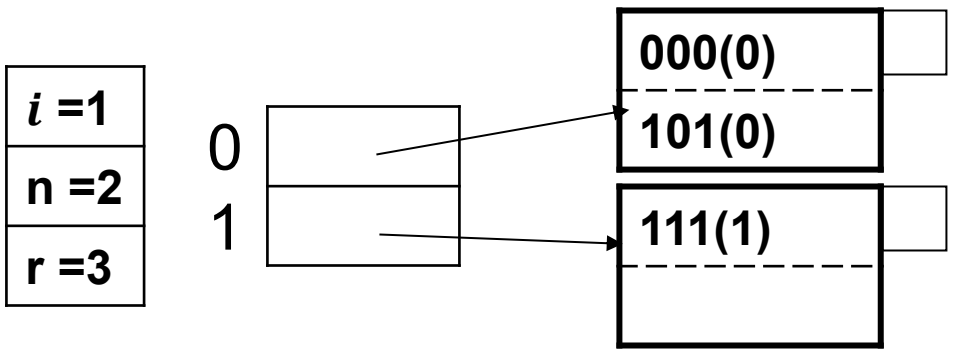
- 桶数n的选择总是使存储块的平均记录数保持与存储块所能容纳的记录总数成一个固定的比例，例如80%。超过此比例，则桶数增长1块，分裂。 ---线性增长，每次增1。
- 存储块并不总是可以分裂，所以允许有溢出块，尽管每个桶的平均溢出块数远小于1。
- 用来做桶数组项序号的二进制位数是⌈log2n⌉，其中n是当前的桶数。这些位总是从散列函数得到的位序列的右端(即低位)开始取。



6.2 数据库索引-线性散列索引

➢(2) 基本思想

- 假定散列函数值的i位正在用来给桶数组项编号，且有一个键值为K的记录要插入到编号为$a_1a_2\ldots a_i$的桶中，即$a_1a_2\ldots a_i$是h(K)的后i位。把$a_1a_2\ldots a_i$当作二进制整数，设它为m。
  - ✓ 如果m<n，则编号为m的桶存在并把记录存入该桶中。
  - ✓ 如果n≤m<2^i，那么桶还不存在，因此我们把记录存入桶m-2^{i-1}，也就是当我们把a_1(它肯定是1)改为0时对应的桶。

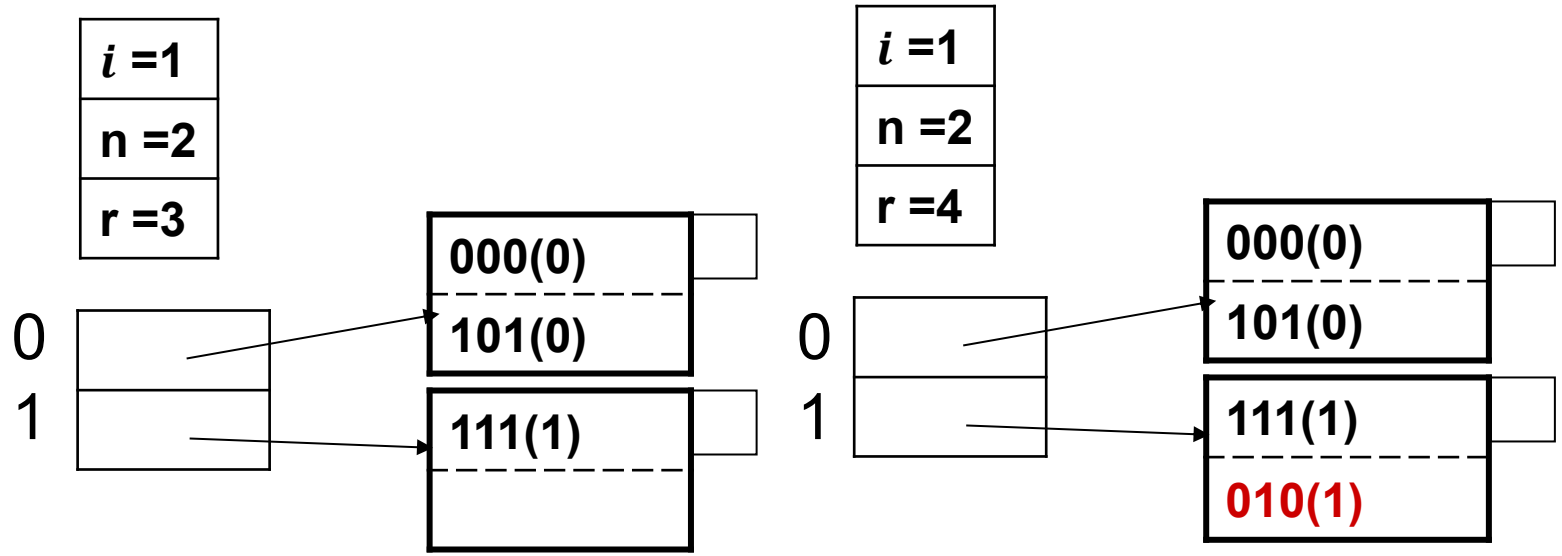


i-当前使用的散列函数的位数
n-当前的桶数
r-当前散列表中的记录总数
要求 r<=1.7n

6.2 数据库索引-线性散列索引

➢(3) 操作示例

- 插入 0101的记录,如右图。
- 此时r/n=4/2=2>1.7, 桶数需要增1, 即要增加一桶,n=3
- 需要分裂。10桶要由00桶分裂而来(低位相同而最高位不同的那一个桶)



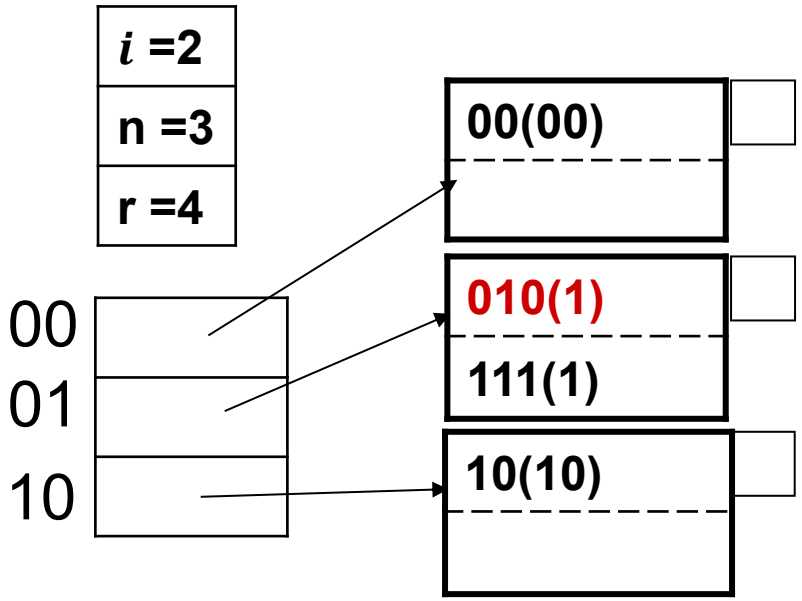
i -当前使用的散列函数的位数
n-当前的桶数
r-当前散列表中的记录总数
要求 r<=1.7n

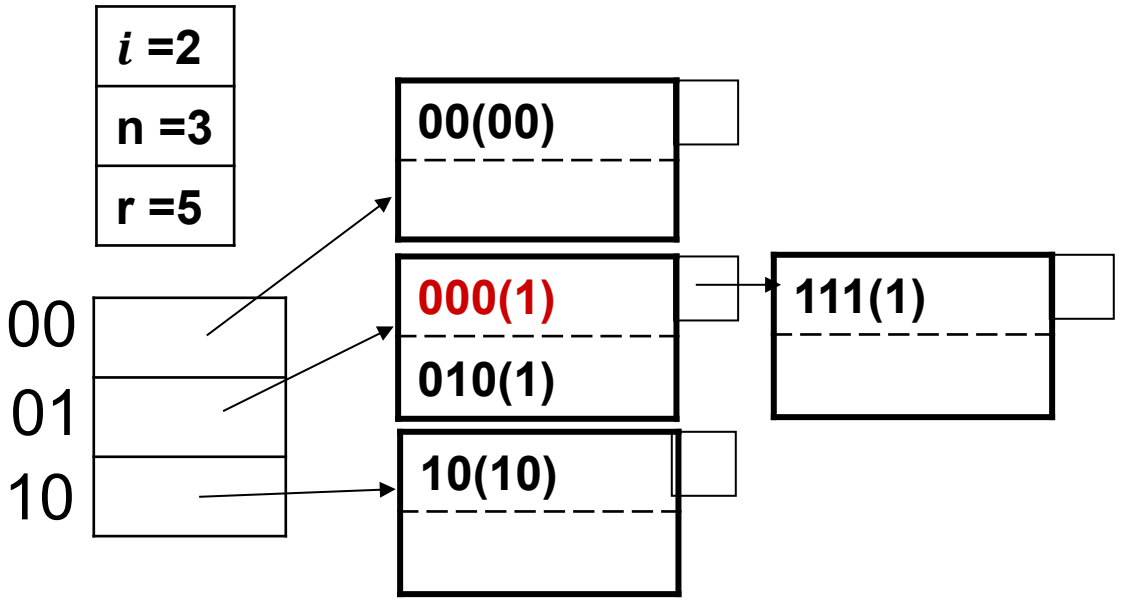
6.2 数据库索引-线性散列索引

➢(3) 操作示例

- 插入 0001的记录,如右图。
- 此时r/n=5/3=1.66<1.7, 桶数无需增加

i-当前使用的散列函数的位数
n-当前的桶数
r-当前散列表中的记录总数
要求 r<=1.7n



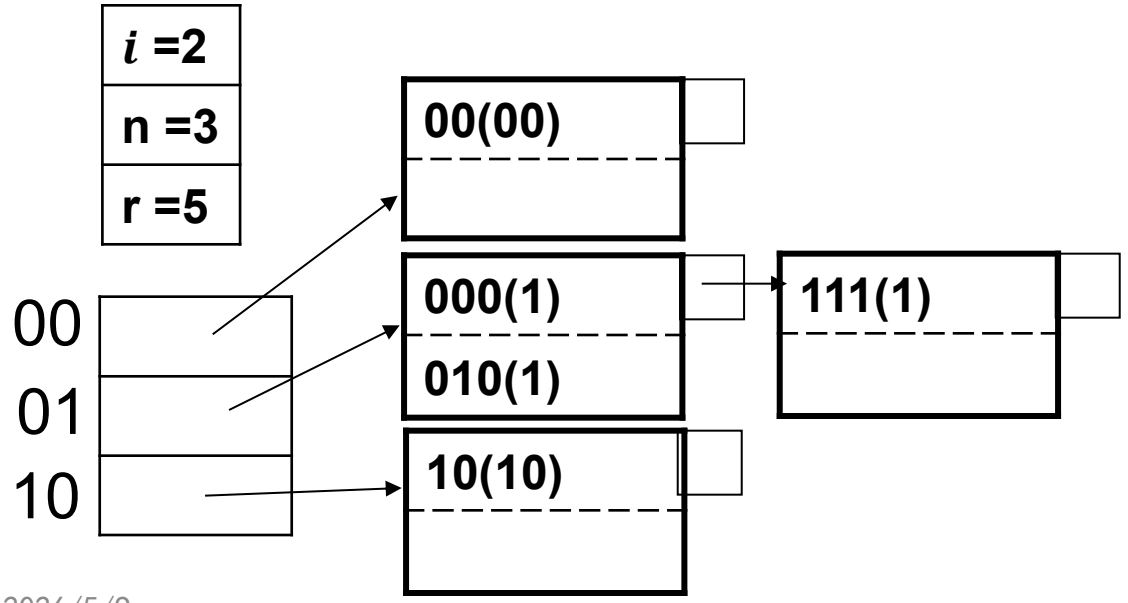


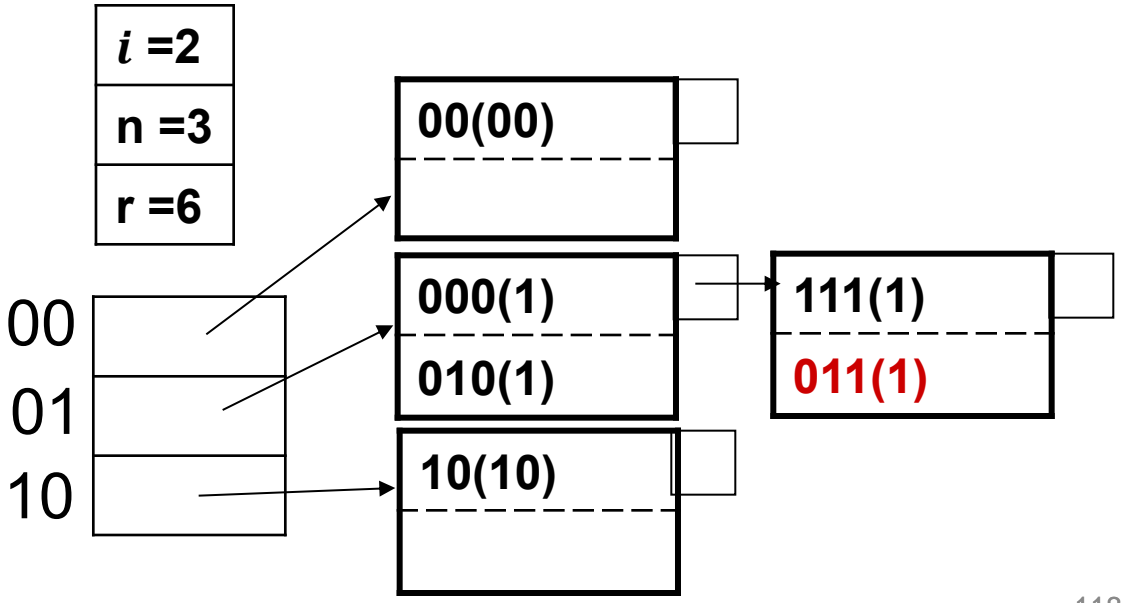
6.2 数据库索引-线性散列索引

i-当前使用的散列函数的位数
n-当前的桶数
i-当前散列表中的记录总数
要求 r<=1.7n

➢(3) 操作示例

- 插入 0111的记录,如右图。
- 因没有11桶，则插入01桶。
- 此时r/n=6/3=2>1.7, 桶数需要增1, 即要增加一桶11, n仍然为3
- 需要分裂。11要由01桶分裂而来(低位相同而最高位不同的那一个桶)

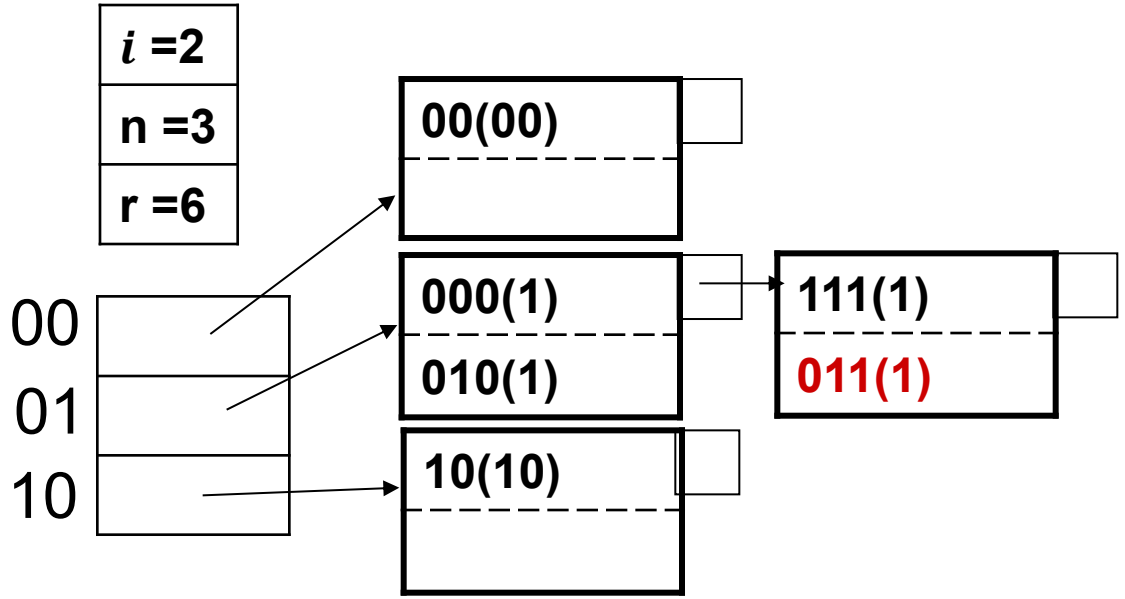




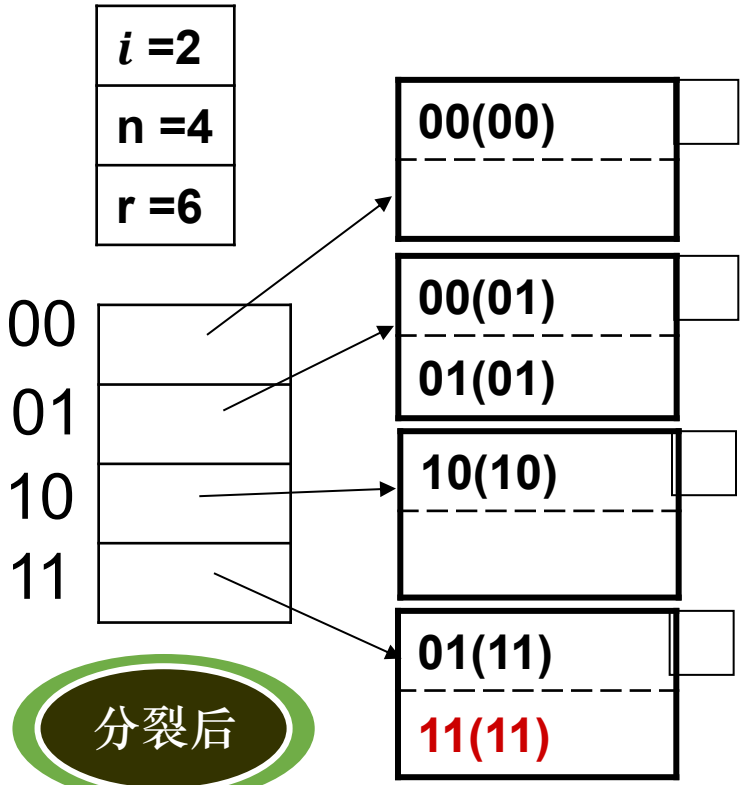
6.2 数据库索引-线性散列索引

➢(3) 操作示例

- *i* -当前使用的散列函数的位数
- n-当前的桶数
- r-当前散列表中的记录总数，要求 r<=1.7n



分裂前



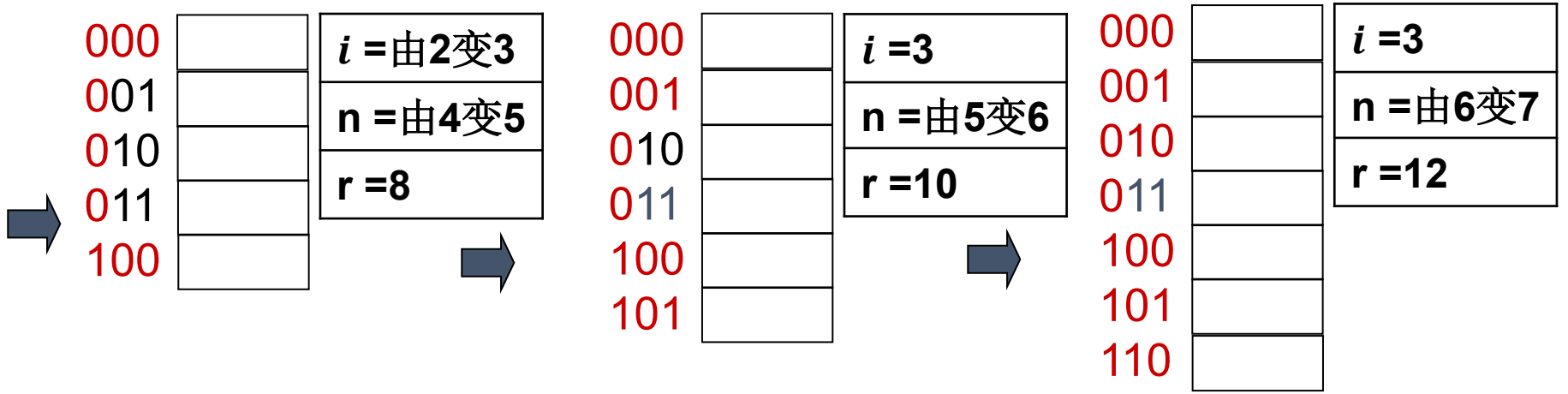
分裂后

6.2 数据库索引-线性散列索引

➢(4) 回顾操作特点



- 键值由低位向高位次序使用。对应索引项存储在由桶数组指针指向的存储块
- 当r/n(当前索引项数除以桶数)大于一定比例时，按次序增加1个桶
- 假设该桶为1a2a3…ai，则该桶由0a2a3..ai对应的块分裂而来
- 当桶数超过2^i个桶时，则使i增1。



6.2 回顾本讲学了什么

➢总结

